



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Visión artificial aplicada a partidos de fútbol utilizando el dataset SoccerNet

Realizado por
Carlos Galea Magro

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
Manuel Carranza García

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de octubre, curso 2025/26

Agradecimientos

Este proyecto significa mucho para mi y pone fin a una etapa de mi vida. Este marca el fin de mis estudios de grado. Entonces me gustaria empezar a dar las gracias a todos los que han hecho posible llegar hasta este punto.

Empezando por mi familia, mis padres y mi hermano siempre estuvieron ahi, me aconsejaron y recibí su apoyo en todo momento durante esta etapa además de demostrarme la importancia de tener un buen equilibrio entre la vida personal y profesional. A mis abuelos y mi tia que asi como mis padres me apoyaron y me dieron la educación y la importancia de ser constante para conseguir lo que quieres.

A mis amigos de la universidad, con los que he compartido buenos momentos y han hecho que esta etapa sea amena y divertida a la vez que un reto. Muchos momentos fueron mas facil durante la carrera gracias a ellos ya que nos ayudábamos mutuamente.

A mi experiencia de Erasmus este último año dónde conocí muchos amigos que me han apoyado pero sobretodo esta experiencia me ha ayudado a crecer como persona tanto por las amistades como por mi mismo me hizo cambiar mi forma de ver el mundo y abrir mi mente.

También quiero agradecer a Manuel Carranza que ha sido quién me ha ayudado y guiado en este proyecto por su dedicación haciendolo posible.

Resumen

El seguimiento multiobjeto (*multi-object tracking*, MOT) de jugadores en vídeo de fútbol es un paso previo necesario para cualquier análisis automático del juego como mapas de calor, distancia recorrida, posesión de balón, pero elegir un algoritmo concreto para esa tarea exige compararlo sobre datos reales de fútbol y no solo sobre los benchmarks genéricos de seguimiento de personas para los que se diseñó originalmente. Este Trabajo de Fin de Grado aborda esa comparación sobre SoccerNet-Tracking, un *dataset* público de vídeo de fútbol con detecciones precomputadas, y propone un sistema completo que va desde el seguimiento hasta la extracción de metadatos deportivos interpretables a partir de las trayectorias resultantes.

El sistema compara dos algoritmos de asociación clásicos, ByteTrack y OC-SORT, evaluados con las métricas estándar de TrackEval (HOTA, MOTA, IDF1) sobre los conjuntos de entrenamiento y de test. Sobre la configuración óptima de cada algoritmo, obtenida mediante un estudio sistemático de hiperparámetros, se construye un segundo pipeline de extracción de metadatos: clasificación de equipos por color de camiseta mediante K-means, mapas de calor de posiciones, distancia y velocidad reales a partir de una homografía calibrada manualmente, y posesión de balón por proximidad. Los resultados, junto con vídeos anotados, se exponen finalmente en una aplicación web.

El conjunto de datos procesado reúne 106 secuencias de vídeo (57 de entrenamiento, 49 de test) de 700 fotogramas cada una y con detecciones precomputadas de jugadores, árbitros y balón. Con la métrica HOTA(0) (evaluada en un único umbral), ByteTrack alcanza 81,43 en test frente a 80,92 de OC-SORT; sin embargo, con la métrica HOTA integrada estándar el resultado se invierte con claridad en ambos conjuntos (80,23 frente a 73,27 en entrenamiento; 75,43 frente a 70,73 en test), porque OC-SORT reutiliza las detecciones del *dataset* sin modificarlas mientras que ByteTrack sí las interpola en su segunda ronda de asociación. Esta discrepancia, documentada y explicada en detalle en la memoria, es uno de los resultados más relevantes del trabajo: dos formas igual de legítimas de medir el mismo sistema pueden llevar a conclusiones opuestas sobre qué algoritmo es mejor.

Palabras clave: SoccerNet, Seguimiento Multiobjeto, Visión Artificial, ByteTrack, OC-SORT, HOTA, Análisis Deportivo.

Abstract

Multi-object tracking (MOT) of players in football video is a necessary prerequisite for any automatic analysis of the game, such as heatmaps, distance covered or ball possession, but choosing a specific algorithm for that task requires comparing it on real football data rather than only on the generic person-tracking benchmarks it was originally designed for. This Final Degree Project addresses that comparison on SoccerNet-Tracking, a public football video dataset with precomputed detections, and proposes a complete system that goes from tracking to the extraction of interpretable sports metadata from the resulting trajectories.

The system compares two classic association algorithms, ByteTrack and OC-SORT, evaluated with the standard TrackEval metrics (HOTA, MOTA, IDF1) on the training and test sets. On top of the optimal configuration of each algorithm, obtained through a systematic hyperparameter study, a second metadata extraction pipeline is built: team classification by jersey colour using K-means, position heatmaps, real distance and speed from a manually calibrated homography, and ball possession by proximity. The results, together with annotated videos, are finally presented in a web application.

The processed dataset gathers 106 video sequences (57 training, 49 test) of 700 frames each, with precomputed detections of players, referees and the ball. Using the HOTA(0) metric (evaluated at a single overlap threshold), ByteTrack reaches 81.43 on test versus 80.92 for OC-SORT; however, under the standard integrated HOTA metric the result is clearly reversed on both sets (80.23 versus 73.27 on training; 75.43 versus 70.73 on test), because OC-SORT reuses the dataset detections unmodified while ByteTrack interpolates them in its second association round. This discrepancy, documented and explained in detail in the report, is one of the most relevant results of the project: two equally legitimate ways of measuring the same system can lead to opposite conclusions about which algorithm is better.

Keywords: SoccerNet, Multi-Object Tracking, Computer Vision, ByteTrack, OC-SORT, HOTA, Sports Analytics.

Índice general

1	Introducción	1
1.1.	Motivación	1
1.2.	Contexto	1
1.3.	El problema	2
1.4.	Objetivos	2
1.4.1.	Objetivo general	2
1.4.2.	Objetivos específicos	2
1.5.	Alcance y delimitación del proyecto	3
1.6.	Estructura del documento	4
2	Fundamentos teóricos y estado del arte	6
2.1.	Inteligencia artificial, aprendizaje automático y visión por computador	6
2.2.	Datasets y el ecosistema SoccerNet	7
2.2.1.	SoccerNet-Tracking	8
2.3.	El problema del seguimiento multiobjeto	9
2.3.1.	Retos específicos del seguimiento en fútbol	10
2.4.	Fundamentos de <i>tracking-by-detection</i>	11
2.4.1.	El filtro de Kalman	11
2.4.2.	Solapamiento entre cajas: IoU	12
2.4.3.	El problema de asignación y el algoritmo húngaro	12
2.5.	Algoritmos de tracking comparados	13
2.5.1.	SORT: el punto de partida	13
2.5.2.	OC-SORT: SORT centrado en la observación	14
2.5.3.	ByteTrack: aprovechar todas las detecciones	14
2.5.4.	Trackers con re-identificación visual: considerados y no implementados	16
2.6.	Métricas de evaluación de sistemas de tracking	16
2.6.1.	CLEAR MOT: MOTA y MOTP	17
2.6.2.	IDF1	17
2.6.3.	HOTA: la métrica principal de este proyecto	17
2.7.	Detección del balón	18
2.8.	Clasificación no supervisada de jugadores por equipo: K-means y el árbitro	19
2.9.	Homografía y proyección de coordenadas	20
2.10.	Trabajos relacionados y posicionamiento de este TFG	22
3	Gestión del proyecto	24
3.1.	Metodología de trabajo	24
3.2.	Fases del proyecto	25
3.3.	Roles y participantes	27
3.4.	Estructura de descomposición del trabajo (EDT)	28
3.5.	Planificación temporal	30
3.5.1.	Elaboración del plan de trabajo	30

3.5.2.	Seguimiento y monitorización	31
3.5.3.	Hitos principales	32
3.5.4.	Revisión de la planificación	32
3.6.	Entorno de desarrollo	36
3.7.	Estimación de recursos y presupuesto	36
3.7.1.	Costes de personal	36
3.7.2.	Costes materiales y operacionales	37
3.7.3.	Desviación respecto al coste planificado	37
3.8.	Plan de gestión de riesgos	39
3.9.	Plan de calidad	41
4	Análisis de requisitos	43
4.1.	Contexto funcional del sistema	43
4.1.1.	Justificación y decisión de la tarea elegida	43
4.2.	Actores y usuarios del sistema	44
4.3.	Requisitos de información	45
4.4.	Requisitos funcionales	49
4.5.	Requisitos no funcionales	54
4.6.	Reglas de negocio	58
4.7.	Casos de uso	60
4.8.	Matriz de trazabilidad	66
5	Diseño	68
5.1.	Arquitectura general	68
5.2.	Diseño del pipeline de tracking	68
5.2.1.	Diseño del filtrado del balón	69
5.2.2.	Diseño de los algoritmos de tracking	69
5.2.3.	Diseño de la ejecución batch sobre un split completo	70
5.3.	Diseño de la evaluación	70
5.4.	Diseño del estudio de hiperparámetros	72
5.5.	Diseño del pipeline de metadatos	72
5.5.1.	Diseño de la clasificación de equipos	72
5.5.2.	Diseño de los mapas de calor	73
5.5.3.	Diseño de la homografía y la estimación de distancia/velocidad	74
5.5.4.	Diseño de la estimación de posesión	75
5.6.	Diseño de datos y artefactos	77
5.7.	Diseño de la aplicación web	77
5.7.1.	Arquitectura de la aplicación web	78
5.7.2.	Diseño de la interfaz de usuario	78
6	Implementación	80
6.1.	Entorno y herramientas	80
6.1.1.	Traslados de equipo de desarrollo	81
6.1.2.	Problemas de entorno encontrados	82
6.2.	Estructura del repositorio	84
6.3.	Implementación del tracking	84
6.3.1.	Scripts de visualización inicial	84

6.3.2.	Filtrado geométrico del balón	85
6.3.3.	ByteTrack y OC-SORT	85
6.3.4.	Ejecución batch y reorganización de resultados	85
6.4.	Implementación de la evaluación	86
6.4.1.	Aclaración metodológica: HOTA frente a HOTA(0)	86
6.5.	Implementación del estudio de hiperparámetros	87
6.6.	Implementación del pipeline de metadatos	88
6.6.1.	Clasificación de equipos	88
6.6.2.	Mapas de calor	89
6.6.3.	Homografía real: distancia y velocidad	89
6.6.4.	Posesión de balón	90
6.7.	Implementación de la aplicación web	90
6.7.1.	Generación de metadatos para OC-SORT	90
6.7.2.	Empaquetado de datos estáticos para la web	91
6.7.3.	Frontend	91
6.8.	Enfoques explorados y descartados	92
6.8.1.	Detección del balón con YOLO	92
6.8.2.	Clasificación de equipos con un tercer grupo para el árbitro	92
6.8.3.	Homografía generalizada a toda la secuencia	93
7	Resultados	94
7.1.	Descripción de experimentos	94
7.2.	Estrategia de validación	94
7.3.	Comparativa de algoritmos: ByteTrack frente a OC-SORT	94
7.3.1.	Por qué HOTA y HOTA(0) son distintas	95
7.4.	Resultados del estudio de hiperparámetros	96
7.5.	Resultados del pipeline de metadatos	97
7.5.1.	Clasificación de equipos	97
7.5.2.	Mapas de calor	98
7.5.3.	Distancia y velocidad por homografía	99
7.5.4.	Posesión de balón	101
7.6.	Limitaciones de los resultados	102
8	Manual de despliegue	103
8.1.	Requisitos previos	103
8.2.	Instalación	103
8.2.1.	Obtención del código	103
8.2.2.	Entorno Python	104
8.2.3.	Descarga del dataset SoccerNet	104
8.3.	Estructura del repositorio	105
8.4.	Ejecución del pipeline de tracking	105
8.5.	Ejecución de la evaluación	106
8.6.	Ejecución del estudio de hiperparámetros	106
8.7.	Ejecución del pipeline de metadatos	107
8.8.	Manual de la aplicación web	108
8.8.1.	Generación del paquete de datos	108
8.8.2.	Instalación y ejecución del frontend	108

8.8.3. Uso de la interfaz	108
8.9. Problemas frecuentes	114
9 Uso de inteligencia artificial	115
9.1. Herramientas	115
9.2. Uso en el desarrollo de software	115
9.3. Uso en documentación	116
9.4. Límites del uso de inteligencia artificial	117
10 Conclusiones y trabajo futuro	118
10.1. Cumplimiento de los objetivos	118
10.2. Resultados principales	118
10.3. Limitaciones	119
10.4. Trabajo futuro	119
10.5. Reflexión final	120
Bibliografía	121
Glosario de términos	124

Índice de figuras

2.1. Relación de contención entre inteligencia artificial, aprendizaje automático, aprendizaje profundo y visión por computador. La visión por computador se apoya hoy predominantemente en aprendizaje profundo, aunque históricamente incluye también técnicas no basadas en redes neuronales (p. ej., K-means, sección 2.8).	7
2.2. Frame de una secuencia de SoccerNet-Tracking, con las cajas delimitadoras y los identificadores de un resultado de tracking superpuestos.	8
2.3. Las dos etapas de un sistema de MOT: detección y asociación. (sección 1.3).	10
2.4. Ciclo predicción-corrección del filtro de Kalman, repetido en cada frame para cada objeto seguido.	12
2.5. Intersección sobre Unión (IoU) entre una caja predicha por Kalman (A) y una detección real (B): la región sombreada de solape, dividida entre el área total ocupada por ambas cajas.	12
2.6. Las dos rondas de asociación de ByteTrack: primero con detecciones de alta confianza, después con las de baja confianza, recuperando así objetos parcialmente ocluidos que SORT descartaría [16].	15
2.7. Efecto de K-means sobre un conjunto de puntos: antes del agrupamiento (izquierda) y después de converger a $k = 3$ grupos, cada uno alrededor de su centroide (derecha) [25].	19
2.8. Campo de fútbol estándar visto en planta [26]. Las medidas FIFA reglamentarias empleadas como referencia para la homografía y para los mapas de calor de este proyecto (sección 6.6) son: 105 m $\times$ 68 m de terreno de juego, área grande 16,5 m $\times$ 40,3 m, área pequeña 5,5 m $\times$ 18,3 m, círculo central de 9,15 m de radio.	21
2.9. Puntos de referencia marcados sobre un frame real de retransmisión, usados para calcular la homografía: las cuatro esquinas del área pequeña, cuyas coordenadas de campo son conocidas por las medidas FIFA (Figura 2.8).	22
3.1. Estructura de descomposición del trabajo (EDT) del proyecto.	30
3.2. (pendiente de incorporar la captura).	32
4.1. Contexto funcional del sistema: el dataset público como entrada, el pipeline offline como bloque de cómputo, y la aplicación web como capa de visualización sobre los resultados.	43
4.2. Actores del sistema: operador del pipeline y usuario analista (izquierda), interactuando respectivamente con el procesamiento offline y la aplicación web, y SoccerNet como sistema externo (derecha). . . .	45
4.3. Casos de uso del pipeline offline: el operador del pipeline y SoccerNet como sistema externo.	60
4.4. Caso de uso de la aplicación web: el usuario analista explora los resultados a través de cuatro comportamientos incluidos.	61

5.1.	Arquitectura general del sistema, a nivel de componentes: el pipeline offline persiste todo su trabajo en disco en vez de mantener estado en memoria, y la aplicación web consume ese mismo almacenamiento ya calculado, sin recalcularlo nada ni compartir proceso con el pipeline. . . .	68
5.2.	Diseño del filtro geométrico de separación del balón: dos condiciones sobre área y proporción de la caja delimitadora, evaluadas sobre cada detección de cada frame antes de invocar al tracker.	69
5.3.	Bucle de procesamiento común a ambos algoritmos de tracking.	70
5.4.	Diseño de la ejecución batch: doble bucle sobre algoritmos y secuencias de un split, reutilizando el bucle de procesamiento de la Figura 5.3 para cada combinación.	70
5.5.	Diseño de la fase de evaluación.	71
5.6.	Diseño del estudio de hiperparámetros: barrido uno a uno sobre el conjunto de test, resumen comparativo y validación final de la configuración óptima de cada algoritmo.	72
5.7.	Dependencias entre los bloques del pipeline de metadatos.	73
5.8.	Diseño de la clasificación de equipos: K-means en dos niveles.	73
5.9.	Resultado real de la clasificación de equipos sobre un frame de SNMOT-118: cada jugador coloreado según el equipo asignado por K-means. . .	74
5.10.	Resultado real del mapa de calor por equipo sobre SNMOT-117.	74
5.11.	Diseño de la homografía	75
5.12.	Resultado real del mapa de calor con homografía real sobre un tramo de SNMOT-116.	76
5.13.	Diseño de la estimación de posesión.	76
5.14.	Arquitectura de la aplicación web: los artefactos exportados por <code>export_bundle.py</code> alimentan una capa de datos (peticiones <code>fetch</code> a JSON/PNG/MP4) que a su vez alimenta los componentes de interfaz de React.	78
5.15.	Croquis de la disposición implementada de la pestaña Explorador de secuencias	79
7.1.	HOTA integrada frente a HOTA(0) para ambos algoritmos. Bajo HOTA integrada, OC-SORT gana en ambos splits; bajo HOTA(0), la lectura se invierte a favor de ByteTrack.	95
7.2.	Efecto de <code>match_thresh</code> sobre ByteTrack (izquierda) y de <code>iou_threshold</code> sobre OC-SORT (derecha), los dos hiperparámetros de mayor impacto individual encontrados en el estudio (sección 6.5).	97
7.3.	Clasificación de equipos sobre un frame de SNMOT-117 (<code>team_visualization.py</code>).	98
7.4.	Mapa de calor general de SNMOT-116, proyección aproximada sin homografía.	99
7.5.	Mapa de calor con homografía real sobre el tramo de SNMOT-118; la zona sombreada indica dónde la calibración no es fiable.	101
8.1.	Pestaña «Comparativa de algoritmos».	110
8.2.	Pestaña «Explorador de secuencias» con SNMOT-116 y ByteTrack. . .	111
8.3.	SNMOT-117 con ByteTrack.	112
8.4.	SNMOT-118 con ByteTrack.	113

Índice de tablas

3.1. Resumen de las fases del proyecto.	26
3.2. Roles asumidos durante el proyecto y sus tareas asociadas.	28
3.3. Diccionario de la EDT: paquetes de trabajo, rol responsable y horas totales.	29
3.4. Cronograma real del proyecto por fase y mes. El tono claro distingue actividad parcial o de repaso/retoque frente al grueso del trabajo de cada fase (tono oscuro).	31
3.5. Hitos principales del proyecto.	33
3.6. Principales desviaciones respecto a la planificación inicial.	35
3.7. Estimación de costes de personal sobre las 320 h reales dedicadas al proyecto.	37
3.8. Costes materiales y operacionales estimados.	38
3.9. Coste total estimado (planificación inicial, 235 h) frente a coste real (320 h) y desviación.	38
3.10. Principales riesgos del proyecto y su mitigación.	40
3.11. Plan de calidad del proyecto.	42
4.1. Actores y usuarios del sistema.	45
4.2. IRQ-001: Detecciones precomputadas de SoccerNet-Tracking.	46
4.3. IRQ-002: <i>Ground truth</i> de tracking.	46
4.4. IRQ-003: Metadatos de secuencia y de partido.	46
4.5. IRQ-004: Resultados de tracking por algoritmo.	47
4.6. IRQ-005: Resultados de evaluación con métricas MOT.	47
4.7. IRQ-006: Resultados del estudio de hiperparámetros.	47
4.8. IRQ-007: Clasificación de jugadores por equipo.	48
4.9. IRQ-008: Metadatos deportivos derivados.	48
4.10. IRQ-009: Datos de visualización para la aplicación web.	48
4.11. RF-001: Descargar el dataset SoccerNet-Tracking.	49
4.12. RF-002: Filtrar el balón de las detecciones.	49
4.13. RF-003: Ejecutar ByteTrack sobre una secuencia.	49
4.14. RF-004: Ejecutar OC-SORT sobre una secuencia.	50
4.15. RF-005: Ejecutar el tracking sobre todas las secuencias de un split.	50
4.16. RF-006: Evaluar los resultados con métricas MOT estándar.	50
4.17. RF-007: Realizar un estudio de hiperparámetros.	51
4.18. RF-008: Determinar y validar la configuración óptima de cada algoritmo.	51
4.19. RF-009: Resumir y graficar el estudio de hiperparámetros.	51
4.20. RF-010: Clasificar a los jugadores por equipo.	52
4.21. RF-011: Visualizar la clasificación de equipos.	52
4.22. RF-012: Generar mapas de calor de ocupación del campo.	52
4.23. RF-013: Calcular una homografía real sobre un tramo estable.	52
4.24. RF-014: Estimar distancia recorrida y velocidad por jugador.	53
4.25. RF-015: Estimar la posesión del balón.	53

4.26. RF-016: Exportar vídeos anotados.	53
4.27. RF-017: Visualizar y explorar los resultados en una aplicación web. . .	54
4.28. RNF-001: Reproducibilidad.	54
4.29. RNF-002: Rendimiento y uso de memoria.	55
4.30. RNF-003: Portabilidad entre sistemas operativos.	55
4.31. RNF-004: Extensibilidad a nuevos algoritmos de tracking.	55
4.32. RNF-005: Trazabilidad de resultados.	56
4.33. RNF-006: Documentación honesta de limitaciones.	56
4.34. RNF-007: Usabilidad de la aplicación web.	56
4.35. RNF-008: Fluidez de la interacción web.	57
4.36. RNF-009: Consistencia ante datos incompletos.	57
4.37. RN-001: Exclusión del balón del tracking de personas.	58
4.38. RN-002: Evaluación final sobre el conjunto de test.	58
4.39. RN-003: Barrido de hiperparámetros de uno en uno.	58
4.40. RN-004: Clasificación de equipos con dos grupos.	59
4.41. RN-005: Homografía real solo sobre tramos verificados.	59
4.42. RN-006: Umbral máximo de distancia para la posesión.	59
4.43. CU-001: Descargar el dataset.	61
4.44. CU-002: Filtrar el balón de las detecciones.	61
4.45. CU-003: Ejecutar tracking (ByteTrack / OC-SORT).	62
4.46. CU-004: Evaluar resultados de tracking.	62
4.47. CU-005: Realizar estudio de hiperparámetros.	62
4.48. CU-006: Clasificar jugadores por equipo.	63
4.49. CU-007: Generar mapas de calor.	63
4.50. CU-008: Calcular homografía y estimar distancia/velocidad.	64
4.51. CU-009: Estimar posesión de balón.	64
4.52. CU-010: Exportar vídeos anotados.	65
4.53. CU-011: Explorar resultados en la aplicación web.	65
4.54. Trazabilidad de los requisitos de información.	66
4.55. Trazabilidad de los requisitos funcionales.	66
4.56. Trazabilidad de requisitos no funcionales y reglas de negocio.	67
5.1. Catálogo de artefactos generados por cada fase del sistema.	77
6.1. Librerías principales empleadas y su función en el sistema.	81
6.2. Problemas de entorno encontrados durante el desarrollo y su solución. .	83
6.3. Paquetes del repositorio y su responsabilidad.	84
6.4. Configuración base y rango de búsqueda de cada hiperparámetro. . . .	87
6.5. Efecto de cada hiperparámetro sobre HOTA(0) (%) en el conjunto de test.	88
7.1. Resultados de ByteTrack y OC-SORT sobre entrenamiento y test con la configuración por defecto.	95
7.2. Resultado de la clasificación de equipos por secuencia.	97
7.3. Jugadores con mayor distancia recorrida en cada tramo analizado con homografía real.	100
7.4. Posesión de balón por equipo y secuencia (estimada por proximidad). .	101

8.1. Requisitos previos para desplegar el sistema completo.	103
8.2. Problemas frecuentes al desplegar el sistema y su solución.	114

Índice de extractos de código

1. Introducción

1.1. Motivación

El análisis de datos es una disciplina de la informática que me apasiona, así como el deporte. El análisis de datos aplicado al deporte ha experimentado un crecimiento notable en la última década. Clubes, televisión, casas de apuestas y compañías de análisis deportivo invierten cada vez más recursos en extraer información objetiva del juego a partir del vídeo: distancias recorridas, mapas de calor, patrones tácticos o detección automática de eventos. Gran parte de esta información depende de una tarea previa fundamental: saber, en cada instante, dónde está cada jugador y quién es. Partiendo de esa base, posteriormente se pueden calcular distintas métricas de interés para el análisis del partido.

Ese problema, conocido en visión por computador como *seguimiento multiobjeto* o *Multi-Object Tracking* (MOT), es especialmente exigente en un partido de fútbol: veintidós jugadores más árbitros y balón se mueven de forma simultánea, se cruzan constantemente, se ocluyen entre sí y comparten una apariencia muy similar entre compañeros de equipo (misma camiseta, tallas y complexiones parecidas). Estas condiciones convierten al fútbol en un escenario particularmente duro para evaluar algoritmos de tracking, mucho más que los entornos de vigilancia urbana o conducción autónoma para los que originalmente se diseñaron muchos de estos algoritmos.

Este proyecto nace de aplicar y comparar de forma rigurosa algoritmos de seguimiento multiobjeto sobre vídeo real de partidos de fútbol profesional, y comprobar hasta qué punto la información de trayectorias que producen puede convertirse en metadatos deportivos útiles, sin necesidad de instrumentación adicional en el terreno de juego, únicamente a partir del vídeo.

1.2. Contexto

SoccerNet es un conjunto de datos y una serie de competiciones académicas centradas en el análisis automático de vídeo de fútbol, con múltiples tareas publicadas (reconocimiento de acciones, segmentación de cámara, reconocimiento de dorsales, seguimiento de jugadores, entre otras). Este proyecto se apoya en la tarea **SoccerNet-Tracking**, que proporciona secuencias de vídeo de partidos profesionales descompuestas en frames individuales, junto con detecciones de personas y balón ya calculadas (`det.txt`) y, para los conjuntos de entrenamiento y test, la posición e identidad real de cada jugador en cada frame (`gt.txt`), que sirve como referencia para medir el rendimiento de los algoritmos.

El uso de un dataset público y con un formato estándar permite que los resultados

obtenidos en este proyecto sean comparables con los de otros equipos de investigación, y que la evaluación se realice con las mismas métricas que utiliza la propia comunidad de SoccerNet en su tabla de clasificación oficial.

1.3. El problema

El seguimiento multiobjeto consiste en asignar, a cada objeto detectado en un vídeo, una identidad que se mantenga consistente a lo largo de todos los frames en los que ese objeto sea visible, incluso cuando desaparece temporalmente de la imagen, por ejemplo, al quedar oculto detrás de otro jugador y vuelve a aparecer poco después. Un buen algoritmo de tracking debe resolver simultáneamente dos problemas relacionados pero distintos:

- **Detección:** encontrar dónde está cada objeto de interés en cada frame. En este proyecto, jugadores, árbitros y balón.
- **Asociación:** decidir qué detección de un frame corresponde a qué identidad ya existente de un frame anterior, o si se trata de un objeto nuevo.

Es importante delimitar aquí, desde el principio, el alcance exacto de este proyecto: el dataset SoccerNet-Tracking proporciona las detecciones ya calculadas para cada frame (`det.txt`). Por tanto, la tarea abordada es específicamente la de **asociación**: mantener identidades consistentes a partir de un conjunto de detecciones dado y no la de detección de jugadores desde cero a partir de la imagen. Esta decisión, natural dado el planteamiento del dataset, permite centrar el esfuerzo y la comparativa en los algoritmos de tracking propiamente dichos (el componente de asociación), que es donde reside la principal diferencia entre los distintos métodos evaluados. Esta distinción se retoma con más detalle en el capítulo 4.

1.4. Objetivos

1.4.1. Objetivo general

Una vez definidas las detecciones del dataset SoccerNet, se propone evaluar y comparar distintos algoritmos de seguimiento multiobjeto sobre secuencias reales de partidos de fútbol del dataset SoccerNet, y explorar en qué medida las trayectorias resultantes permiten extraer metadatos deportivos de utilidad para el análisis de un partido.

1.4.2. Objetivos específicos

A partir de este objetivo general se han definido los siguientes objetivos específicos:

- **OE-01:** Estudiar la estructura del dataset SoccerNet-Tracking (secuencias de frames, detecciones precomputadas y *ground truth*) y delimitar la tarea abordada como un problema de asociación en mayor medida que de detección.
- **OE-02:** Implementar varios algoritmos de tracking de tipo *tracking-by-detection*, incluyendo un filtrado geométrico que separe el balón de los jugadores antes de la asociación.
- **OE-03:** Ejecutar los algoritmos escogidos (**ByteTrack** y **OC-SORT**) sobre la totalidad de las secuencias de los conjuntos de entrenamiento y de test del dataset.
- **OE-04:** Evaluar ambos algoritmos con las métricas estándar del campo (HOTA, MOTA, IDF1) mediante **TrackEval**, comparando su comportamiento entre el conjunto de entrenamiento y el de test.
- **OE-05:** Realizar un estudio sistemático de hiperparámetros de cada algoritmo, variando un parámetro a la vez, y determinar una configuración óptima para cada uno de ellos.
- **OE-06:** Diseñar y aplicar un método de clasificación de jugadores por equipo a partir del color dominante de la camiseta mediante K-means.
- **OE-07:** Generar mapas de calor de ocupación del campo a partir de las trayectorias obtenidas, tanto de forma global como separados por equipo.
- **OE-08:** Calcular una homografía real sobre tramos de vídeo con cámara estable, como prueba de concepto para estimar la distancia recorrida y la velocidad de los jugadores en unidades reales.
- **OE-09:** Estimar la posesión del balón por proximidad espacial entre el balón y el jugador más cercano, y si es posible, generar un resumen de posesión de balón por jugador y agregarla por equipo.
- **OE-10:** Documentar las limitaciones técnicas encontradas en cada etapa del proyecto, incluyendo enfoques explorados y descartados, de modo que la memoria refleje fielmente tanto lo conseguido como las dificultades reales del proceso.
- **OE-11:** Desarrollar una aplicación web que sirva como demostrador visual e interactivo de los resultados del proyecto obtenidos, permitiendo explorar los resultados de tracking y el análisis de datos generado. También se destina a personas que no sean expertas en el área para facilitar su comprensión y visualización.

1.5. Alcance y delimitación del proyecto

Como se ha introducido en la sección 1.3, este proyecto parte de las detecciones del dataset y se centra en la etapa de asociación (tracking), no en el diseño de un detector de objetos propio. De todas formas, conviene precisar otros límites del proyecto desde el principio:

- Los algoritmos comparados corresponden a la familia de métodos *tracking-by-detection* clásicos, sin componente de reidentificación visual profunda (*re-ID*); la incorporación de algoritmos con re-ID se plantea como posible ampliación (véase capítulo 10).
- La proyección de posiciones de píxel a coordenadas reales del campo se realiza mediante dos enfoques de distinta fiabilidad: una aproximación por rango observado (sin calibración de cámara), usada de forma general, y una homografía real calculada manualmente sobre tramos concretos con cámara estable, usada como prueba de concepto para el cálculo de distancia y velocidad. Ambos enfoques, y sus limitaciones, se detallan en el capítulo 6.
- La posesión del balón se estima por proximidad espacial entre el balón detectado y el jugador más cercano, una aproximación razonable pero distinta del concepto de posesión que manejaría un analista táctico humano.

1.6. Estructura del documento

La memoria de este proyecto se organiza en los siguientes capítulos:

- **Capítulo 2 – Fundamentos teóricos y estado del arte.** Presenta los conceptos teóricos necesarios para entender el proyecto: el problema del seguimiento multiobjeto, los algoritmos de tracking comparados, las métricas de evaluación empleadas, la clasificación de jugadores por color y los fundamentos de la homografía entre otros; y cierra situando este TFG frente a los resultados del reto oficial de SoccerNet.
- **Capítulo 3 – Gestión del proyecto.** Describe la metodología de trabajo seguida, la planificación temporal, la estructura de descomposición del trabajo, y una estimación de los recursos y costes que habría requerido este proyecto en un contexto profesional.
- **Capítulo 4 – Análisis de requisitos.** Recoge los actores del sistema, los requisitos de información, funcionales y no funcionales, las reglas de negocio y los casos de uso.
- **Capítulo 5 – Diseño.** Presenta la arquitectura general del sistema y las decisiones de diseño tomadas.
- **Capítulo 6 – Implementación.** Detalla cómo se han implementado el tracking, la evaluación, el estudio de hiperparámetros, el pipeline de metadatos y la aplicación web, incluyendo los problemas técnicos encontrados y los enfoques que se probaron y no funcionaron.
- **Capítulo 7 – Resultados.** Recoge la comparativa final entre ByteTrack y OC-SORT, los resultados del estudio de hiperparámetros y del pipeline de metadatos, y las limitaciones detectadas en cada uno.
- **Capítulo 8 – Manual de despliegue.** Explica paso a paso cómo instalar, configurar y ejecutar el sistema, incluida la aplicación web.

- **Capítulo 9 – Uso de inteligencia artificial.** Detalla el uso realizado de herramientas de inteligencia artificial durante el desarrollo del proyecto.
- **Capítulo 10 – Conclusiones y trabajo futuro.** Resume las conclusiones del proyecto y plantea posibles líneas futuras de trabajo.

Tras la bibliografía se incluye un glosario con los términos técnicos y las siglas usadas a lo largo de la memoria.

2. Fundamentos teóricos y estado del arte

Este capítulo presenta, de lo general a lo particular, todos los conceptos teóricos necesarios para comprender el resto de este proyecto. Se parte de las nociones más básicas: qué es la inteligencia artificial, qué es un dataset y por qué existe SoccerNet. Y se profundiza progresivamente hasta el detalle técnico de cada algoritmo, métrica y técnica empleados: el problema del seguimiento multiobjeto, los algoritmos de *tracking-by-detection* comparados (ByteTrack y OC-SORT) y los que se consideraron pero no llegaron a implementarse, las métricas de evaluación estándar del campo (MOTA, IDF1, HOTA), la clasificación de jugadores por color mediante K-means, y los fundamentos de la homografía aplicada a la proyección de coordenadas. Las decisiones concretas de este proyecto como parámetros, configuración, qué resultado se obtuvo no se abordan aquí, sino en los capítulos de diseño, implementación y resultados (capítulos 5, 6 y 7). El capítulo se cierra situando este TFG frente a trabajos relacionados y al reto oficial de SoccerNet (sección 2.10).

2.1. Inteligencia artificial, aprendizaje automático y visión por computador

La **inteligencia artificial** (IA) es el campo de la informática dedicado a construir sistemas y programas capaces de realizar tareas que normalmente requerían de la inteligencia humana, como el aprendizaje, el razonamiento y la percepción [1]. Estas tareas incluyen reconocer un objeto en una imagen, entender una frase o decidir la mejor ruta en un mapa. Dentro de ese campo tan amplio, el **aprendizaje automático** (*machine learning*) es el subconjunto de la IA que no programa explícitamente la solución a un problema, sino que la *aprende* a partir de los patrones presentes en los datos de entrenamiento, para después aplicar lo aprendido sobre datos nuevos [2]: en lugar de escribir a mano las reglas que distinguen un gato de un perro en una fotografía, se le muestran al sistema miles de fotografías etiquetadas y es el propio sistema el que extrae, mediante un proceso de optimización, los patrones que le permiten generalizar a fotografías nuevas que no ha visto antes.

Dentro del aprendizaje automático, el **aprendizaje profundo** (*deep learning*) es el subconjunto de técnicas basadas en redes neuronales artificiales de varias capas, cuyo diseño se inspira de forma general en la estructura del cerebro humano, capaces de aprender directamente representaciones útiles de los datos en bruto –p. ej., píxeles de una imagen– sin necesidad de que un experto humano diseñe a mano las características relevantes; es este subcampo el que sostiene hoy la mayor parte de los avances más recientes de la IA, desde la visión artificial y la IA generativa hasta la conducción autónoma o la robótica [3]. Es precisamente en visión artificial donde el aprendizaje

profundo ha hecho posibles, en la última década, los avances más visibles: la **visión por computador** es la disciplina que estudia cómo dotar a una máquina de la capacidad de procesar, analizar e interpretar información significativa a partir de imágenes o vídeo [4], y de ella forman parte tanto la detección de objetos (encontrar dónde está cada objeto en una imagen) como el seguimiento multiobjeto, tarea de este proyecto que se define formalmente en la sección 2.3.

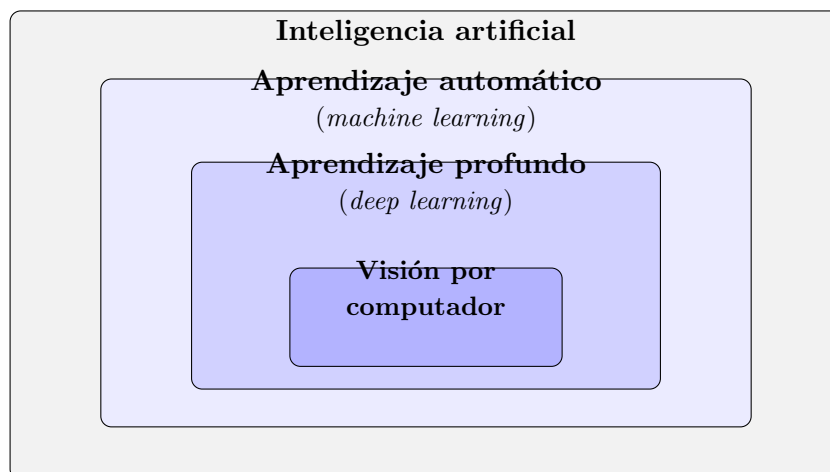


Figura 2.1: Relación de contención entre inteligencia artificial, aprendizaje automático, aprendizaje profundo y visión por computador. La visión por computador se apoya hoy predominantemente en aprendizaje profundo, aunque históricamente incluye también técnicas no basadas en redes neuronales (p. ej., K-means, sección 2.8).

El **análisis de datos** es, en paralelo, la disciplina que convierte datos generados por sensores o, como en este proyecto, algoritmos de visión por computador, en información interpretable: medias, distribuciones, mapas de calor, tendencias o comparativas [5]. La intersección de ambos campos es precisamente lo que estructura este proyecto: primero se usa visión por computador para producir datos de trayectorias a partir de vídeo (capítulos 5 y 6), y después se aplica análisis de datos sobre esas trayectorias para obtener metadatos deportivos interpretables (mapas de calor, distancias, posesión), como se detalla en la sección 6.6.

2.2. Datasets y el ecosistema SoccerNet

Todo sistema de aprendizaje automático necesita datos con los que aprender y, sobre todo, con los que poder evaluarse de forma objetiva. Un **dataset** (conjunto de datos) es, en este contexto, una colección organizada de ejemplos (en visión por computador, típicamente imágenes o vídeos) acompañados de anotaciones de referencia (*ground truth*) que indican cuál es la respuesta correcta para cada ejemplo [6]. Un **benchmark** es, además, un dataset acompañado de un protocolo de evaluación estandarizado (qué métricas se calculan, cómo se dividen los datos en entrenamiento y test) que permite comparar de forma justa los resultados obtenidos por distintos investigadores o equipos, incluso años después y sin que hayan trabajado nunca juntos. Sin datasets públicos con estas características, cualquier comparación

entre algoritmos, como la que constituye el núcleo de este proyecto, carecería de validez: cada autor podría estar evaluando sobre datos distintos, más favorables a su propio método. MOTChallenge [7] y el propio SoccerNet-Tracking [8], presentados a continuación, son dos ejemplos concretos de este tipo de benchmark aplicados al seguimiento multiobjeto.

SoccerNet [9] es un ecosistema de datasets y competiciones académicas anuales centrado en el análisis automático de vídeo de fútbol profesional, mantenido por un consorcio de investigadores de varias universidades. Ofrece múltiples tareas sobre el mismo conjunto de partidos de origen (reconocimiento de acciones, segmentación del tipo de plano de cámara, reconocimiento del dorsal del jugador, seguimiento multiobjeto, entre otras), cada una con su propio subconjunto de datos, su propio formato de anotación y su propio protocolo de evaluación. La sección 4.1.1 del capítulo 4 detalla el proceso de exploración de estas tareas alternativas antes de decidirse por la tarea de tracking.

2.2.1. SoccerNet-Tracking

Este proyecto se apoya específicamente en **SoccerNet-Tracking** [8], la tarea de seguimiento multiobjeto del ecosistema SoccerNet. El dataset está compuesto por 200 secuencias de vídeo de 30 segundos cada una, extraídas de partidos profesionales reales y seleccionadas por representar situaciones de juego especialmente exigentes para el tracking (aglomeraciones de jugadores, cambios bruscos de dirección, oclusiones). Cada secuencia se distribuye ya descompuesta en frames individuales (formato de imagen), en lugar de como un fichero de vídeo, lo que facilita el procesamiento frame a frame que exigen los algoritmos de tracking.



Figura 2.2: Frame de una secuencia de SoccerNet-Tracking, con las cajas delimitadoras y los identificadores de un resultado de tracking superpuestos.

Cada secuencia incluye, junto con los frames, los siguientes ficheros:

- **img1/**: carpeta con los frames individuales de la secuencia, numerados de forma correlativa.
- **det/det.txt**: detecciones *precomputadas* de cada objeto (jugadores, árbitros y balón) en cada frame, obtenidas con un detector de objetos ya entrenado por los organizadores del dataset. Esta es la entrada real de los algoritmos de tracking evaluados en este proyecto, como se explica en la sección 2.3.
- **gt/gt.txt**: la posición e identidad *real* de cada objeto en cada frame, usada como referencia para calcular las métricas de evaluación de la sección 2.6.
- **seqinfo.ini** y **gameinfo.ini**: metadatos de la secuencia (resolución de imagen, *frame rate*) y del partido de origen.

Tanto **det.txt** como **gt.txt** siguen el formato estándar del protocolo **MOTChallenge** [7], el benchmark de referencia histórico en seguimiento multiobjeto (orientado originalmente a vigilancia urbana de peatones, no a deporte), que SoccerNet-Tracking reutiliza para que las herramientas de evaluación ya existentes (en particular TrackEval, empleada en este proyecto (sección 6.4)) funcionen sin modificaciones. Cada línea de estos ficheros describe un objeto en un frame concreto:

```
frame_id, obj_id, x, y, w, h, confianza, -1, -1, -1
```

donde **frame_id** identifica el frame, **obj_id** identifica al objeto (en **gt.txt**, su identidad real y consistente a lo largo de toda la secuencia; en **det.txt**, este campo no se usa), y **x**, **y**, **w**, **h** describen la posición y el tamaño de la caja delimitadora (*bounding box*) del objeto en píxeles, con (**x**, **y**) la esquina superior izquierda. Los tres últimos campos, fijos a -1, corresponden a información adicional (clase de objeto, visibilidad) que MOTChallenge contempla para otros dominios pero que SoccerNet-Tracking no utiliza.

El dataset se organiza en tres particiones o splits: un conjunto de **entrenamiento** (57 secuencias, con **gt.txt** disponible, usado para desarrollar y ajustar los algoritmos), un conjunto de **test** (con **gt.txt** disponible, reservado para la evaluación final y no consultado durante el ajuste, tal y como exige una evaluación honesta) y un conjunto de **challenge** (sin **gt.txt**, usado únicamente para el ranking oficial de la competición SoccerNet, no utilizado en este proyecto).

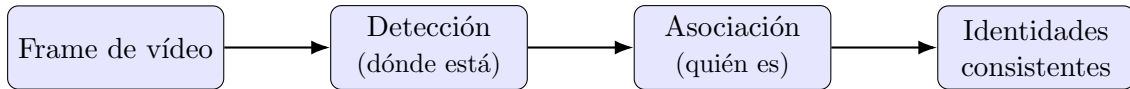
2.3. El problema del seguimiento multiobjeto

El **seguimiento multiobjeto** (*Multi-Object Tracking*, MOT) consiste en asignar, a cada objeto de interés que aparece en un vídeo, una identidad numérica que se mantenga consistente a lo largo de todos los frames en los que ese objeto sea visible incluso si desaparece temporalmente de la imagen, por ejemplo al quedar oculto detrás de otro jugador, y reaparece poco después [10]. Un sistema de MOT completo resuelve, en general, dos subproblemas relacionados pero conceptualmente distintos, representados en la Figura 2.3:

1. **Detección**: localizar dónde está cada objeto de interés en cada frame individual, sin ninguna noción de continuidad temporal, es decir, sin saber todavía si el

objeto detectado en el frame t es el mismo que se detectó en el frame $t - 1$. Esta tarea se resuelve habitualmente con un detector de objetos entrenado con aprendizaje profundo (p. ej., la familia YOLO, discutida en la sección 2.7).

2. **Asociación:** dado el conjunto de detecciones de cada frame, decidir qué detección del frame actual corresponde a qué identidad ya existente de frames anteriores, o si se trata de un objeto que aparece por primera vez. Este es el problema que resuelven específicamente los algoritmos de tracking comparados en este proyecto (sección 2.5).



Este proyecto se centra únicamente en la asociación.

Figura 2.3: Las dos etapas de un sistema de MOT: detección y asociación. (sección 1.3).

Como se explicó en la sección 1.3, este proyecto se sitúa solo en la segunda mitad de ese proceso: dado que SoccerNet-Tracking proporciona las detecciones ya calculadas en `det.txt` (sección 2.2.1), la tarea abordada es específicamente la asociación, también llamada *tracking-by-detection* en la literatura, por oposición a los métodos de *joint detection and tracking* que entrenan un único modelo capaz de detectar y asociar a la vez. Esta decisión, natural dado el planteamiento del dataset, es también la que sigue la comparativa oficial de SoccerNet-Tracking, lo que permite que los resultados de este proyecto sean comparables con los de otros participantes de la competición.

2.3.1. Retos específicos del seguimiento en fútbol

El fútbol es un escenario particularmente exigente para el MOT, mucho más que los entornos de vigilancia urbana o conducción autónoma para los que originalmente se diseñaron muchos de los algoritmos clásicos del campo, hoy ampliamente aplicados en escenarios reales de ese tipo [10]. Entre los factores que lo hacen especialmente difícil:

- **Número elevado y constante de objetos:** veintidós jugadores, uno o varios árbitros y el balón se mueven de forma simultánea durante todo el partido, frente a los escenarios más dispersos de la vigilancia urbana.
- **Apariencia muy similar entre objetos:** los jugadores de un mismo equipo visten la misma camiseta, lo que hace que los métodos de asociación basados en apariencia visual (re-identificación, sección 2.5.4) sean menos discriminativos que en otros dominios, donde cada persona suele vestir de forma distinta.
- **Oclusiones frecuentes:** los jugadores se cruzan y agrupan constantemente (disputas de balón, saques de esquina, barreras en faltas), lo que provoca oclusiones parciales o totales que ponen a prueba la capacidad de un algoritmo de mantener la identidad de un jugador mientras está oculto y de recuperarla correctamente al reaparecer.

- **Movimiento no lineal:** a diferencia de un peatón o un vehículo, que en general mantienen una trayectoria relativamente predecible durante fracciones de segundo, un jugador de fútbol puede cambiar de dirección y de velocidad de forma muy brusca (un regate, una parada súbita), lo que penaliza a los modelos de movimiento más simples, como se discute en la sección 2.5.2.
- **El balón como caso extremo:** al margen de las personas, el balón es un objeto pequeño, de movimiento muy rápido y no lineal (rebotes, golpes), y visualmente casi indistinguible de otros elementos redondeados y claros del campo. La sección 2.7 detalla cómo se ha abordado su detección en este proyecto.

2.4. Fundamentos de *tracking-by-detection*

Antes de describir los algoritmos concretos comparados en este proyecto (sección 2.5), conviene introducir los dos bloques matemáticos sobre los que se construyen prácticamente todos los métodos clásicos de *tracking-by-detection*: un modelo que predice dónde estará cada objeto en el siguiente frame, y un procedimiento que decide qué detección real se asocia a cada predicción.

2.4.1. El filtro de Kalman

El **filtro de Kalman** [11] es un algoritmo recursivo que estima el estado de un sistema dinámico, en este caso, la posición y la velocidad de un objeto que se mueve a partir de observaciones ruidosas a lo largo del tiempo. Funciona mediante un ciclo de dos fases que se repite en cada frame, representado en la Figura 2.4:

1. **Predicción:** a partir del estado estimado en el frame anterior (posición y velocidad), el filtro predice dónde debería estar el objeto en el frame actual, asumiendo un modelo de movimiento simple (típicamente, velocidad constante).
2. **Corrección** (o actualización): cuando llega la observación real de ese frame en este caso, la detección que finalmente se le asocie mediante el procedimiento de la sección 2.4.3, el filtro combina esa observación con su propia predicción, ponderando cada una según su incertidumbre estimada, para obtener una estimación corregida del estado.

Este ciclo tiene una consecuencia práctica importante para el tracking: aun cuando un objeto deja de detectarse durante varios frames (por ejemplo, por una oclusión), el filtro de Kalman sigue pudiendo predecir su posición probable basándose en su movimiento anterior, lo que permite mantener viva su identidad y recuperar la asociación correcta cuando vuelve a detectarse. Los tres algoritmos comparados y considerados en este proyecto (SORT, OC-SORT y ByteTrack, sección 2.5) usan variantes de un filtro de Kalman como modelo de movimiento subyacente.

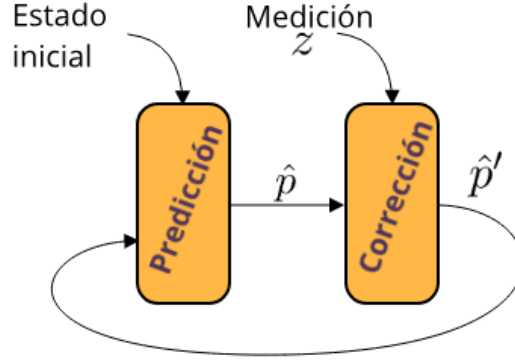


Figura 2.4: Ciclo predicción-corrección del filtro de Kalman, repetido en cada frame para cada objeto seguido.

2.4.2. Solapamiento entre cajas: IoU

Para decidir si una detección de un frame corresponde al mismo objeto que una predicción de Kalman (o que otra detección de un frame anterior), los métodos de *tracking-by-detection* necesitan una medida de similitud entre dos cajas delimitadoras. La más utilizada, por su simplicidad y bajo coste computacional, es la **Intersección sobre Unión** (*Intersection over Union*, IoU): dadas dos cajas A y B , se define como

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (2.1)$$

es decir, el área de la región donde ambas cajas se solapan, dividida entre el área total que cubren entre las dos. La IoU vale 1 cuando las cajas coinciden exactamente y 0 cuando no se solapan en absoluto, lo que la hace directamente interpretable como un porcentaje de acuerdo espacial (véase la Figura 2.5).

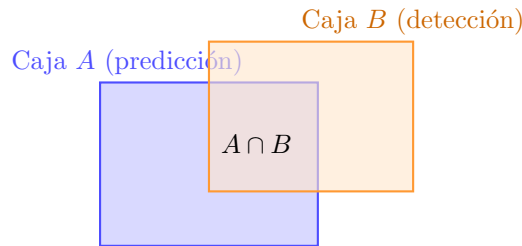


Figura 2.5: Intersección sobre Unión (IoU) entre una caja predicha por Kalman (A) y una detección real (B): la región sombreada de solape, dividida entre el área total ocupada por ambas cajas.

2.4.3. El problema de asignación y el algoritmo húngaro

Una vez calculada una matriz de similitud (por ejemplo, de IoU) entre todas las predicciones activas y todas las detecciones de un frame, queda por resolver el

problema de asignación: encontrar la correspondencia uno a uno entre predicciones y detecciones que maximice la similitud total, sin que ninguna detección se asocie a más de una predicción ni viceversa. Resolver este problema por fuerza bruta (probar todas las combinaciones posibles) tiene un coste factorial en el número de objetos, inviable ya con unas pocas decenas de jugadores en pantalla.

El **algoritmo húngaro** [12] resuelve este problema de forma exacta y en tiempo polinómico. Dada una matriz de coste M (que puede construirse, por ejemplo, como $1 - \text{IoU}$, de modo que minimizar coste equivale a maximizar solapamiento), el algoritmo encuentra la asignación completa de coste mínimo mediante una serie de transformaciones de la matriz que no alteran la asignación óptima pero reducen progresivamente el problema, hasta poder leer la solución directamente. En la práctica, ninguno de los algoritmos de tracking usados en este proyecto implementa el algoritmo húngaro desde cero: todos delegan en la implementación ya optimizada de `scipy` o de librerías equivalentes, a través de la librería `boxmot` descrita en la sección 6.1.

Un detalle que puede ser importante a la hora de entender ByteTrack en la sección 2.5.3 es que no toda asignación con coste finito se acepta: se suele imponer un umbral mínimo de similitud (p. ej., IoU mínima) por debajo del cual una predicción y una detección no se consideran candidatas a asociarse, aunque el algoritmo húngaro las hubiera emparejado matemáticamente. Las predicciones que no encuentran ninguna detección compatible se marcan como no observadas en ese frame, mientras que las detecciones que no encuentran ninguna predicción compatible inician una identidad nueva.

2.5. Algoritmos de tracking comparados

Con los fundamentos de las secciones anteriores, esta sección describe los dos algoritmos efectivamente implementados y comparados en este proyecto ByteTrack y OC-SORT así como el algoritmo histórico del que ambos derivan (SORT) y las familias de algoritmos que se consideraron pero finalmente no se implementaron, con la justificación de por qué.

2.5.1. SORT: el punto de partida

SORT (*Simple Online and Realtime Tracking*, 13, 14) es el algoritmo que sentó las bases del *tracking-by-detection* moderno: para cada objeto seguido, mantiene un filtro de Kalman (sección 2.4.1) que predice su posición en el frame actual; calcula la IoU (sección 2.4.2) entre esas predicciones y todas las detecciones del frame; resuelve la asignación óptima con el algoritmo húngaro (sección 2.4.3); y actualiza cada filtro de Kalman con la detección que le ha correspondido. Su principal virtud, que explica su influencia posterior, es la simplicidad: no requiere ningún modelo de apariencia visual ni entrenamiento adicional más allá del propio detector de objetos, lo que lo hace extremadamente rápido.

SORT tiene, sin embargo, una limitación conocida que motivó buena parte de las mejoras posteriores: solo considera para la asociación las detecciones de *alta confianza* del detector, descartando directamente las de confianza intermedia o baja. En escenarios con oclusiones frecuentes como el fútbol, un jugador parcialmente ocluido suele generar una detección de confianza más baja (o directamente no generar ninguna), lo que provoca que SORT pierda su identidad con más facilidad de la deseable y genere más *cambios de identidad* (*ID switches*) de los necesarios. Esta limitación es precisamente la que aborda ByteTrack (sección 2.5.3).

En `boxmot`, la librería usada en este proyecto (sección 6.1), no existe una implementación del SORT original independiente: la clase disponible más próxima es OCSORT, su evolución directa descrita en la sección 2.5.2. Por ese motivo, y como se explica en el capítulo 6, el script de este proyecto que originalmente pretendía implementar SORT (`sort_alg.py`) contiene en realidad OC-SORT, quedando el nombre del fichero como huella de esa decisión.

2.5.2. OC-SORT: SORT centrado en la observación

OC-SORT (*Observation-Centric SORT*, 15) identifica y corrige dos problemas concretos de SORT, sin abandonar su simplicidad y velocidad. El primero es lo que sus autores llaman *acumulación de error durante la desaparición*: cuando un objeto deja de detectarse durante varios frames (oclusión), el filtro de Kalman de SORT sigue extrapolando su posición usando exclusivamente su propio modelo de movimiento, sin ninguna corrección, de modo que el error de la predicción crece sin control cuanto más dura la oclusión; al reaparecer el objeto, la predicción puede estar ya demasiado lejos de la posición real para una asociación correcta. El segundo es la sensibilidad del filtro de Kalman a los movimientos no lineales (sección 2.3), especialmente marcados en fútbol.

OC-SORT resuelve ambos problemas centrando el filtro en las *observaciones reales* en lugar de en el estado interno acumulado por Kalman: en lugar de corregir el filtro paso a paso con predicciones intermedias durante toda la desaparición, cuando el objeto reaparece OC-SORT recalcula directamente su trayectoria a partir de la última observación real conocida antes de desaparecer y la primera observación real al reaparecer, evitando así la deriva acumulada. Este enfoque, más robusto ante oclusiones largas y movimientos bruscos, es el que motivó su inclusión en la comparativa de este proyecto junto con ByteTrack.

2.5.3. ByteTrack: aprovechar todas las detecciones

ByteTrack [16, 14] aborda directamente la limitación de SORT descrita en la sección 2.5.1: en lugar de descartar las detecciones de baja confianza, las aprovecha en una *segunda ronda* de asociación. Su procedimiento, para cada frame, es el siguiente:

1. Se separan las detecciones del frame en dos grupos según un umbral de confianza: de **alta** y de **baja** confianza.

2. **Primera asociación:** se emparejan las predicciones de Kalman con las detecciones de alta confianza, con el procedimiento estándar de IoU y algoritmo húngaro (secciones 2.4.2 y 2.4.3).
3. **Segunda asociación:** las predicciones que no encontraron pareja en el paso anterior normalmente son objetos parcialmente ocluidos cuya detección, si existe, es de baja confianza. Estas predicciones se intentan emparejar de nuevo, esta vez únicamente contra las detecciones de baja confianza.
4. Las predicciones que siguen sin pareja tras ambas rondas se mantienen como activas sin observación (apoyándose en el filtro de Kalman) durante un número limitado de frames antes de eliminarse definitivamente y las detecciones de alta confianza que no encontraron ninguna predicción compatible inician identidades nuevas.

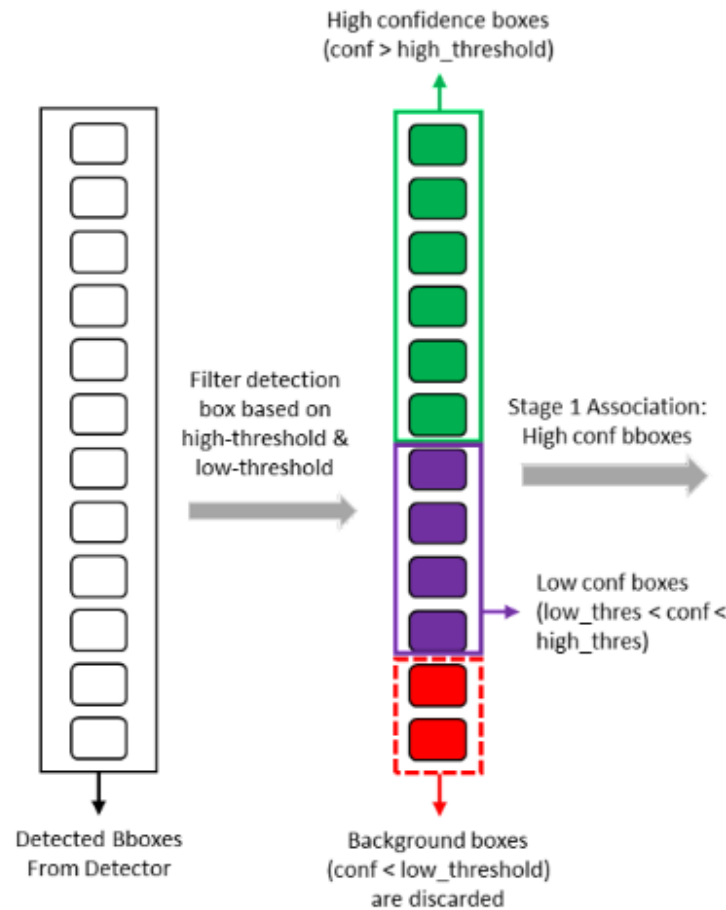


Figura 2.6: Las dos rondas de asociación de ByteTrack: primero con detecciones de alta confianza, después con las de baja confianza, recuperando así objetos parcialmente ocluidos que SORT descartaría [16].

Esta segunda ronda es, precisamente, lo que hace a ByteTrack especialmente adecuado para el fútbol: un jugador ocluido detrás de otro suele seguir generando una detección débil (de baja confianza) en lugar de ninguna detección, y ByteTrack es

capaz de aprovechar esa señal parcial en lugar de descartarla, a diferencia de SORT y, en menor medida, de OC-SORT.

2.5.4. Trackers con re-identificación visual: considerados y no implementados

Los tres algoritmos descritos hasta ahora (SORT, OC-SORT, ByteTrack) pertenecen a la familia de trackers *sin apariencia*: solo usan la posición y el tamaño de las cajas delimitadoras para asociar, nunca el aspecto visual del objeto. Existe una familia paralela de algoritmos que incorpora, además, un modelo de **re-identificación** (*Re-ID*): una red neuronal, entrenada por separado, que extrae un vector numérico (*embedding*) representativo de la apariencia visual de cada objeto, de modo que dos detecciones del mismo objeto puedan reconocerse por similitud de apariencia y no solo por proximidad espacial. **DeepSORT** [17, 14] fue el primer algoritmo relevante en incorporar esta idea sobre la base de SORT, y su enfoque se ha extendido después a variantes más recientes como StrongSORT o BoT-SORT, todas ellas también disponibles en `boxmot`.

El tutor de este proyecto planteó explícitamente, en una de las reuniones de seguimiento (sección 3.3), la posibilidad de ampliar la comparativa con algoritmos de esta familia. Finalmente no se ha incorporado ningún tracker con Re-ID a este proyecto, por dos motivos principales. En primer lugar, una comparación justa exigiría fijar el mismo modelo de Re-ID para todos los algoritmos comparados (por ejemplo, `osnet_x0.25_msmt17`, disponible directamente en `boxmot`), lo que añade una variable más –la calidad del propio modelo de Re-ID, no entrenado por el equipo del proyecto ni específico de fútbol– a una comparativa que, sin esa variable, ya resultaba suficientemente rica con el estudio de hiperparámetros de los dos algoritmos sin apariencia (capítulo 6). En segundo lugar, como se ha señalado en la sección 2.3, la apariencia visual es un criterio especialmente poco discriminativo en fútbol (los compañeros de equipo visten de forma prácticamente idéntica), lo que hace previsible que el margen de mejora aportado por Re-ID en este dominio concreto sea menor que en otros escenarios de MOT donde cada persona viste de forma distinta.

2.6. Métricas de evaluación de sistemas de tracking

Comparar dos algoritmos de tracking exige una forma objetiva de medir su rendimiento sobre el *ground truth* (`gt.txt`, sección 2.2.1). Esta sección presenta las tres familias de métricas empleadas en este proyecto y calculadas con `TrackEval` (sección 6.4): MOTA/MOTP (las más clásicas), IDF1 (centrada en la consistencia de identidad) y HOTA (la más reciente y equilibrada, adoptada como métrica principal de este proyecto).

2.6.1. CLEAR MOT: MOTA y MOTP

Las métricas **CLEAR MOT** [18] fueron, durante más de una década, el estándar de facto en la evaluación de sistemas de tracking. **MOTA** (*Multiple Object Tracking Accuracy*) resume en un único porcentaje tres tipos de error acumulados a lo largo de toda la secuencia: falsos negativos (objetos reales no detectados), falsos positivos (detecciones que no corresponden a ningún objeto real) y cambios de identidad (*ID switches*, cuando el algoritmo confunde la identidad de un objeto con la de otro). **MOTP** (*Multiple Object Tracking Precision*), complementaria a MOTA, mide la precisión espacial media de las cajas correctamente emparejadas (cuánto se solapan, en promedio, con el *ground truth*), sin penalizar los errores de asociación que ya recoge MOTA.

La principal limitación de MOTA, señalada posteriormente varias veces en este documento [19], es que pondera con mucho más peso los errores de detección (falsos positivos y negativos) que los errores de asociación (cambios de identidad): un algoritmo puede detectar correctamente a casi todos los jugadores en casi todos los frames y aun así confundir sistemáticamente sus identidades, obteniendo una MOTA engañosamente alta pese a ser, en la práctica, poco útil para cualquier análisis que dependa de trayectorias individuales consistentes, como el pipeline de metadatos de este mismo proyecto (sección 6.6).

2.6.2. IDF1

IDF1 [20] nace precisamente para corregir ese desequilibrio, centrando la evaluación en la consistencia de identidad en lugar de en la detección frame a frame. Se calcula como la media armónica (F1) entre la precisión y la exhaustividad (*recall*) de identidad, calculadas tras resolver una única asignación óptima, de nuevo mediante el algoritmo húngaro, sección 2.4.3 entre las trayectorias predichas por el algoritmo y las trayectorias reales del *ground truth* a lo largo de *toda* la secuencia, no frame a frame. En la práctica, IDF1 penaliza con mucha más severidad que MOTA los cambios de identidad, y es una de las tres métricas reportadas en los resultados de este proyecto (capítulo 7).

2.6.3. HOTA: la métrica principal de este proyecto

HOTA (*Higher Order Tracking Accuracy*, 19, 21) es la métrica más reciente de las tres, propuesta explícitamente para resolver el desequilibrio entre detección y asociación que arrastraban tanto MOTA (sesgada hacia la detección) como IDF1 (sesgada hacia la asociación). HOTA se define como la media geométrica de dos componentes independientes:

$$\text{HOTA} = \sqrt{\text{DetA} \times \text{AssA}}, \quad (2.2)$$

donde **DetA** (*Detection Accuracy*) mide, de forma análoga a MOTA, qué tan bien se detectan los objetos, y **AssA** (*Association Accuracy*) mide, de forma análoga a

IDF1, qué tan bien se mantienen las identidades a lo largo del tiempo. Al tomar la media geométrica en lugar de una suma ponderada, HOTA exige que un algoritmo sea bueno en ambos aspectos simultáneamente: un valor muy alto en uno de los dos componentes no puede compensar un valor muy bajo en el otro, a diferencia de lo que ocurriría con una media aritmética.

Una particularidad adicional de HOTA, relevante para la interpretación de los resultados de este proyecto (capítulo 7), es que ambos componentes se calculan y promedian sobre un rango de umbrales de solapamiento IoU, en lugar de sobre un único umbral fijo como hacen MOTA e IDF1, que en TrackEval, se calculan típicamente al 50 % de IoU. Esta característica hace de HOTA una métrica más exigente y completa, ya que resume el comportamiento del algoritmo tanto ante exigencias de solapamiento laxas como estrictas, y es la razón por la que se ha adoptado como métrica principal de comparación en este proyecto.

2.7. Detección del balón

Aunque, como se ha explicado en la sección 2.3, este proyecto no aborda la detección de objetos (se parte de `det.txt` ya calculado), existe un matiz importante: SoccerNet-Tracking incluye en ese mismo fichero las detecciones del balón mezcladas con las de jugadores y árbitros, sin distinguir explícitamente de qué tipo de objeto se trata cada detección. Separar el balón del resto de objetos es, por tanto, un paso de preprocesamiento necesario antes de aplicar cualquier algoritmo de tracking, ya que el balón sigue una dinámica de movimiento completamente distinta a la de una persona, es más rápido y con rebotes. Por tanto, no tiene sentido tratarlo con el mismo filtro de Kalman ni contarle en las métricas de tracking de personas.

La primera aproximación explorada para resolver este paso fue emplear un detector de objetos entrenado con aprendizaje profundo, concretamente **YOLO** (*You Only Look Once*, 22, 23) en su versión ligera YOLOv8n, aplicado directamente sobre los frames para localizar el balón de forma independiente al `det.txt` del dataset. Este enfoque se descartó tras las primeras pruebas: el modelo, sin reentrenar específicamente para balones de fútbol en las condiciones de cámara e iluminación de las retransmisiones de SoccerNet, obtenía resultados muy pobres, tanto falsos negativos (balón no detectado) como falsos positivos sobre otros objetos redondeados y claros del campo, y entrenarlo específicamente para esta única tarea auxiliar quedaba fuera del alcance proporcional de este proyecto.

Se optó, en su lugar, por un **filtro geométrico** simple aplicado directamente sobre las detecciones ya existentes en `det.txt`: una detección se clasifica como balón si su área en píxeles es menor que un umbral y su proporción ancho/alto es próxima a 1 (es decir, aproximadamente cuadrada, coherente con la silueta de un balón visto desde prácticamente cualquier ángulo), condiciones que rara vez cumple una persona. Este filtro, con los umbrales exactos empleados, se detalla en la sección 6.3 del capítulo 6; se trata de una heurística simple y con limitaciones conocidas (puede confundir a un jugador muy lejano y pequeño con el balón), pero suficiente para el propósito de excluir al balón del tracking de personas sin necesidad de entrenar un modelo adicional.

2.8. Clasificación no supervisada de jugadores por equipo: K-means y el árbitro

Una vez obtenidas las trayectorias de los jugadores (capítulo 6), buena parte de los metadatos deportivos de interés como mapas de calor por equipo, posesión por equipo (sección 6.6) requieren saber a qué equipo pertenece cada jugador. SoccerNet-Tracking no proporciona esa información directamente, por lo que este proyecto la infiere a partir del color dominante de la camiseta de cada jugador, mediante **K-means**.

K-means [24] es un algoritmo de **aprendizaje no supervisado**: a diferencia de los algoritmos de aprendizaje supervisado que aprenden a partir de ejemplos ya etiquetados con la respuesta correcta (sección 2.1), K-means agrupa datos sin ninguna etiqueta previa, únicamente a partir de su similitud mutua. Dado un conjunto de puntos (en este proyecto, colores representados como vectores de tres componentes en el espacio BGR) y un número k de grupos deseado, el algoritmo procede de forma iterativa:

1. Se inicializan k centroides (puntos representativos de cada grupo), habitualmente de forma aleatoria o mediante alguna heurística de inicialización.
2. **Asignación**: cada punto del conjunto de datos se asigna al centroide más cercano (típicamente, en distancia euclídea).
3. **Actualización**: cada centroide se recalcula como la media de todos los puntos que se le han asignado.
4. Los pasos 2 y 3 se repiten hasta que las asignaciones dejan de cambiar (convergencia) o se alcanza un número máximo de iteraciones.

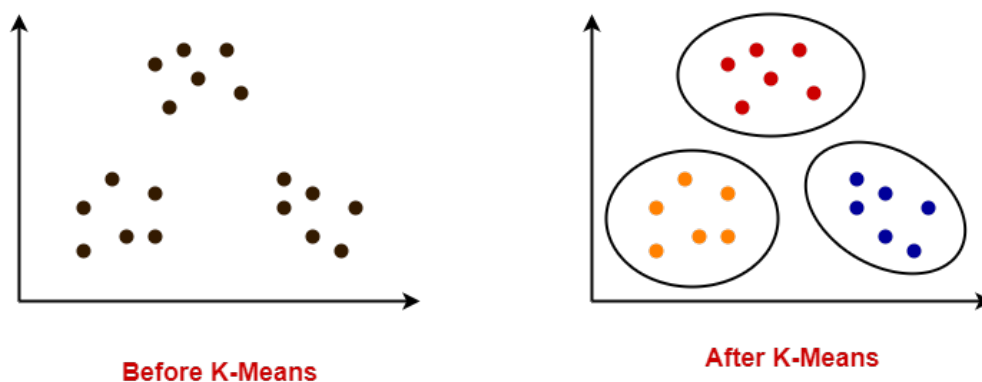


Figura 2.7: Efecto de K-means sobre un conjunto de puntos: antes del agrupamiento (izquierda) y después de converger a $k = 3$ grupos, cada uno alrededor de su centroide (derecha) [25].

Formalmente, lo que este procedimiento iterativo minimiza es la suma de las distancias euclídeas al cuadrado entre cada punto y el centroide de su grupo [24]:

$$\arg \min_S \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2, \quad (2.3)$$

donde S_i es el conjunto de puntos asignados al grupo i -ésimo y μ_i su centroide. El paso de **actualización** de cada centroide como la media de sus puntos asignados es, precisamente, el que reduce este error en cada iteración, hasta alcanzar un mínimo local.

En este proyecto, K-means se aplica en dos niveles, detallados con sus parámetros concretos en la sección 6.6: primero, con $k = 1$, para extraer el color dominante de la camiseta de cada detección individual de jugador (el color medio de una región recortada del torso); después, con $k = 2$ sobre el conjunto de todos esos colores dominantes, asumiendo que existen exactamente dos equipos con colores de camiseta distintos entre sí.

Esa asunción de $k = 2$ es, precisamente, la limitación más relevante de este enfoque y una de las cosas consideradas pero no resueltas en este proyecto: el árbitro, con una camiseta de color distinto a la de ambos equipos, no encaja en ninguno de los dos grupos y termina asignado por K-means al centroide más próximo por color, es decir, a uno de los dos equipos de forma incorrecta. Se valoró pero no se llegó a implementar una variante con $k = 3$ combinada con una heurística adicional (el grupo con menos detecciones totales, dado que solo hay un árbitro principal frente a unos once jugadores por equipo, se etiquetaría como árbitro en lugar de como tercer equipo); esta mejora queda documentada como línea de trabajo futuro en el capítulo 10.

2.9. Homografía y proyección de coordenadas

Las posiciones de los jugadores que produce el tracking están expresadas en **coordenadas de píxel** de la imagen de la cámara de retransmisión: un sistema de referencia bidimensional propio de cada frame, sin relación directa con las distancias reales sobre el terreno de juego. Para poder calcular metadatos con sentido físico (distancia recorrida en metros, velocidad en km/h, o simplemente representar las posiciones sobre un dibujo del campo con proporciones correctas (sección 6.6)) es necesario transformar esas coordenadas de píxel a **coordenadas reales del campo**, expresadas en metros sobre un plano cenital.

Esa transformación, cuando la cámara se puede aproximar como una proyección perspectiva pura sobre un plano. El terreno de juego es, a efectos prácticos, plano, se describe matemáticamente mediante una **homografía**: una matriz H de 3×3 que relaciona las coordenadas homogéneas de un punto en el plano imagen, $\mathbf{p} = (x, y, 1)^\top$, con las coordenadas homogéneas del punto correspondiente en el plano del campo, $\mathbf{p}' = (x', y', 1)^\top$, mediante

$$\mathbf{p}' \sim H \mathbf{p}, \quad (2.4)$$

donde $\sim$ indica igualdad salvo un factor de escala, propio de las coordenadas homogéneas [27]. La matriz H tiene 8 grados de libertad (9 elementos menos 1 por el factor de escala libre), por lo que puede calcularse de forma exacta a partir de 4 pares de puntos correspondientes conocidos –un punto en la imagen y su posición real equivalente en el campo– mediante un sistema de ecuaciones lineales; disponer de más

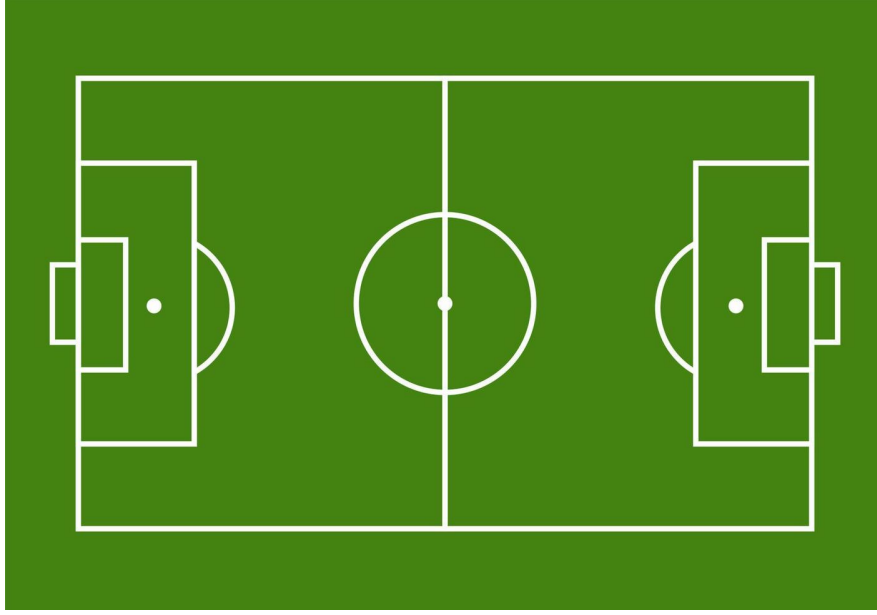


Figura 2.8: Campo de fútbol estándar visto en planta [26]. Las medidas FIFA reglamentarias empleadas como referencia para la homografía y para los mapas de calor de este proyecto (sección 6.6) son: $105\text{ m} \times 68\text{ m}$ de terreno de juego, área grande $16,5\text{ m} \times 40,3\text{ m}$, área pequeña $5,5\text{ m} \times 18,3\text{ m}$, círculo central de $9,15\text{ m}$ de radio.

de 4 correspondencias permite, además, calcular H por mínimos cuadrados y obtener una estimación más robusta frente a errores de medición en los puntos de referencia.

En un escenario ideal, en el que la cámara de retransmisión permaneciera completamente fija durante todo el partido, bastaría con calcular H una única vez, identificando manualmente 4 puntos de referencia reconocibles en el campo, por ejemplo, las esquinas del área pequeña, cuyas medidas FIFA son conocidas y fijas ($5,5\text{ m} \times 18,3\text{ m}$, Figura 2.8) y sus coordenadas de píxel correspondientes en un frame representativo. Esta es, de hecho, la aproximación seguida en este proyecto, pero limitada deliberadamente a tramos concretos de vídeo con cámara estable, dado que las cámaras de retransmisión real de SoccerNet se mueven de forma continua (paneos, zooms) a lo largo de toda la secuencia (lo que exigiría recalcular H de forma dinámica, frame a frame, un problema de calibración de cámara considerablemente más complejo y fuera del alcance de este proyecto). Las implicaciones prácticas de esta limitación, y la alternativa de proyección aproximada empleada para el resto del dataset, se detallan en la sección 6.6.3.



Figura 2.9: Puntos de referencia marcados sobre un frame real de retransmisión, usados para calcular la homografía: las cuatro esquinas del área pequeña, cuyas coordenadas de campo son conocidas por las medidas FIFA (Figura 2.8).

2.10. Trabajos relacionados y posicionamiento de este TFG

El propio ecosistema SoccerNet organiza, además del dataset, un reto anual de seguimiento multiobjeto (*SoccerNet Player Tracking Challenge*) con un servidor de evaluación público y un *leaderboard* de equipos participantes, lo que ofrece el punto de comparación más directo y verificable para situar los resultados de este proyecto frente al estado del arte práctico.

La edición 2023 del reto [28], con 7 equipos y 83 envíos, resulta muy informativa sobre qué enfoques resultan competitivos en la práctica: el equipo ganador (Kalisteo) alcanzó $\text{HOTA} = 75,61$ combinando un detector YOLO-X afinado con un tracker propio basado en el algoritmo húngaro y un filtro de Kalman con compensación de movimiento de cámara; y, de forma especialmente relevante para las decisiones de este proyecto, varios de los equipos mejor clasificados construyeron su solución precisamente sobre **ByteTrack** o la familia **SORT/OC-SORT**, por ejemplo, SAIVA Tracking sobre ByteTrack, ZTrackers sobre YOLOv5 combinado con ByteTrack, y MOT4MOT sobre DeepOCSORT, lo que confirma que ambos algoritmos constituyen una base representativa y competitiva del estado del arte práctico.

Puesto en perspectiva, el HOTA integrado en test obtenido en este proyecto con OC-SORT (75,43), resulta muy próximo al del equipo ganador del reto oficial 2023 (75,61) pese a partir de una posición menos favorable en dos aspectos: sin ningún entrenamiento ni ajuste de detector específico para fútbol, y con los algoritmos de tracking en su configuración por defecto para esa comparación concreta. No se trata, por tanto, de una comparación estrictamente equivalente ni de una pretensión de superar

el estado del arte del reto, sino de una referencia que sitúa los resultados de este TFG en un rango realista y defendible frente a soluciones desarrolladas específicamente para competir.

La aportación de este proyecto no busca, en consecuencia, competir por la mejor posición en ese *leaderboard*, sino dos objetivos distintos que el propio reto no cubre: primero, una comparación controlada y metodológicamente rigurosa entre dos algoritmos clásicos de *tracking-by-detection* bajo condiciones idénticas: mismas detecciones, mismas métricas, mismo estudio sistemático de hiperparámetros, que documenta explícitamente una discrepancia metodológica entre HOTA y HOTA(0) no discutida en los informes de resultados del propio reto; y segundo, la construcción de un pipeline completo de extracción de metadatos deportivos interpretables y su exposición en una aplicación web, que va más allá de la propia tarea de tracking en la que se detiene el reto oficial.

3. Gestión del proyecto

Todo proyecto de ingeniería del software requiere, además de la resolución técnica del problema planteado, una gestión del propio proceso de trabajo: qué metodología se sigue, cómo se reparte el esfuerzo en el tiempo, qué recursos se necesitan y qué riesgos pueden comprometer el resultado final. Este capítulo documenta esa gestión para el presente proyecto: la metodología de trabajo adoptada, las fases en que se ha dividido el desarrollo, los roles ejercidos, la planificación temporal (inicial y revisada), la estimación de recursos y presupuesto, y los planes de gestión de riesgos y de calidad aplicados.

3.1. Metodología de trabajo

Este proyecto se ha desarrollado siguiendo un enfoque iterativo e incremental, guiado por instrucciones de seguimiento con el tutor. En lugar de fijar de antemano una especificación cerrada de todo el sistema, cada bloque de trabajo se ha planteado como una iteración corta que produce un resultado funcional y evaluable, cuyas conclusiones determinan el siguiente paso. Este enfoque es especialmente adecuado en un proyecto como este, en el que algunas decisiones como qué algoritmos comparar, qué metadatos extraer, hasta dónde llevar la calibración de cámara dependían de resultados intermedios que no podían conocerse de antemano: no tenía sentido, por ejemplo, comprometerse a un pipeline completo de metadatos antes de comprobar que la comparativa de algoritmos de tracking daba resultados sólidos y defendibles.

Dentro de ese marco iterativo, cada ciclo de trabajo ha seguido aproximadamente la misma secuencia: (1) plantear una pregunta concreta o un objetivo acotado a partir de las conclusiones del ciclo anterior y, en las ideas claves de las instrucciones del tutor; (2) implementar la solución mínima que permita responder a esa pregunta; (3) evaluar el resultado con datos reales, normalmente mediante las métricas MOT estándar descritas en el capítulo 2; y (4) plantear la idea y decisiones tomadas al tutor y documentar la decisión tomada (incluida la opción de descartar el enfoque probado) antes de pasar al siguiente ciclo. Este último punto ha sido determinante para la redacción de esta memoria: al quedar registrada la razón de cada decisión en el momento en que se tomó, ha sido posible construir la memoria de forma detallada, incluyendo no solo los resultados finales, sino también el proceso seguido para llegar hasta ellos.

El seguimiento del trabajo entre reuniones se ha apoyado en un documento de anotaciones propio, actualizado de forma continua, donde se registran decisiones tomadas, resultados numéricos, problemas de entorno encontrados y su solución, e ideas evaluadas y descartadas. Este documento ha sido la fuente principal para poder documentar todo el proceso de desarrollo en esta memoria.

3.2. Fases del proyecto

Sobre la metodología iterativa descrita, el trabajo se ha organizado en siete fases principales, que se recorrieron de forma mayoritariamente secuencial pero con solapamientos puntuales entre fases adyacentes; en particular entre la fase 4 (estudio de hiperparámetros) y la fase 5 (pipeline de metadatos), ya que la configuración óptima de cada algoritmo se fue consolidando mientras ya se trabajaba en los primeros metadatos, y entre la fase 5 y la fase 7 (redacción de la memoria), que se ha llevado en paralelo a las últimas iteraciones técnicas en lugar de posponerse por completo al final del proyecto.

La primera fase correspondió a la exploración de las distintas tareas que ofrece el ecosistema SoccerNet antes de comprometerse con ninguna de ellas, descrita con más detalle en la sección 4.1.1 del capítulo 4. Una vez elegido el tracking como tarea del proyecto, las fases 2 y 3 cubrieron la implementación de los dos algoritmos comparados y su evaluación con métricas estándar, primero sobre el conjunto de entrenamiento y, a petición del tutor, también sobre el conjunto de test. Las fases 4 y 5 constituyen el núcleo experimental del proyecto: el estudio sistemático de hiperparámetros y el diseño del pipeline de extracción de metadatos deportivos sobre la configuración óptima resultante. La fase 6, desarrollo de la aplicación web, se completó como un MVP fiel al diseño planificado en el capítulo 5, según se detalla en la sección 3.5.4. La Tabla 3.1 resume estas siete fases, sus actividades principales y el resultado obtenido en cada una.

Tabla 3.1: Resumen de las fases del proyecto.

Fase del proyecto	Actividades principales	Resultado obtenido
1. Exploración inicial	Estudio de diferentes tareas alternativas de SoccerNet (reconocimiento de dorsales, segmentación de cámara, tracking) y primeras pruebas de visualización sobre el <i>ground truth</i>	Elección de la tarea (tracking) y del enfoque de asociación sobre detecciones precomputadas
2. Implementación de trackers	ByteTrack y OC-SORT con boxmot , filtrado geométrico del balón, ejecución sobre el conjunto de entrenamiento	Resultados de tracking para las 57 secuencias de train
3. Evaluación	Integración de TrackEval, cálculo de HOTA/MOTA/IDF1 sobre train y, posteriormente, sobre test	Comparativa inicial ByteTrack vs. OC-SORT
4. Estudio de hiperparámetros	Barrido uno a uno de los parámetros de cada algoritmo sobre test	Configuración óptima de ByteTrack y de OC-SORT
5. Pipeline de metadatos	Clasificación de equipos, mapas de calor, homografía (distancia/velocidad), posesión de balón	Metadatos deportivos extraídos sobre la configuración óptima
6. Aplicación web	Desarrollo de una interfaz de demostración y análisis de datos	MVP funcional
7. Redacción de la memoria	Documentación de todo el proceso, en paralelo a las últimas iteraciones técnicas	Este documento

3.3. Roles y participantes

El proyecto ha sido realizado por una única persona, Carlos, que ha asumido de forma sucesiva los distintos roles que en un contexto profesional de mayor envergadura recaerían en perfiles y personas distintas. Definir estos roles explícitamente, aunque el proyecto sea individual, tiene dos utilidades prácticas: por un lado, permite razonar sobre el reparto real del esfuerzo entre tipos de tarea muy distintos entre sí (no es lo mismo el tiempo dedicado a ajustar un hiperparámetro que el dedicado a redactar un requisito o a depurar un error de entorno); por otro, sirve de base para la estimación de costes de la sección 3.7, que reparte las horas totales del proyecto entre estos mismos roles con una tarifa de mercado distinta para cada uno.

A continuación se describe cada rol, con su definición general –qué función cumple ese perfil en un proyecto de software típico– y la función concreta que ha desempeñado dentro de este proyecto:

- **Jefe de proyecto.** Es el responsable de la planificación temporal, del seguimiento del avance frente a lo planificado y de la gestión de riesgos del proyecto; en un contexto de equipo, también coordina al resto de perfiles y es el interlocutor principal con el cliente o, en este caso, con el tutor académico. En este proyecto, este rol ha cubierto la elaboración y revisión de la planificación (sección 3.5), la preparación y el seguimiento de las reuniones con el tutor, la gestión de riesgos (sección 3.8) y, en la fase final, la organización y redacción de esta memoria.
- **Analista de software.** Es el responsable de traducir las necesidades del proyecto en una especificación formal: actores, requisitos, reglas de negocio y casos de uso, de forma que el alcance del sistema quede acotado y sea trazable frente a los objetivos del proyecto. En este proyecto, este rol se ha ejercido principalmente en la elaboración del capítulo 4, definiendo el catálogo de requisitos y la matriz de trazabilidad que lo cierra.
- **Ingeniero de Visión Artificial.** Es el perfil técnico especializado en el diseño, implementación y evaluación de algoritmos de visión por computador; en este proyecto equivale al rol que en un proyecto de ciencia de datos más general correspondería a un científico de datos, adaptado específicamente al dominio del seguimiento multiobjeto. Ha sido, el rol con mayor dedicación del proyecto: implementación y comparación de los algoritmos de tracking, diseño e interpretación del estudio de hiperparámetros, y diseño del pipeline de extracción de metadatos, incluyendo la decisión de qué enfoques quedaban únicamente como prueba de concepto y cuáles se generalizaban al resto del dataset.
- **Desarrollador.** Es el responsable de la implementación del código del sistema, de su organización en un repositorio mantenible y de la corrección de errores. En este proyecto, este rol ha cubierto la implementación de los scripts del repositorio y su organización en los paquetes correspondientes, así como la implementación de la aplicación web(capítulo 6).

La Tabla 3.2 resume estos cuatro roles junto con sus tareas asociadas, a modo de referencia rápida que se retoma en la sección 3.7 para estimar el coste del proyecto en

un contexto profesional.

Tabla 3.2: Roles asumidos durante el proyecto y sus tareas asociadas.

Rol	Tareas asociadas
Jefe de proyecto	Planificación temporal, seguimiento del avance, redacción y cierre de la memoria.
Analista de software	Definición del alcance funcional, catálogo de requisitos, casos de uso y matriz de trazabilidad (capítulo 4).
Ingeniero de Visión Artificial	Implementación y comparación de los algoritmos de tracking, diseño del estudio de hiperparámetros, diseño del pipeline de metadatos (equipos, heatmaps, homografía, posesión).
Desarrollador	Implementación y organización de los scripts del repositorio, corrección de errores y desarrollo de la aplicación web.

Además de estos cuatro roles internos, el tutor del proyecto, Manuel Carranza García, del departamento de Lenguajes y Sistemas Informáticos de la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Sevilla [29], ha ejercido de supervisor a lo largo de todo el desarrollo mediante reuniones periódicas de seguimiento. Su papel ha sido de gran importancia, ya que muchas de las decisiones clave del proyecto responden directamente a su orientación, entre ellas, la propuesta de las tres líneas de ampliación que estructuran buena parte de la segunda mitad del proyecto: el estudio de hiperparámetros, el pipeline de metadatos y la aplicación web.

3.4. Estructura de descomposición del trabajo (EDT)

La estructura de descomposición del trabajo (EDT, o *Work Breakdown Structure*) divide el proyecto en unidades de trabajo cada vez más pequeñas y manejables, hasta llegar a paquetes de trabajo lo bastante concretos como para poder estimarles una duración y asignarles un responsable. Se organiza en tres niveles: el nivel superior corresponde al proyecto completo, el nivel intermedio agrupa las fases descritas en la sección 3.2, y el nivel inferior recoge los paquetes de trabajo concretos dentro de cada fase. La Figura 3.1 representa esta descomposición de forma gráfica.

Cada paquete de trabajo del nivel inferior se detalla en el llamado diccionario de la EDT, que recoge para cada uno su descripción, el rol responsable (sección 3.3) y las horas dedicadas. La Tabla 3.3 recoge este diccionario completo, ya consolidado con las horas realmente dedicadas al cierre del proyecto –no solo las previstas en la planificación inicial, ampliadas en varios paquetes por las causas descritas en la sección 3.5.4–; las horas por rol coinciden, sumadas, con las horas totales por rol de la Tabla 3.7 de la sección 3.7.

Tabla 3.3: Diccionario de la EDT: paquetes de trabajo, rol responsable y horas totales.

ID	Paquete de trabajo	Rol	Horas
0.1	Traslados y reconfiguración del entorno de desarrollo (portátil, sobremesa, macOS)	Desarrollador	10 h
1.1	Exploración de tareas de SoccerNet, valoración de alternativas y primeras pruebas de visualización	Ingeniero de Visión Artificial	20 h
1.2	Comprensión del formato del dataset	Ingeniero de Visión Artificial	5 h
1.3	Análisis de requisitos (capítulo 4)	Analista de software	15 h
2.1	Implementación de ByteTrack y OC-SORT sobre boxmot	Ingeniero de Visión Artificial	15 h
2.2	Filtrado geométrico del balón y ejecución batch sobre train	Ingeniero de Visión Artificial	10 h
3.1	Integración de TrackEval y evaluación sobre train	Ingeniero de Visión Artificial	20 h
3.2	Evaluación sobre test	Ingeniero de Visión Artificial	10 h
4.1	Estudio de hiperparámetros de ByteTrack	Ingeniero de Visión Artificial	15 h
4.2	Estudio de hiperparámetros de OC-SORT	Ingeniero de Visión Artificial	15 h
5.1	Clasificación de equipos y mapas de calor (con y sin homografía)	Ingeniero de Visión Artificial	20 h
5.2	Homografía real (distancia/velocidad) y posesión de balón	Ingeniero de Visión Artificial	25 h
6.1	Desarrollo de la aplicación web	Desarrollador	40 h
6.2	Ajustes y revisión de la aplicación web (retoques de septiembre)	Desarrollador	15 h
8.1	Organización del repositorio y corrección de errores	Desarrollador	20 h
9.1	Redacción de la memoria	Jefe de proyecto	65 h
	Total		320 h

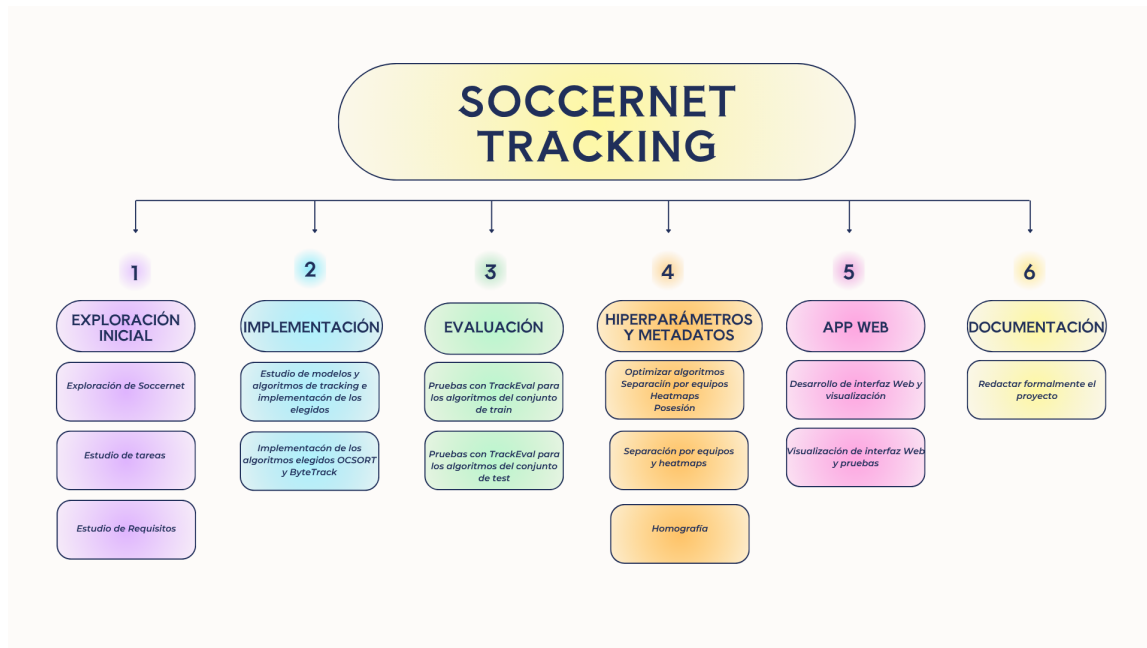


Figura 3.1: Estructura de descomposición del trabajo (EDT) del proyecto.

Esta cifra final, superior a las 235h estimadas en la planificación inicial (sección 3.5), no es un error de previsión menor sino el reflejo de cómo evolucionó realmente el proyecto: la sección 3.5.4 detalla las causas concretas de esa desviación de horas.

3.5. Planificación temporal

3.5.1. Elaboración del plan de trabajo

El proyecto se inició a finales de abril de 2026, con una disponibilidad de tiempo muy variable a lo largo de los meses siguientes debido al solapamiento con la época de exámenes del resto de asignaturas del grado. Esta irregularidad, previsible desde el principio dado el calendario académico, condicionó la forma de planificar: en lugar de repartir el esfuerzo de manera uniforme mes a mes, la planificación inicial asumió explícitamente periodos de baja dedicación durante los exámenes de mayo y concentró el grueso del trabajo en los meses sin solapamiento académico. La planificación inicial, elaborada durante las primeras semanas del proyecto, fue la siguiente:

- **Abril (20–30)** – Arranque. Exploración de tareas y elección definitiva de la tarea (tracking). Primeros scripts de visualización. Implementación de ByteTrack y OC-SORT. Ejecución sobre las secuencias de entrenamiento. Preparación del entorno de evaluación con TrackEval.
- **Mayo** – Ritmo muy reducido por exámenes (aproximadamente 20 horas en todo el mes, con prácticamente inactividad entre el 7 y el 25). Objetivo: obtener las primeras métricas reales y mantener una reunión de seguimiento con el tutor.

- **Junio** – Núcleo del proyecto (unas 120 horas estimadas): evaluación en test, estudio de hiperparámetros, pipeline de metadatos, y comienzo de la redacción de la memoria en paralelo.
- **Julio** – Cierre (unas 80 horas estimadas): desarrollo de la aplicación web y finalización de la memoria.

La estimación total de esta planificación inicial fue de unas 235 horas, de las cuales aproximadamente 65 se reservaron como margen para imprevistos y tiempo de tutorías, con una fecha de entrega prevista a finales de julio de 2026. La Tabla 3.4 representa esta misma planificación en forma de cronograma, situando cada fase del proyecto (sección 3.2) sobre el mes o los meses en que se ha concentrado su desarrollo, incluyendo la revisión de la planificación descrita en la sección 3.5.4.

Tabla 3.4: Cronograma real del proyecto por fase y mes. El tono claro distingue actividad parcial o de repaso/retoque frente al grueso del trabajo de cada fase (tono oscuro).

Fase	Abr	May	Jun	Jul	Ago	Sep	Oct
1. Exploración inicial	✓						
2. Implementación de trackers	✓						
3. Evaluación	✓	✓					
4. Estudio de hiperparámetros			✓				
5. Pipeline de metadatos			✓	✓			
6. Aplicación web					✓	✓	
7. Redacción de la memoria			✓	✓	✓	✓	✓

Como refleja el cronograma, la fase 7 no se concentra limpiamente en un único tramo: arranca de forma parcial en junio y julio, se convierte en el grueso real del trabajo en agosto y septiembre, y se cierra con una revisión final en octubre antes de la entrega. La fase 6 sigue un patrón similar a menor escala: desarrollo principal de la aplicación web en agosto, con retoques y correcciones puntuales en septiembre (sección 6.7.3 del capítulo 6).

3.5.2. Seguimiento y monitorización

El seguimiento del tiempo dedicado a este proyecto se ha apoyado en una herramienta de *time-tracking* dedicada (Figura 3.2), complementada por el documento de anotaciones descrito en la sección 3.1 y por el historial de commits del repositorio Git, cuya fecha permite además reconstruir con razonable precisión

cuándo se completó cada hito (Tabla 3.5). Las horas totales por paquete de trabajo de la Tabla 3.3 y la desviación cuantificada en la sección 3.5.4 proceden de esta herramienta.

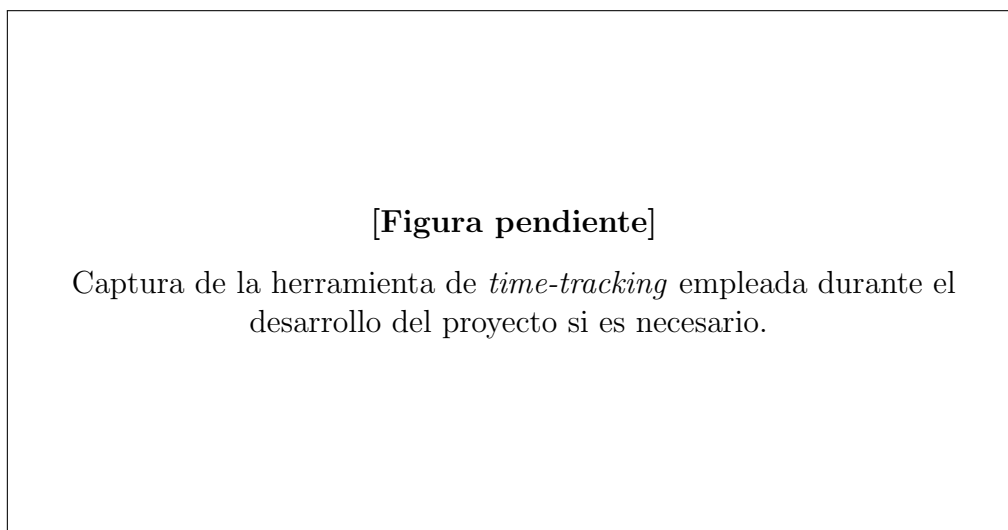


Figura 3.2: (pendiente de incorporar la captura).

3.5.3. Hitos principales

La Tabla 3.5 resume los hitos principales alcanzados hasta el cierre de este documento, reconstruidos a partir del documento de anotaciones y del historial de commits del repositorio. Las fechas de junio en adelante son aproximadas, ya que varias tareas se solaparon durante el desarrollo, como es propio de la metodología iterativa descrita en la sección 3.1.

3.5.4. Revisión de la planificación

Como es habitual en un proyecto de esta naturaleza, la planificación inicial no se ha cumplido de forma literal, y merece la pena detenerse en por qué, ya que las causas de la desviación son en sí mismas informativas sobre el trabajo realizado. Se identifican cinco causas principales, ninguna de ellas aislada, sino acumuladas a lo largo del proyecto:

- **Alcance técnico mayor de lo previsto:** el estudio de hiperparámetros y el pipeline de metadatos resultaron más extensos de lo estimado inicialmente. Cada nuevo metadato (clasificación de equipos, mapas de calor, homografía, posesión) exigió varias iteraciones de prueba y ajuste antes de dar resultados fiables, un coste que la planificación inicial (elaborada antes de conocer las dificultades reales de calibrar una homografía manual o de separar correctamente al árbitro en la clasificación por color) no pudo anticipar.
- **Redacción de la memoria más tardía y más costosa de lo previsto:** la planificación inicial contemplaba escribir la memoria en paralelo desde junio y

Tabla 3.5: Hitos principales del proyecto.

Fecha	Hito
20–25 abr. 2026	Exploración de tareas de SoccerNet y elección de tracking como tarea del proyecto.
26 abr. 2026	OC-SORT y ByteTrack implementados sobre boxmot.
28–29 abr. 2026	<code>run_all_train.py</code> ejecutado sobre las 57 secuencias de train; primera evaluación con TrackEval y exportación de vídeos anotados.
Mayo 2026	Evaluación repetida sobre el conjunto de test a petición del tutor; traslado del proyecto a un equipo macOS.
Junio 2026	Estudio de hiperparámetros de ByteTrack y OC-SORT; configuración óptima encontrada para ambos algoritmos.
Junio–julio 2026	Pipeline de metadatos: mapa de calor general, clasificación de equipos por color y mapas de calor por equipo.
Julio 2026	Homografía real sobre tramos de cámara estable (distancia y velocidad); posesión de balón por proximidad.
Agosto 2026	Revisión de la planificación, traslado de la entrega a la convocatoria de octubre; desarrollo de la aplicación web; grueso de la redacción sistemática de la memoria.
Septiembre 2026	Ajustes y correcciones sobre la aplicación web; continuación y revisión de la memoria.
Octubre 2026	Revisión final de la memoria y entrega.

cerrarla en julio (sección 3.5). En la práctica, la redacción sistemática –con el nivel de detalle y de contraste contra el código real que exige esta memoria– no arrancó en serio hasta agosto, y el grueso del trabajo se concentró entre julio y agosto, con continuación y revisión en septiembre y una última revisión en octubre. La documentación continua de decisiones y resultados intermedios, necesaria para poder redactar con este nivel de detalle, ha supuesto por sí misma un esfuerzo no contemplado en la planificación inicial.

- **Periodo de exámenes y viajes durante el verano:** además del solapamiento ya previsto con los exámenes de mayo (sección 3.5), varios periodos de viaje durante el verano redujeron la disponibilidad real de tiempo en semanas concretas, sin que se hubiera reservado margen explícito para ello en la planificación inicial.
- **Retraso inicial por la exploración de alternativas:** la fase de exploración inicial de tareas de SoccerNet (sección 3.2) llevó más tiempo del previsto, al valorarse varias alternativas antes de comprometerse con el tracking como tarea del proyecto (sección 4.1.1).
- **Incidencias con el equipo de desarrollo:** un problema con el portátil personal con el que se inició el proyecto obligó a migrar a un portátil macOS a mitad del desarrollo, en paralelo al uso ya habitual de un equipo de sobremesa (sección 3.6); recrear el entorno en cada cambio de equipo consumió tiempo no planificado.

Por este motivo, de acuerdo con el tutor, la entrega del proyecto se ha trasladado de la convocatoria de julio a la **convocatoria de octubre de 2026**, lo que ha permitido completar con mayor profundidad tanto el pipeline de metadatos como la propia memoria, en lugar de recortar alcance para cumplir la fecha original. Como consecuencia directa de estas causas, la dedicación real asciende a las **320 h** recogidas en el diccionario de la EDT (Tabla 3.3), 85h por encima de las 235h de la planificación inicial. La Tabla 3.6 resume las principales desviaciones respecto al plan inicial.

Tabla 3.6: Principales desviaciones respecto a la planificación inicial.

Elemento	Plan inicial	Desarrollo real
Estudio de hiperparámetros	Barrido rápido, sin cuantificar horas de antemano	Varias iteraciones por algoritmo hasta encontrar una configuración óptima defendible
Pipeline de metadatos	Un mapa de calor general	Equipos, heatmaps por equipo, homografía (distancia/velocidad) y posesión
Redacción de la memoria	En paralelo desde junio, cerrada en julio	Grueso del trabajo entre julio y agosto, continuación en septiembre y revisión final en octubre
Aplicación web	Prevista en julio	Desarrollada en agosto de 2026, con retoques en septiembre, tras completar el pipeline de metadatos y en paralelo a la redacción de los últimos capítulos de esta memoria
Horas totales dedicadas	235 h (planificación inicial)	320 h (Tabla 3.3)
Fecha de entrega	Convocatoria de julio de 2026	Convocatoria de octubre de 2026

3.6. Entorno de desarrollo

El desarrollo se ha realizado en Python 3.11, dentro de un entorno virtual (`venv_tfg`) aislado dentro del propio repositorio del proyecto, con las dependencias fijadas en `requirements.txt` para garantizar la reproducibilidad de los resultados de forma correcta. Las herramientas principales empleadas son OpenCV (lectura y anotación de frames), `boxmot` (implementaciones de los algoritmos de tracking), `trackeval` (evaluación con métricas MOT estándar), `scikit-learn` (clasificación de equipos por color mediante K-means) y `matplotlib` (generación de gráficas y mapas de calor). El desglose completo de versiones y la justificación de las decisiones técnicas más relevantes sobre el entorno (incluyendo los problemas encontrados durante su configuración) se detalla en la sección 6.1 del capítulo 6.

El proyecto se inició en un portátil personal HP Pavilion de 8 GB de RAM, y a mitad del desarrollo se trasladó a un ordenador de sobremesa con GPU AMD Radeon RX 6900 XT (16 GB VRAM) como entorno principal. Un problema con el portátil original obligó, más adelante, a migrar a un portátil macOS, usado desde entonces en paralelo al equipo de sobremesa según la disponibilidad; recrear el entorno de desarrollo en cada uno de estos cambios de equipo (problema P15, sección 6.1) consumió tiempo no contemplado en la planificación inicial (sección 3.5.4). Dado que los algoritmos de tracking utilizados en este proyecto (ByteTrack, OC-SORT) son clásicos y no requieren redes neuronales de gran tamaño en tiempo de inferencia, todo el desarrollo se ha realizado sobre CPU sin que ello supusiera una limitación relevante; la GPU AMD, al no ser compatible con CUDA, se planteó como posible aceleración para líneas de trabajo futuras que incorporen re-identificación basada en redes neuronales (véase capítulo 10).

3.7. Estimación de recursos y presupuesto

Al tratarse de un proyecto académico individual, en este caso Trabajo Fin de Grado, no ha existido un presupuesto real asociado a su desarrollo. No obstante, y siguiendo la práctica habitual en la documentación de proyectos de ingeniería del software, se presenta a continuación una estimación de los recursos que hubiera requerido su realización en un contexto profesional, apoyada en los cuatro roles descritos en la sección 3.3 y en el diccionario de la EDT de la sección 3.4.

3.7.1. Costes de personal

Las tarifas por hora empleadas en la Tabla 3.7 proceden de la **Instrucción 2/2020, de 30 de junio, de la Dirección General de Transformación Digital de la Junta de Andalucía, sobre perfiles y precios de referencia de la contratación de bienes y servicios TIC** [30], que fija precios de referencia (IVA excluido) para los perfiles profesionales TIC más comunes según el acuerdo técnico europeo CWA 16458-1:2018. Como esta instrucción no contempla un perfil específico de *ingeniería de visión artificial* —una especialización posterior a esa taxonomía—, se

ha usado como equivalente el perfil de **arquitecto de sistemas**, el más próximo por nivel de especialización técnica; el resto de roles de este proyecto sí tienen un perfil directamente equivalente en la instrucción. La estimación se aplica sobre las horas totales realmente dedicadas (320 h, Tabla 3.3), repartidas por rol de acuerdo con el diccionario de la EDT.

Tabla 3.7: Estimación de costes de personal sobre las 320 h reales dedicadas al proyecto.

Rol	Perfil equivalente (Junta de Andalucía)	Horas	€/h	Coste
Jefe de proyecto	Gestor de proyecto	65 h	44,12 €	2 867,80 €
Analista de software	Analista de negocio	15 h	33,17 €	497,55 €
Ingeniero de Visión Artificial	Arquitecto de sistemas	155 h	33,45 €	5 184,75 €
Desarrollador	Desarrollador	85 h	25,67 €	2 181,95 €
Total		320 h		10 732,05 €

3.7.2. Costes materiales y operacionales

Además de los costes de personal, un proyecto de estas características conllevaría los costes materiales y operacionales que recoge la Tabla 3.8. La mayoría son recursos sin coste directo, por tratarse de equipo ya disponible o de herramientas de código abierto; el único coste realmente cuantificable es el consumo eléctrico, calculado a partir de una potencia media de consumo de 150 W –una cifra intermedia entre el portátil y el equipo de sobremesa descritos en la sección 3.6, ya que no todas las 320 h de dedicación se han realizado con el equipo de mayor consumo– y las 320 h totales del proyecto (Tabla 3.3): $150 \text{ W} \times 320 \text{ h} = 48 \text{ kWh}$, que al precio medio doméstico de la electricidad en España (en torno a 0,16 €/kWh) supone un coste estimado de **7,68 €** a lo largo de todo el proyecto.

Este bajo coste de infraestructura es, precisamente, uno de los factores que hace atractivo el enfoque seguido en este proyecto (véase la justificación de la elección de tarea en la sección 4.1.1): al partir de detecciones ya calculadas y de algoritmos de tracking clásicos disponibles como librería, el proyecto es reproducible por terceros sin inversión en infraestructura específica.

3.7.3. Desviación respecto al coste planificado

Como se describe en la sección 3.5.4, la dedicación real (320 h) ha superado en 85 h la planificación inicial de 235 h, principalmente por el mayor alcance del estudio de

Tabla 3.8: Costes materiales y operacionales estimados.

Concepto	Coste
Hardware (equipo de desarrollo ya disponible, sin GPU dedicada necesaria, sección 3.6)	Sin coste incremental
Software y librerías (Python, OpenCV, <code>boxmot</code> , <code>trackeval</code> , <code>scikit-learn</code> , <code>matplotlib</code> , Git)	0 € (código abierto)
Dataset (SoccerNet-Tracking)	0 € (acceso académico gratuito)
Asistentes de inteligencia artificial (Claude Code, capítulo 9)	No cuantificado en esta estimación
Consumo eléctrico ($150\text{ W} \times 320\text{ h} = 48\text{ kWh}$ a $0,16\text{ €/kWh}$)	7,68 €
Total cuantificado	7,68 €

hiperparámetros y del pipeline de metadatos, y por el tiempo dedicado a documentar el proceso con el nivel de detalle de esta memoria. A diferencia de lo que ocurriría sin un registro fiable de horas, en este proyecto sí se ha dispuesto de una herramienta de *time-tracking* dedicada (sección 3.5.2), lo que permite ofrecer una cifra de coste real y no solo una desviación cualitativa: aplicando las mismas tarifas de la Tabla 3.7 sobre el reparto por rol de las 235 h de la planificación inicial (30 h jefe de proyecto, 15 h analista, 130 h ingeniero de Visión Artificial, 60 h desarrollador) y el mismo criterio de consumo eléctrico sobre esas 235 h ($150\text{ W} \times 235\text{ h} = 35,25\text{ kWh}$, a $0,16\text{ €/kWh}$), se obtiene el coste total estimado que recoge la Tabla 3.9, comparado con el coste real final.

Tabla 3.9: Coste total estimado (planificación inicial, 235 h) frente a coste real (320 h) y desviación.

Concepto	Coste estimado	Coste real	Desviación
Personal (Tabla 3.7)	7 709,85 €	10 732,05 €	+3 022,20 €
Consumo eléctrico (Tabla 3.8)	5,64 €	7,68 €	+2,04 €
Total	7 715,49 €	10 739,73 €	+3 024,24 € (+39 %)

La desviación de coste total, en torno al 39 %, es algo mayor que la desviación de horas (85 h sobre 235 h, un 36 %) porque las horas añadidas se concentran en los roles de tarifa más alta (jefe de proyecto e ingeniero de Visión Artificial); el consumo eléctrico, al ser proporcional a las horas totales, se desvía exactamente en esa misma proporción (36 %) pero su magnitud absoluta es tan pequeña que apenas afecta al total.

La Tabla [3.6](#) documenta el resto de desviaciones respecto a la planificación inicial, no solo la de coste.

3.8. Plan de gestión de riesgos

La gestión de riesgos de este proyecto no se ha limitado a una lista elaborada una única vez al inicio, sino que se ha revisado en cada reunión de seguimiento con el tutor, incorporando riesgos nuevos a medida que se hacían evidentes. Por ejemplo, el riesgo asociado a la dependencia de `boxmot` no era previsible hasta que la propia librería cambió de interfaz durante el desarrollo del proyecto. Para cada riesgo se ha estimado su probabilidad e impacto de forma cualitativa (alta/media/baja), se han definido medidas preventivas aplicadas durante el desarrollo. La Tabla [3.10](#) recoge los cinco riesgos principales identificados durante la planificación y el desarrollo del proyecto.

Tabla 3.10: Principales riesgos del proyecto y su mitigación.

Riesgo	Medidas preventivas	Plan de contingencia
Disponibilidad de tiempo irregular por exámenes	Margen amplio en la planificación inicial (aprox. 65 h de colchón)	Traslado de la entrega a la convocatoria de octubre (sección 3.5.4)
Dependencia de <code>boxmot</code> , librería en evolución activa con cambios de interfaz entre versiones	Versión antigua fijada en <code>requirements.txt</code> y código versionado en Git (problema P4, sección 6.1)	Congelar la versión funcional y documentar el problema en vez de migrar a mitad de proyecto
Resultados de calibración de cámara no generalizables a todo el dataset	Acotar expresamente la homografía real a una prueba de concepto sobre tramos con cámara estable (sección 6.6.3)	Documentar la limitación en la memoria en vez de invertir tiempo buscar otras formas.
Cambio de equipo de desarrollo a mitad de proyecto (traslado a macOS)	Dependencias fijadas en <code>requirements.txt</code> y código versionado en Git	Recrear el entorno desde cero en el nuevo equipo (problema P15, sección 6.1)
Redacción de la memoria más tardía y más costosa de lo previsto	Documento de anotaciones actualizado de forma continua durante todo el desarrollo (sección 3.1), que evita depender de la memoria para no perder el registro de decisiones	Traslado de la entrega a la convocatoria de octubre y redacción en paralelo a las últimas iteraciones técnicas (sección 3.5.4)

3.9. Plan de calidad

La calidad de este proyecto no se ha evaluado únicamente por si el código se ejecuta sin errores, sino por un conjunto más amplio de criterios propios de un proyecto de análisis de datos: que los resultados de tracking sean válidos y evaluables con herramientas estándar, que la comparación entre algoritmos y configuraciones sea reproducible, que cada metadato extraído documente explícitamente sus limitaciones en lugar de presentarse como una medida exacta, y que el conjunto del proceso quede justificado por escrito. La Tabla [3.11](#) resume los criterios de calidad aplicados en cada área del proyecto y cómo se ha comprobado su cumplimiento.

Tabla 3.11: Plan de calidad del proyecto.

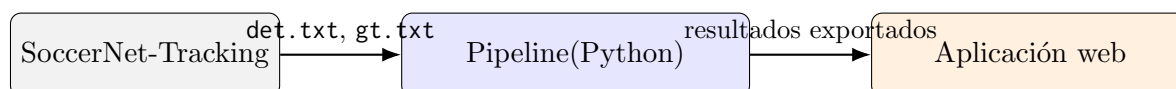
Área	Criterio de calidad	Comprobación
Resultados de tracking	Formato MOT válido y coherente con trackeval	Ejecución de TrackEval sin errores sobre todas las secuencias de train y test
Evaluación	Comparación de algoritmos con métricas estándar (HOTA, MOTA, IDF1)	Revisión cruzada de los CSV/gráficas generados por TrackEval
Hiperparámetros	Barrido reproducible, un parámetro a la vez sobre el mismo split	Configuración óptima verificada con una segunda ejecución independiente (<code>*_optimal.py</code>)
Metadatos	Cada metadato documenta explícitamente su limitación (balón, árbitro, homografía parcial)	Revisión visual de las figuras generadas frente a los datos crudos
Reproducibilidad	Entorno fijado en <code>requirements.txt</code> , resultados intermedios persistidos en disco	Recreación del entorno en dos equipos distintos (sobremesa Windows y macOS) sin incidencias relevantes
Documentación	Cada decisión técnica queda justificada en la memoria	Contraste sistemático de las anotaciones propias con el código real del repositorio (véase capítulo 9)
Anomalías	Las trayectorias con saltos de posición incompatibles con un movimiento humano real se detectan y excluyen en vez de aceptarse sin más	Revisión visual de las trayectorias proyectadas por homografía; exclusión manual documentada de los <i>track_id</i> anómalos (sección 6.6.3)
Explicabilidad de las predicciones	Cada resultado del pipeline es trazable a una regla o un parámetro explícito, no a un modelo de caja negra	Revisión manual de casos concretos (p. ej., por qué una detección se clasifica como balón o a qué equipo se asigna un jugador) contra los umbrales exactos documentados en el capítulo 6
Código	Organización en paquetes por responsabilidad, sin lógica duplicada entre scripts	Revisión de la estructura del repositorio (Tabla 6.3); reutilización explícita de la lógica de <code>src/metadata/</code> desde <code>src/web_export/</code> en vez de duplicarla
Optimización	Los artefactos generados no consumen más recursos de los necesarios (tiempo de cómputo, tamaño en disco)	Muestreo de 1 de cada 5 <i>frames</i> en la clasificación de equipos (sección 6.6.1); elección del <i>fourcc</i> <code>avc1</code> para los vídeos de la web, que redujo su tamaño en torno a un 75 % (sección 6.7.2)
Aplicación web	Interfaz funcional fiel al diseño planificado, sin necesidad de <i>backend</i>	Verificación manual de ambas pestañas con datos reales; <code>npm run build</code> sin errores y sitio servido en local con <code>npm run preview</code> (sección 8.8)

4. Análisis de requisitos

Este capítulo describe el análisis funcional del sistema desarrollado en este proyecto, en el capítulo se describen: los actores que interactúan con él, los requisitos de información, los requisitos funcionales y no funcionales que debe satisfacer, las reglas de negocio, y los casos de uso que formalizan cómo se usa. Finalmente el capítulo se cierra con una matriz de trazabilidad que relaciona cada requisito con los objetivos específicos definidos en la sección 1.4 del capítulo 1, de modo que pueda comprobarse que ningún objetivo queda sin cubrir y que ningún requisito queda fuera de los fines del proyecto.

4.1. Contexto funcional del sistema

El sistema desarrollado en este proyecto aborda la comparación de algoritmos de seguimiento multiobjeto sobre vídeo de fútbol y la extracción de metadatos deportivos a partir de las trayectorias resultantes. Se organiza en dos bloques con una naturaleza muy distinta, representados en la Figura 4.1: un **pipeline offline** en Python, que concentra todo el cómputo pesado, y una **aplicación web** de visualización (sección 6.7), que consume los resultados ya calculados por el pipeline. El planteamiento se detalla en el capítulo 5.



El pipeline consume el dataset y produce resultados persistidos en disco (CSV, PNG, MP4); la aplicación web los consume ya calculados, sin ejecutar ninguna fase del pipeline.

Figura 4.1: Contexto funcional del sistema: el dataset público como entrada, el pipeline offline como bloque de cómputo, y la aplicación web como capa de visualización sobre los resultados.

4.1.1. Justificación y decisión de la tarea elegida

Antes de comprometerse con el seguimiento multiobjeto como tarea de este proyecto, se exploraron brevemente otras tareas del ecosistema SoccerNet (sección 2.2), como parte de la primera fase del proyecto descrita en la sección 3.2. Concretamente, se valoraron tres alternativas:

- **Reconocimiento de dorsales** (*jersey number recognition*): predecir el número de un jugador a partir de una imagen recortada. Se consideró una opción

técnicamente válida y de alta viabilidad que era adecuada para el contexto del proyecto, ya que trataba de trabajar sobre imágenes estáticas, no sobre secuencias, pero con menor profundidad técnica para una comparativa de algoritmos, el enfoque que finalmente se decidió dar a este proyecto.

- **Segmentación de cámara** (*camera segmentation*): clasificar en cada instante del vídeo el tipo de plano de cámara (plano general, primer plano, repetición). Se descartó sin llegar a explorarse ni implementarse: se valoró como menos interesante técnicamente y de menor impacto visual en una demostración que las otras dos alternativas.
- **Seguimiento multiobjeto** (MOT, sección 2.3): la tarea finalmente elegida.

La decisión final se apoyó en varios factores concretos, verificados mediante pruebas exploratorias reales antes de comprometerse con la tarea: el dataset SoccerNet-Tracking incluye detecciones ya calculadas (`det.txt`, sección 2.2.1), lo que elimina la barrera de tener que entrenar o adaptar un detector de objetos propio; en apenas unas horas de exploración inicial fue posible pasar de cero a visualizaciones funcionales sobre el *ground truth* (scripts descritos en la sección 6.3); el tutor confirmó que era válido plantear el trabajo como una comparativa rigurosa de varios algoritmos de *tracking-by-detection* (sección 2.5), en lugar de limitarse a implementar uno solo; y la posibilidad de una demostración web con mapa de calor de trayectorias se valoró como la más atractiva visualmente de las tres alternativas exploradas.

Como consecuencia directa de esta decisión, y como se ha detallado en la sección 2.5, se seleccionó la librería `boxmot`, la cual agrupa varias implementaciones de algoritmos de tracking bajo una interfaz común, de modo que cambiar de algoritmo se reduce a cambiar una línea de código, ideal para una comparativa, fijando específicamente su versión 10.0.43, ya que versiones posteriores de la librería (a partir de la v18) dejaron de mostrar las clases de cada algoritmo por separado, unificándolas bajo una única interfaz, lo cual no se quería en este proyecto, ya que dificultaba precisamente la comparación que el proyecto necesitaba. Para la evaluación se adoptó `TrackEval` (sección 6.4), por ser la herramienta de referencia oficial del propio benchmark de SoccerNet y que funciona muy bien con el formato MOT estándar que produce `boxmot`.

4.2. Actores y usuarios del sistema

La aplicación web, una vez desarrollada, consumirá artefactos estáticos sin sesión de usuario. Por ese motivo, los actores no son usuarios con inicio de sesión, sino roles o perspectivas de uso frente al sistema. En el desarrollo, es solo el autor del proyecto quien asume los roles internos, pero se describen por separado porque responden a objetivos distintos frente al sistema, del mismo modo que se distinguieron los roles de gestión en la sección 3.3.

Se identifican dos actores primarios y un actor secundario, representados en la Figura 4.2 y descritos en la Tabla 4.1: el **operador del pipeline**, que ejecuta las distintas fases del procesamiento offline, el **usuario analista** que consulta la aplicación

web para explorar e interpretar los resultados y SoccerNet, como sistema externo del que el pipeline obtiene los datos de entrada.

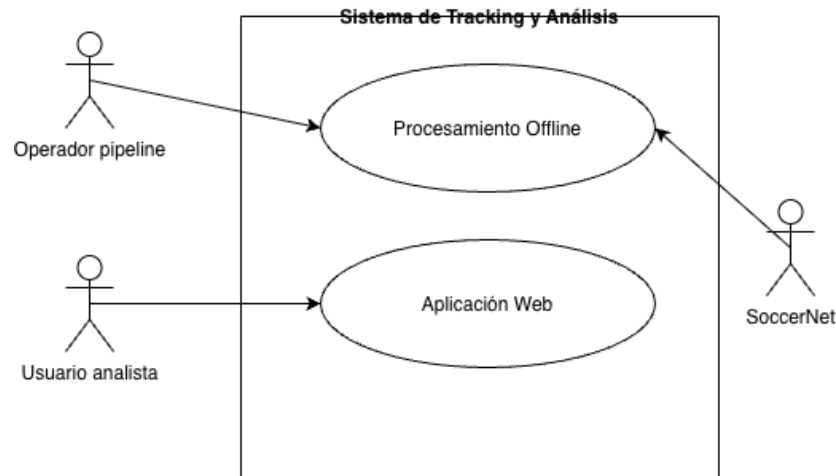


Figura 4.2: Actores del sistema: operador del pipeline y usuario analista (izquierda), interactuando respectivamente con el procesamiento offline y la aplicación web, y SoccerNet como sistema externo (derecha).

Tabla 4.1: Actores y usuarios del sistema.

Actor	Tipo	Descripción
Operador del pipeline	Primario	Ejecuta las fases del procesamiento offline: descarga del dataset, tracking, evaluación, estudio de hiperparámetros y extracción de metadatos.
Usuario analista	Primario	Consulta la aplicación web para explorar los vídeos anotados, las métricas comparativas y los metadatos deportivos generados.
SoccerNet	Secundario	Sistema externo del que se obtienen las secuencias de vídeo, las detecciones precomputadas y el <i>ground truth</i> (sección 2.2.1).

4.3. Requisitos de información

Los requisitos de información indican qué datos debe recopilar, generar y conservar el sistema para ejecutar el pipeline. Se recogen en las Tablas 4.2 a 4.10.

Tabla 4.2: IRQ-001: Detecciones precomputadas de SoccerNet-Tracking.

IRQ-001 – Detecciones precomputadas de SoccerNet-Tracking	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	<code>frame_id</code> , <code>x</code> , <code>y</code> , <code>w</code> , <code>h</code> , <code>confianza</code> (archivo <code>det.txt</code> por secuencia)
Descripción	El sistema debe leer, para cada secuencia, las detecciones ya calculadas de jugadores, árbitros y balón, como entrada de los algoritmos de tracking (sección 2.3).

Tabla 4.3: IRQ-002: *Ground truth* de tracking.

IRQ-002 – <i>Ground truth</i> de tracking	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	<code>frame_id</code> , <code>obj_id</code> , <code>x</code> , <code>y</code> , <code>w</code> , <code>h</code> (archivo <code>gt.txt</code> , solo en train y test)
Descripción	El sistema debe leer la posición e identidad real de cada objeto por frame, como referencia para calcular las métricas de evaluación (sección 2.6).

Tabla 4.4: IRQ-003: Metadatos de secuencia y de partido.

IRQ-003 – Metadatos de secuencia y de partido	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	Resolución de imagen y <i>frame rate</i> (<code>seqinfo.ini</code>);
Descripción	El sistema debe conservar esta información de contexto de cada secuencia, necesaria para interpretar correctamente sus detecciones y para la configuración de TrackEval.

Tabla 4.5: IRQ-004: Resultados de tracking por algoritmo.

IRQ-004 – Resultados de tracking por algoritmo	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	<code>frame,id,x,y,w,h,conf,-1,-1,-1</code> (formato MOT), un fichero <code>.txt</code> por secuencia y algoritmo, en <code>results_train/</code> , <code>results_test/</code> y <code>results_hyperparam/</code>
Descripción	El sistema debe persistir las trayectorias resultantes de cada algoritmo de tracking sobre cada secuencia, en el formato estándar que consume TrackEval.

Tabla 4.6: IRQ-005: Resultados de evaluación con métricas MOT.

IRQ-005 – Resultados de evaluación con métricas MOT	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	HOTA, HOTA(0), DetA, AssA, MOTA, IDF1, IDSW por secuencia y combinado
Descripción	El sistema debe conservar las métricas calculadas por TrackEval para cada algoritmo, tanto por secuencia como combinadas, como base del análisis comparativo del capítulo 7.

Tabla 4.7: IRQ-006: Resultados del estudio de hiperparámetros.

IRQ-006 – Resultados del estudio de hiperparámetros	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	<code>algo, tracker, param, value</code> , HOTA, MOTA, IDF1, IDSW (<code>summary_table.csv</code> y gráficas en <code>evaluation_hyperparam/</code>)
Descripción	El sistema debe conservar, para cada valor probado de cada hiperparámetro, las métricas resultantes, con el fin de justificar la configuración óptima seleccionada.

Tabla 4.8: IRQ-007: Clasificación de jugadores por equipo.

IRQ-007 – Clasificación de jugadores por equipo	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	<code>frame</code> , <code>track_id</code> , <code>team</code> , <code>b</code> , <code>g</code> , <code>r</code> y <code>track_id</code> , <code>team</code> , en <code>metadata_output/teams/</code>
Descripción	El sistema debe conservar la asignación de cada jugador a un equipo (0 o 1), obtenida mediante K-means sobre el color de camiseta (sección 2.8).

Tabla 4.9: IRQ-008: Metadatos deportivos derivados.

IRQ-008 – Metadatos deportivos derivados	
Versión	v1.0
Rol	Operador del pipeline
Datos específicos	Mapas de calor, distancia, velocidad y posesión, todos los atributos relacionados con el rendimiento de los jugadores en <code>metadata_output/</code>
Descripción	El sistema debe persistir los metadatos deportivos derivados de las trayectorias: ocupación del campo, distancia y velocidad por homografía, y posesión de balón (sección 6.6).

Tabla 4.10: IRQ-009: Datos de visualización para la aplicación web.

IRQ-009 – Datos de visualización para la aplicación web	
Versión	v1.0
Rol	Usuario analista
Datos específicos	Vídeos anotados y resultados agregados de evaluación e hiperparámetros preparados para gráficas comparativas
Descripción	El sistema deberá exportar, a partir de los artefactos ya generados por el pipeline, el subconjunto de datos que consumirá la aplicación web.

4.4. Requisitos funcionales

En las Tablas 4.11 a 4.27 se detallan los requisitos funcionales del sistema, cada uno con su identificador, versión, rol responsable, importancia y descripción.

Tabla 4.11: RF-001: Descargar el dataset SoccerNet-Tracking.

RF-001 – Descargar el dataset SoccerNet-Tracking	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Muy alta
Descripción	El sistema debe descargar las secuencias de entrenamiento, test y <i>challenge</i> de SoccerNet-Tracking mediante la API oficial de descarga (<code>src/utils/socccernet_download.py</code>).

Tabla 4.12: RF-002: Filtrar el balón de las detecciones.

RF-002 – Filtrar el balón de las detecciones	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Muy alta
Descripción	El sistema debe separar las detecciones del balón de las de jugadores y árbitros, antes de pasar las detecciones restantes al tracker.

Tabla 4.13: RF-003: Ejecutar ByteTrack sobre una secuencia.

RF-003 – Ejecutar ByteTrack sobre una secuencia	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Muy alta
Descripción	El sistema debe ejecutar el algoritmo ByteTrack (sección 2.5.3) sobre las detecciones de jugadores de una secuencia y persistir los resultados en formato MOT (IRQ-004).

Tabla 4.14: RF-004: Ejecutar OC-SORT sobre una secuencia.

RF-004 – Ejecutar OC-SORT sobre una secuencia	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Muy alta
Descripción	El sistema debe ejecutar el algoritmo OC-SORT (sección 2.5.2) sobre las detecciones de jugadores de una secuencia y persistir los resultados en formato MOT (IRQ-004).

Tabla 4.15: RF-005: Ejecutar el tracking sobre todas las secuencias de un split.

RF-005 – Ejecutar el tracking sobre todas las secuencias de un split	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Muy alta
Descripción	El sistema debe ejecutar ambos algoritmos de tracking sobre la totalidad de las secuencias de un split (train o test) de forma automatizada (<code>run_all_train.py</code> , <code>run_all_test.py</code>), reasignando los identificadores a una numeración secuencial 1..N por secuencia.

Tabla 4.16: RF-006: Evaluar los resultados con métricas MOT estándar.

RF-006 – Evaluar los resultados con métricas MOT estándar	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Muy alta
Descripción	El sistema debe generar el <i>seqmap</i> correspondiente y ejecutar <code>TrackEval</code> sobre los resultados de cada algoritmo, calculando HOTA, MOTA e IDF1 por secuencia y de forma combinada (sección 2.6).

Tabla 4.17: RF-007: Realizar un estudio de hiperparámetros.

RF-007 – Realizar un estudio de hiperparámetros	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	El sistema debe evaluar, para cada algoritmo, el efecto de variar un único hiperparámetro a la vez sobre un rango de valores predefinido, manteniendo el resto en su configuración base.

Tabla 4.18: RF-008: Determinar y validar la configuración óptima de cada algoritmo.

RF-008 – Determinar y validar la configuración óptima de cada algoritmo	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	El sistema debe seleccionar, a partir de los resultados del estudio de hiperparámetros, la combinación de valores con mejores parámetros para cada algoritmo, y volver a ejecutarla y evaluarla de forma independiente para confirmar el resultado.

Tabla 4.19: RF-009: Resumir y graficar el estudio de hiperparámetros.

RF-009 – Resumir y graficar el estudio de hiperparámetros	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Media
Descripción	El sistema debe consolidar todos los resultados del estudio de hiperparámetros en una tabla comparativa y generar una gráfica de barras por parámetro y algoritmo (<code>summarize_hyperparam.py</code>).

Tabla 4.20: RF-010: Clasificar a los jugadores por equipo.

RF-010 – Clasificar a los jugadores por equipo	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	El sistema debe extraer el color dominante de la camiseta de cada detección de jugador y agruparlas en dos equipos mediante K-means (sección 2.8), determinando el equipo mayoritario de cada <code>track_id</code> .

Tabla 4.21: RF-011: Visualizar la clasificación de equipos.

RF-011 – Visualizar la clasificación de equipos	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Media
Descripción	El sistema debe generar una imagen de un frame concreto con cada jugador coloreado según el equipo asignado, para verificar visualmente la clasificación (<code>team_visualization.py</code>).

Tabla 4.22: RF-012: Generar mapas de calor de ocupación del campo.

RF-012 – Generar mapas de calor de ocupación del campo	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	El sistema debe generar mapas de calor de las posiciones de los jugadores proyectados sobre un campo estándar dibujado a escala (sección 2.9).

Tabla 4.23: RF-013: Calcular una homografía real sobre un tramo estable.

RF-013 – Calcular una homografía real sobre un tramo estable	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Media
Descripción	El sistema debe calcular una matriz de homografía real a partir de 4 puntos de referencia identificados manualmente en un tramo de vídeo con cámara estable, y validar su error de reproyección (sección 6.6.3).

Tabla 4.24: RF-014: Estimar distancia recorrida y velocidad por jugador.

RF-014 – Estimar distancia recorrida y velocidad por jugador	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Media
Descripción	El sistema debe usar la homografía de RF-013 para convertir las posiciones de píxel de cada jugador en coordenadas reales del campo, y calcular a partir de ellas la distancia total recorrida y la velocidad media y de pico durante el tramo analizado.

Tabla 4.25: RF-015: Estimar la posesión del balón.

RF-015 – Estimar la posesión del balón	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Media
Descripción	El sistema debe determinar, para cada frame con balón detectado, qué jugador está más cerca y asignar la posesión de ese frame a su equipo, siempre que la distancia no supere el umbral de RN-006, y agregar el resultado en porcentaje de posesión por equipo.

Tabla 4.26: RF-016: Exportar vídeos anotados.

RF-016 – Exportar vídeos anotados	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Baja
Descripción	El sistema debe generar, para una secuencia y algoritmo concretos, un vídeo .mp4 con las cajas delimitadoras y los identificadores de tracking superpuestos sobre cada frame.

Tabla 4.27: RF-017: Visualizar y explorar los resultados en una aplicación web.

RF-017 – Visualizar y explorar los resultados en una aplicación web	
Versión	v1.0
Rol	Usuario analista
Importancia	Alta
Descripción	La aplicación web deberá permitir explorar los vídeos anotados, comparar las métricas de ambos algoritmos y consultar los metadatos deportivos generados, sin ejecutar cómputo en el navegador (capítulo 5).

4.5. Requisitos no funcionales

En las Tablas 4.28 a 4.36 se definen los requisitos no funcionales que fijan las características de calidad del sistema, complementando los criterios ya avanzados en el plan de calidad de la sección 3.9.

Tabla 4.28: RNF-001: Reproducibilidad.

RNF-001 – Reproducibilidad	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Muy alta
Descripción	El entorno de ejecución debe quedar completamente fijado en <code>requirements.txt</code> , y los resultados intermedios deben persistirse en disco, de modo que cualquier resultado pueda reproducirse o auditarse sin repetir fases ya completadas.

Tabla 4.29: RNF-002: Rendimiento y uso de memoria.

RNF-002 – Rendimiento y uso de memoria	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	El pipeline debe procesar cada secuencia de forma independiente, frame a frame, sin necesidad de cargar en memoria el dataset completo ni varias secuencias a la vez.

Tabla 4.30: RNF-003: Portabilidad entre sistemas operativos.

RNF-003 – Portabilidad entre sistemas operativos	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	El código debe usar rutas relativas y construidas con <code>os.path</code> , de modo que el pipeline funcione sin modificaciones en Windows y en macOS, dado el cambio de equipo de desarrollo documentado en la sección 3.6.

Tabla 4.31: RNF-004: Extensibilidad a nuevos algoritmos de tracking.

RNF-004 – Extensibilidad a nuevos algoritmos de tracking	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Media
Descripción	El sistema debe apoyarse en la interfaz común de <code>boxmot</code> para los algoritmos de tracking, de modo que incorporar un nuevo algoritmo disponible en la librería (p. ej., una variante con Re-ID, sección 2.5.4) no requiera rediseñar el resto del pipeline.

Tabla 4.32: RNF-005: Trazabilidad de resultados.

RNF-005 – Trazabilidad de resultados	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	Cada artefacto generado (resultado, figura, tabla) debe poder relacionarse sin ambigüedad con la secuencia, el algoritmo y la configuración de hiperparámetros que lo produjeron, mediante un nombrado consistentes en los directorios de salida.

Tabla 4.33: RNF-006: Documentación honesta de limitaciones.

RNF-006 – Documentación honesta de limitaciones	
Versión	v1.0
Rol	Operador del pipeline
Importancia	Alta
Descripción	Cada metadato extraído debe documentar explícitamente sus limitaciones metodológicas en lugar de presentarse como una medida exacta, en línea con la metodología descrita en la sección 3.1.

Tabla 4.34: RNF-007: Usabilidad de la aplicación web.

RNF-007 – Usabilidad de la aplicación web	
Versión	v1.0
Rol	Usuario analista
Importancia	Media
Descripción	La aplicación web deberá permitir seleccionar secuencia y algoritmo, y navegar entre vídeo, métricas y metadatos sin recargas de página completas.

Tabla 4.35: RNF-008: Fluidez de la interacción web.

RNF-008 – Fluidez de la interacción web	
Versión	v1.0
Rol	Usuario analista
Importancia	Media
Descripción	Al consumir exclusivamente artefactos estáticos ya precalculados por el pipeline (JSON, imágenes, vídeo), sin cómputo ni llamadas a servidor en tiempo de uso (sección 6.7), la aplicación web debe responder de forma percibida como inmediata al cambiar de pestaña, secuencia o algoritmo, sin bloqueos apreciables de la interfaz.

Tabla 4.36: RNF-009: Consistencia ante datos incompletos.

RNF-009 – Consistencia ante datos incompletos	
Versión	v1.0
Rol	Usuario analista
Importancia	Alta
Descripción	Cuando una secuencia no dispone de un metadato concreto (p. ej. SNMOT-117, sin homografía real calculada por no cumplir el criterio de cámara estable de RN-005), la interfaz debe reflejarlo de forma explícita en el panel correspondiente en lugar de fallar, mostrar datos vacíos sin explicación o mezclar resultados de otra secuencia.

4.6. Reglas de negocio

Las reglas de negocio definen las normas, operaciones y restricciones que acotan el comportamiento del sistema y que actúan de forma transversal sobre los requisitos funcionales para lograr los objetivos del proyecto. Se recogen en las Tablas 4.37 a 4.42.

Tabla 4.37: RN-001: Exclusión del balón del tracking de personas.

RN-001 – Exclusión del balón del tracking de personas	
Descripción	Ninguna detección clasificada como balón por el filtro geométrico (sección 2.7) puede pasarse al algoritmo de tracking de personas.

Tabla 4.38: RN-002: Evaluación final sobre el conjunto de test.

RN-002 – Evaluación final sobre el conjunto de test	
Descripción	Las conclusiones comparativas entre algoritmos deben basarse en los resultados del conjunto de test, no en los del conjunto de entrenamiento, para evitar conclusiones sesgadas por sobreajuste (sección 4.1.1).

Tabla 4.39: RN-003: Barrido de hiperparámetros de uno en uno.

RN-003 – Barrido de hiperparámetros de uno en uno	
Descripción	El estudio de hiperparámetros debe variar un único parámetro cada vez, manteniendo el resto fijos en su valor base, para poder atribuir el efecto observado a ese parámetro concreto.

Tabla 4.40: RN-004: Clasificación de equipos con dos grupos.

RN-004 – Clasificación de equipos con dos grupos	
Descripción	La clasificación de jugadores por color asume exactamente $k = 2$ grupos (sección 2.8); el árbitro no se distingue como grupo aparte, limitación documentada explícitamente en los resultados (RNF-006).

Tabla 4.41: RN-005: Homografía real solo sobre tramos verificados.

RN-005 – Homografía real solo sobre tramos verificados	
Descripción	La homografía real (RF-013) solo se aplica sobre tramos de vídeo verificados manualmente como de cámara estable; no se generaliza al resto de una secuencia ni a secuencias distintas de las verificadas (sección 2.9).

Tabla 4.42: RN-006: Umbral máximo de distancia para la posesión.

RN-006 – Umbral máximo de distancia para la posesión	
Descripción	La posesión del balón (RF-015) solo se asigna a un jugador si la distancia en píxeles entre el balón y ese jugador no supera un umbral máximo; por encima de ese umbral, el frame se considera balón suelto y no se asigna posesión a ningún equipo.

4.7. Casos de uso

Esta sección define los casos de uso que contempla el sistema, cada uno descrito mediante una tabla. Las Figuras 4.3 y 4.4 muestran el diagrama de casos de uso separado en los del pipeline offline y los de la aplicación web, por legibilidad. Las Tablas 4.43 a 4.53 recogen el detalle de cada uno.

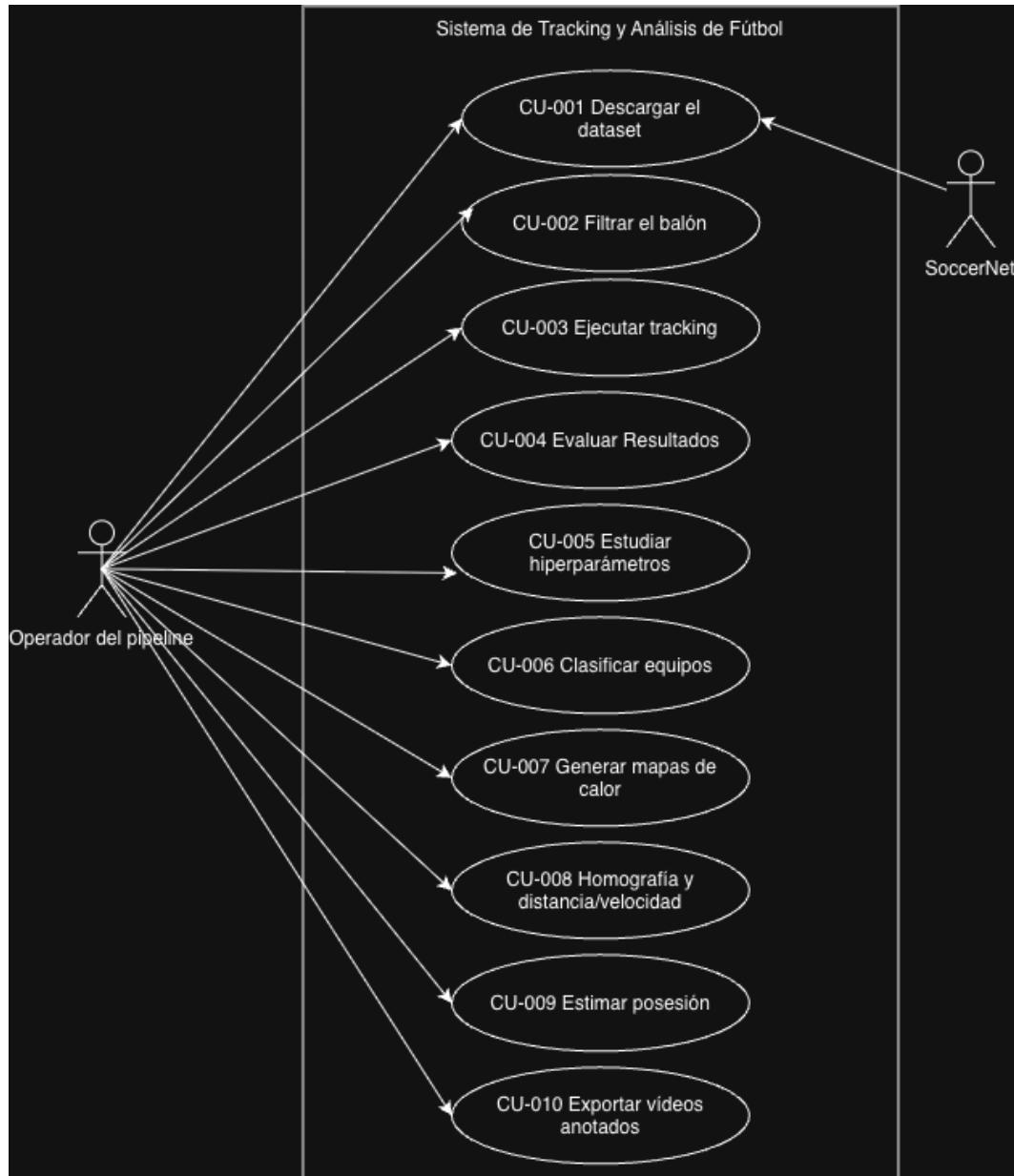


Figura 4.3: Casos de uso del pipeline offline: el operador del pipeline y SoccerNet como sistema externo.

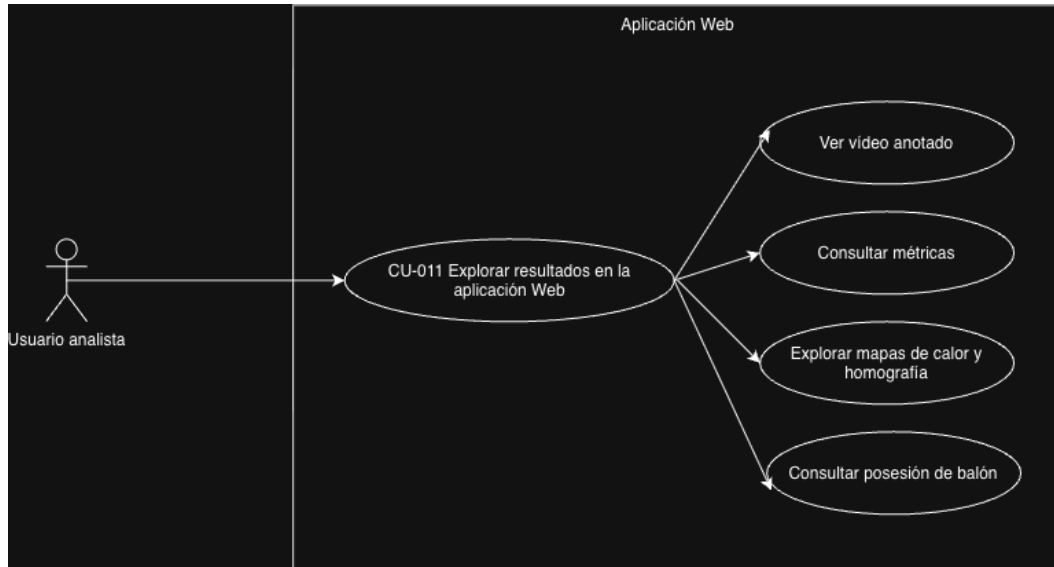


Figura 4.4: Caso de uso de la aplicación web: el usuario analista explora los resultados a través de cuatro comportamientos incluidos.

Tabla 4.43: CU-001: Descargar el dataset.

CU-001 – Descargar el dataset	
Actor	Operador del pipeline
Descripción	El operador ejecuta la descarga de las secuencias de SoccerNet-Tracking mediante la API oficial.
Precondiciones	Acceso a la API de SoccerNet.
Flujo principal	1) El operador ejecuta <code>soccernet_download.py</code> . 2) El sistema descarga las secuencias de train, test y <i>challenge</i> . 3) Las almacena en la estructura de carpetas esperada por el resto del pipeline.
Postcondiciones	Dataset disponible localmente para el resto de casos de uso.

Tabla 4.44: CU-002: Filtrar el balón de las detecciones.

CU-002 – Filtrar el balón de las detecciones	
Actor	Operador del pipeline
Descripción	El sistema separa las detecciones de balón del resto antes de pasarlas al tracker.
Precondiciones	<code>det.txt</code> de la secuencia disponible.
Flujo principal	1) El sistema lee cada detección del frame. 2) Calcula su área y su ratio ancho y alto. 3) La clasifica como balón o jugador según los umbrales del filtro geométrico.
Postcondiciones	Detecciones de jugadores separadas de las de balón, listas para el tracking.

Tabla 4.45: CU-003: Ejecutar tracking (ByteTrack / OC-SORT).

CU-003 – Ejecutar tracking	
Actor	Operador del pipeline
Descripción	El operador ejecuta uno o ambos algoritmos de tracking sobre una o todas las secuencias de un split.
Precondiciones	Detecciones de jugadores filtradas disponibles.
Flujo principal	1) El operador lanza <code>run_all_train.py</code> o <code>run_all_test.py</code> . 2) El sistema inicializa el tracker elegido. 3) Actualiza el tracker frame a frame con las detecciones. 4) Persiste los resultados en formato MOT (IRQ-004).
Postcondiciones	Resultados de tracking disponibles para su evaluación.

Tabla 4.46: CU-004: Evaluar resultados de tracking.

CU-004 – Evaluar resultados de tracking	
Actor	Operador del pipeline
Descripción	El operador evalúa los resultados de tracking con <code>TrackEval</code> .
Precondiciones	Resultados de tracking y <i>ground truth</i> disponibles.
Flujo principal	1) El sistema genera el <i>seqmap</i> del split. 2) Ejecuta <code>TrackEval</code> sobre los resultados de cada algoritmo. 3) Persiste las métricas por secuencia y combinadas.
Postcondiciones	Métricas de evaluación disponibles para la comparativa (capítulo 7).

Tabla 4.47: CU-005: Realizar estudio de hiperparámetros.

CU-005 – Realizar estudio de hiperparámetros	
Actor	Operador del pipeline
Descripción	El operador ejecuta el barrido de hiperparámetros de un algoritmo y obtiene su configuración óptima.
Precondiciones	Detecciones de test filtradas y evaluación disponibles.
Flujo principal	1) El sistema recorre cada valor del rango de un hiperparámetro. 2) Ejecuta el tracking y la evaluación con ese valor. 3) Registra la métrica resultante. 4) Selecciona el valor con mejor HOTA y lo confirma con una ejecución independiente (RF-008).
Postcondiciones	Configuración óptima determinada y validada para el algoritmo.

Tabla 4.48: CU-006: Clasificar jugadores por equipo.

CU-006 – Clasificar jugadores por equipo	
Actor	Operador del pipeline
Descripción	El operador ejecuta la clasificación de equipos por color de camiseta sobre una secuencia.
Precondiciones	Resultados de tracking con la configuración óptima disponibles.
Flujo principal	1) El sistema recorta la zona de camiseta de cada detección de jugador. 2) Extrae su color dominante con K-means ($k = 1$). 3) Agrupa todos los colores en dos equipos con K-means ($k = 2$). 4) Determina el equipo mayoritario de cada <code>track.id</code> .
Postcondiciones	Asignación de equipo por jugador persistida.

Tabla 4.49: CU-007: Generar mapas de calor.

CU-007 – Generar mapas de calor	
Actor	Operador del pipeline
Descripción	El operador genera los mapas de calor de ocupación del campo, globales y por equipo.
Precondiciones	Resultados de tracking y, para el mapa por equipo, clasificación de equipos disponibles.
Flujo principal	1) El sistema proyecta las posiciones de los jugadores sobre el campo dibujado a escala. 2) Calcula la densidad de posiciones mediante estimación KDE. 3) Genera y persiste la figura.
Postcondiciones	Mapas de calor disponibles.

Tabla 4.50: CU-008: Calcular homografía y estimar distancia/velocidad.

CU-008 – Calcular homografía y estimar distancia/velocidad	
Actor	Operador del pipeline
Descripción	El operador calcula una homografía real sobre un tramo verificado y estima distancia y velocidad de los jugadores.
Precondiciones	Tramo de cámara estable identificado manualmente ; resultados de tracking disponibles.
Flujo principal	1) El operador identifica 4 puntos de referencia en un frame representativo. 2) El sistema calcula la matriz de homografía y valida su error de reproyección. 3) Proyecta las posiciones de cada jugador a metros reales. 4) Calcula distancia total y velocidad media y de pico.
Postcondiciones	Distancia y velocidad por jugador persistidas.

Tabla 4.51: CU-009: Estimar posesión de balón.

CU-009 – Estimar posesión de balón	
Actor	Operador del pipeline
Descripción	El operador calcula el porcentaje de posesión de balón por equipo, por proximidad espacial.
Precondiciones	Clasificación de equipos y detecciones de balón disponibles.
Flujo principal	1) El sistema calcula, para cada frame con balón detectado, el jugador más cercano. 2) Comprueba que la distancia no supera el umbral. 3) Asigna la posesión de ese frame al equipo de ese jugador. 4) Agrega el resultado en porcentaje por equipo.
Postcondiciones	Porcentaje de posesión por equipo persistido.

Tabla 4.52: CU-010: Exportar vídeos anotados.

CU-010 – Exportar vídeos anotados	
Actor	Operador del pipeline
Descripción	El operador genera un vídeo con las cajas e identificadores de tracking superpuestos, para su uso en la memoria y en la demo.
Precondiciones	Resultados de tracking disponibles.
Flujo principal	1) El operador selecciona secuencia y algoritmo. 2) El sistema dibuja las cajas e identificadores sobre cada frame. 3) Codifica el vídeo resultante en formato .mp4.
Postcondiciones	Vídeo anotado disponible en <code>videos/</code> .

Tabla 4.53: CU-011: Explorar resultados en la aplicación web.

CU-011 – Explorar resultados en la aplicación web	
Actor	Usuario analista
Descripción	El usuario explora los resultados del proyecto a través de dos vistas: una comparativa global de métricas entre algoritmos, y un explorador por secuencia y algoritmo con el vídeo anotado y los metadatos deportivos generados, sin recálculo en el navegador. Incluye: ver vídeo anotado, consultar métricas comparativas, explorar mapas de calor y homografía, y consultar posesión de balón.
Precondiciones	Datos de visualización exportados disponibles.
Flujo principal	1) El usuario abre la aplicación web, que muestra por defecto la comparativa global de métricas entre algoritmos. 2) El usuario cambia a la pestaña de exploración por secuencias y selecciona secuencia y algoritmo. 3) La interfaz muestra el vídeo anotado y los paneles de equipos y posesión. 4) El usuario consulta, opcionalmente, los mapas de calor y la homografía de esa secuencia.
Postcondiciones	El usuario obtiene una interpretación visual de los resultados del proyecto, agregados o por secuencia concreta.

4.8. Matriz de trazabilidad

La matriz de trazabilidad relaciona los requisitos de este capítulo con los objetivos específicos OE-01 a OE-11 definidos en la sección 1.4 del capítulo 1. Sirve para comprobar que cada objetivo queda cubierto por al menos un requisito y que ningún requisito queda desligado de los fines del proyecto. La trazabilidad se reparte en tres tablas, correspondientes a los requisitos de información (Tabla 4.54), los requisitos funcionales (Tabla 4.55) y los requisitos no funcionales junto con las reglas de negocio (Tabla 4.56).

Tabla 4.54: Trazabilidad de los requisitos de información.

Req.	OE-01	OE-02	OE-03	OE-04	OE-05	OE-06	OE-07	OE-08	OE-09	OE-10	OE-11
IRQ-001	X										
IRQ-002	X			X							
IRQ-003	X										
IRQ-004		X	X	X	X						
IRQ-005				X	X						
IRQ-006					X						
IRQ-007						X	X		X		
IRQ-008							X	X	X		
IRQ-009											X

Tabla 4.55: Trazabilidad de los requisitos funcionales.

Req.	OE-01	OE-02	OE-03	OE-04	OE-05	OE-06	OE-07	OE-08	OE-09	OE-10	OE-11
RF-001	X										
RF-002		X									
RF-003		X									
RF-004		X									
RF-005			X								
RF-006				X							
RF-007					X						
RF-008					X						
RF-009					X						
RF-010						X					
RF-011						X					
RF-012							X				
RF-013								X			
RF-014								X			
RF-015									X		
RF-016										X	
RF-017											X

Tabla 4.56: Trazabilidad de requisitos no funcionales y reglas de negocio.

Req.	OE-01	OE-02	OE-03	OE-04	OE-05	OE-06	OE-07	OE-08	OE-09	OE-10	OE-11
RNF-001										X	
RNF-002			X								
RNF-003										X	
RNF-004		X									
RNF-005										X	
RNF-006										X	
RNF-007											X
RNF-008											X
RNF-009											X
RN-001		X									
RN-002				X							
RN-003					X						
RN-004						X					
RN-005								X			
RN-006									X		

La lectura de las tres tablas muestra que los once objetivos específicos reciben al menos una traza y que ningún requisito queda aislado. El objetivo OE-10 (documentar las limitaciones técnicas) recibe trazas exclusivamente de requisitos no funcionales, coherente con su naturaleza transversal: no se satisface con una funcionalidad concreta, sino con un criterio de calidad aplicado a todas las demás (sección 3.9).

5. Diseño

Este capítulo presenta el diseño del sistema desarrollado en este proyecto. Este capítulo se centra en el *qué* y el *por qué* de cada diseño, no en el *cómo* se ha implementado, que se detalla en el capítulo 6.

5.1. Arquitectura general

Como se avanzó en el capítulo 4 (Figura 4.1), el sistema se organiza en dos bloques muy distintos: un **pipeline offline** en Python, y una **aplicación web** de visualización que consume los resultados ya calculados. La Figura 5.1 representa esta arquitectura a nivel de componentes: no las funcionalidades concretas de cada fase –eso se detalla en las secciones siguientes de este capítulo–, sino los grandes bloques del sistema y cómo se comunican entre sí.

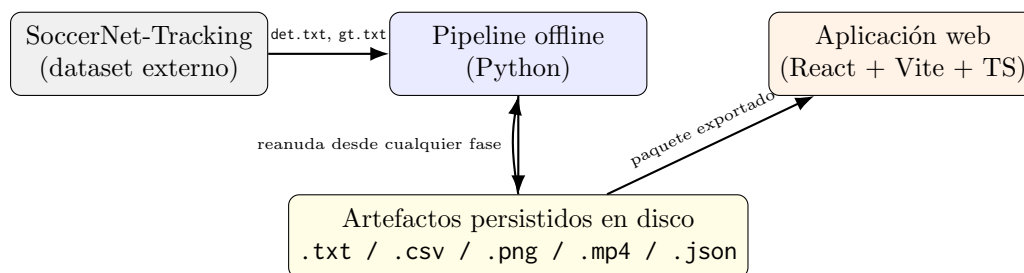


Figura 5.1: Arquitectura general del sistema, a nivel de componentes: el pipeline offline persiste todo su trabajo en disco en vez de mantener estado en memoria, y la aplicación web consume ese mismo almacenamiento ya calculado, sin recalcularlo nada ni compartir proceso con el pipeline.

Esta arquitectura por artefactos persistidos tiene tres ventajas de diseño aprovechadas en este proyecto. Primero, permite **reanudar** el trabajo desde cualquier fase sin repetir las anteriores: el estudio de hiperparámetros (sección 5.4) se apoya en resultados de tracking ya calculados, y el pipeline de metadatos (sección 5.5) se apoya en la configuración óptima ya validada, sin necesidad de volver a ejecutar el tracking desde cero cada vez. Segundo, favorece la **reproducibilidad** (RNF-001): cualquier resultado puede auditarse inspeccionando el artefacto correspondiente. Tercero, es la base que permite a la aplicación web (sección 5.7) limitarse a consumir artefactos estáticos.

5.2. Diseño del pipeline de tracking

El pipeline de tracking transforma las detecciones precomputadas de una secuencia en trayectorias con identidad consistente, aplicando el algoritmo de tracking

correspondiente (sección 2.5). Se compone de tres bloques de diseño: el filtrado del balón, la ejecución del algoritmo de tracking sobre una única secuencia, y la orquestación de esa ejecución sobre la totalidad de un split.

5.2.1. Diseño del filtrado del balón

Como se justificó en la sección 2.7, el balón se separa de las detecciones de personas mediante un filtro geométrico, no mediante un detector entrenado. El diseño de este filtro es deliberadamente simple: para cada detección del frame, se evalúan dos condiciones independientes sobre su caja delimitadora: su área en píxeles y su proporción ancho/alto, y solo si ambas se cumplen a la vez la detección se aparta del flujo de tracking de personas, como ilustra la Figura 5.2. Los valores concretos de los dos umbrales empleados se detallan y justifican en la sección 6.3.

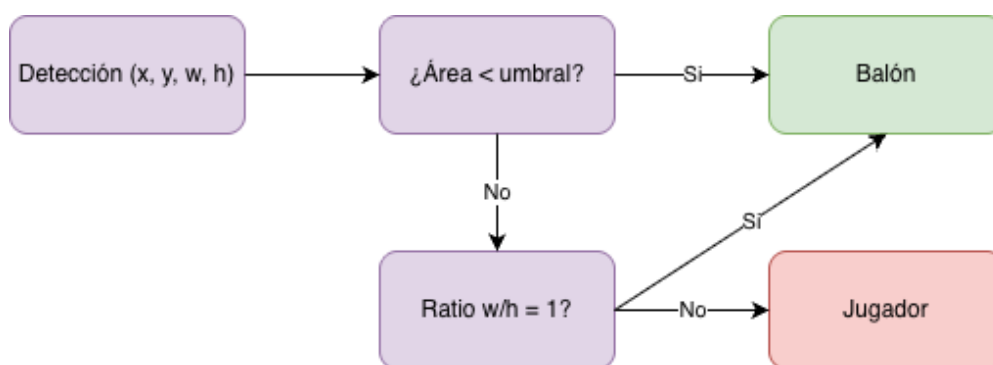


Figura 5.2: Diseño del filtro geométrico de separación del balón: dos condiciones sobre área y proporción de la caja delimitadora, evaluadas sobre cada detección de cada frame antes de invocar al tracker.

5.2.2. Diseño de los algoritmos de tracking

Los dos algoritmos comparados (ByteTrack y OC-SORT) se ejecutan a través de la interfaz común que expone la librería **boxmot** (sección 4.1.1), lo que permite que el diseño del bucle principal de procesamiento sea idéntico para ambos y solo cambie la clase de tracker instanciada. Este diseño responde directamente al requisito de extensibilidad (RNF-004): incorporar un tercer algoritmo disponible en **boxmot** no exige rediseñar el bucle, solo instanciarlo. La Figura 5.3 representa este bucle, común a ambos algoritmos.

El paso de **reasignación de identificador** merece un diseño propio: los algoritmos de **boxmot** generan identificadores internos arbitrarios y no necesariamente consecutivos, mientras que el formato MOT esperado por **TrackEval** y por la interpretación humana de los resultados es más claro con una numeración secuencial 1..N por secuencia. El diseño mantiene, por tanto, una tabla de correspondencia *identificador interno del tracker* $\rightarrow$ *identificador secuencial*, poblada de forma que cada vez que se encuentra un nuevo identificador interno, se le asigna el siguiente número secuencial disponible.

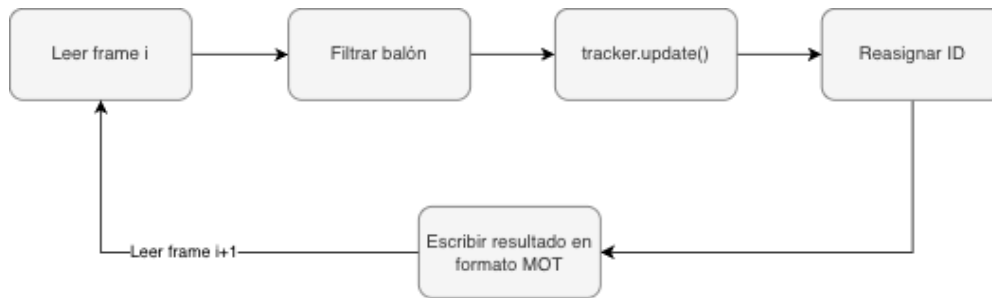


Figura 5.3: Bucle de procesamiento común a ambos algoritmos de tracking.

5.2.3. Diseño de la ejecución batch sobre un split completo

Ejecutar manualmente cada algoritmo sobre cada una de las secuencias de un split sería inviable en la práctica y, sobre todo, propenso a inconsistencias entre secuencias (parámetros distintos por descuido, secuencias olvidadas). El diseño de la ejecución batch resuelve esto con un orquestador que recorre automáticamente todas las secuencias de un directorio de split, aplicando a cada una el mismo bucle de la Figura 5.3 para cada uno de los algoritmos configurados, como se representa en la Figura 5.4.

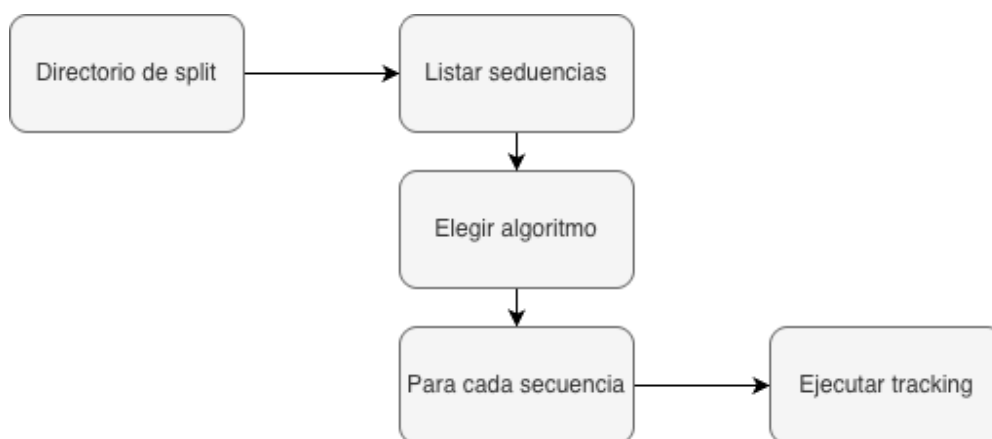


Figura 5.4: Diseño de la ejecución batch: doble bucle sobre algoritmos y secuencias de un split, reutilizando el bucle de procesamiento de la Figura 5.3 para cada combinación.

5.3. Diseño de la evaluación

La evaluación traduce los resultados de tracking y el *ground truth* en las métricas de la sección 2.6, delegando el cálculo en `TrackEval` en lugar de reimplementar las métricas. El diseño de esta fase se limita, por tanto, a preparar correctamente la entrada que `TrackEval` espera y a configurar qué métricas calcular, como resume la Figura 5.5.

Un aspecto de diseño relevante, es que esta misma fase de evaluación se ejecuta **dos veces con distinto propósito**: una primera vez sobre el conjunto de entrenamiento, durante el desarrollo inicial, y una segunda vez sobre el conjunto de

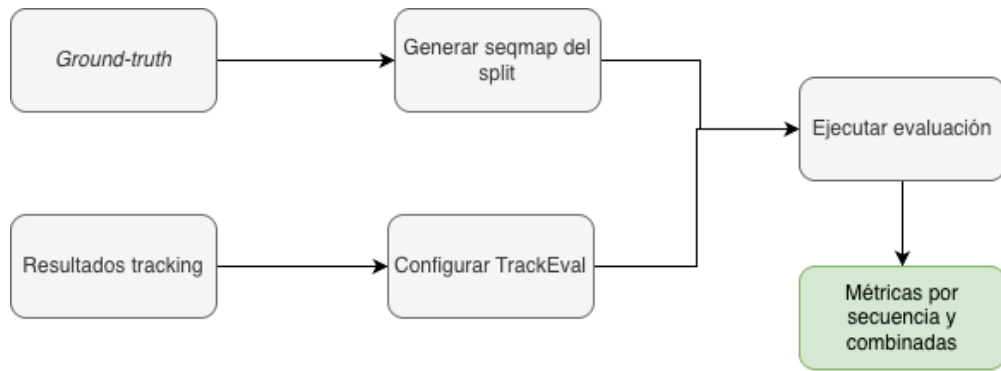


Figura 5.5: Diseño de la fase de evaluación.

test, cuyos resultados son los que finalmente se usan para las conclusiones comparativas. El diseño mantiene ambas ejecuciones completamente separadas.

5.4. Diseño del estudio de hiperparámetros

El estudio de hiperparámetros reutiliza el bucle de tracking (Figura 5.3) y la fase de evaluación (Figura 5.5) como bloques ya diseñados, y añade sobre ellos un bucle de búsqueda *uno a uno*. La Figura 5.6 representa este diseño.

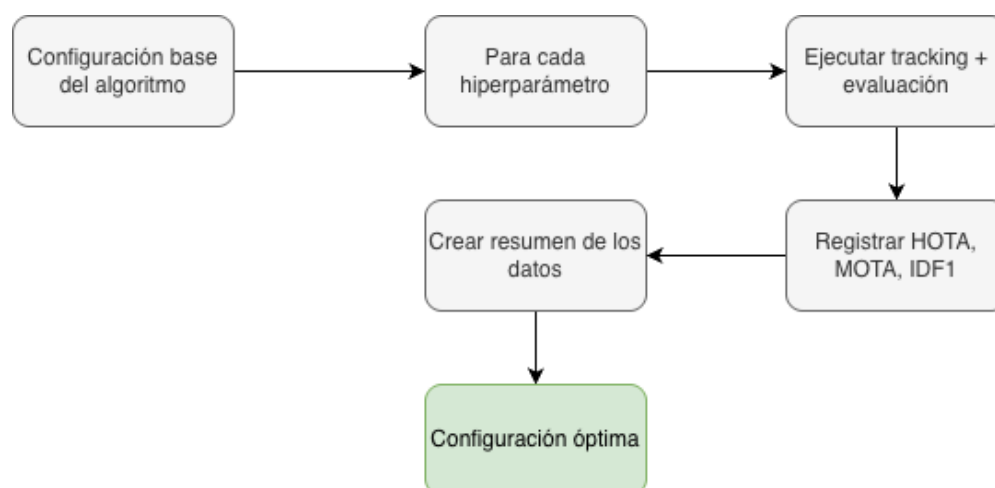


Figura 5.6: Diseño del estudio de hiperparámetros: barrido uno a uno sobre el conjunto de test, resumen comparativo y validación final de la configuración óptima de cada algoritmo.

Este diseño es una decisión deliberada: una búsqueda combinada crece exponencialmente con el número de parámetros y de secuencias de test evaluadas en cada combinación, mientras que el barrido uno a uno, con un coste lineal, permite **atribuir con claridad** el efecto observado a un parámetro concreto, algo que una rejilla combinada por ejemplo dificultaría al mezclar los efectos de varios parámetros a la vez.

5.5. Diseño del pipeline de metadatos

El pipeline de metadatos parte de los resultados de tracking generados con la configuración óptima de ByteTrack (determinada en la fase anterior) y produce los cuatro tipos de metadatos deportivos de este proyecto. Como muestra la Figura 5.7, estos cuatro bloques no son independientes entre sí.

5.5.1. Diseño de la clasificación de equipos

La clasificación de equipos se diseña en dos niveles de K-means encadenados (sección 2.8), representados en la Figura 5.8: primero, un K-means con $k = 1$ extrae el color dominante de la camiseta de *cada* detección de jugador, recortando la región central-superior de su caja delimitadora para evitar incluir piernas, césped o balón en el recorte; después, un segundo K-means con $k = 2$ agrupa *todos* los colores

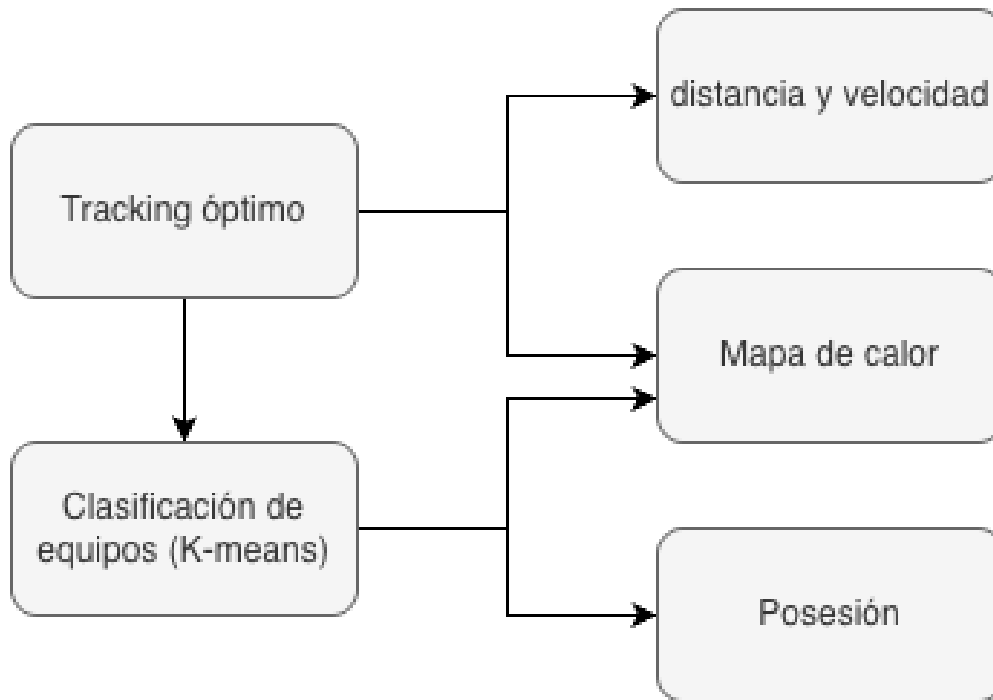


Figura 5.7: Dependencias entre los bloques del pipeline de metadatos.

dominantes obtenidos, asumiendo dos equipos con colores de camiseta distintos (RN-004). Por eficiencia, el primer nivel no se aplica a todas las detecciones de la secuencia, sino a un muestreo de frames, suficiente para determinar el equipo mayoritario de cada `track_id` sin procesar la secuencia completa.

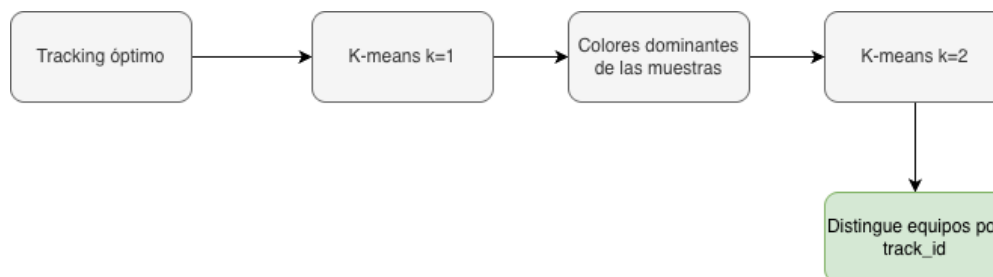


Figura 5.8: Diseño de la clasificación de equipos: K-means en dos niveles.

5.5.2. Diseño de los mapas de calor

Los mapas de calor de ocupación del campo se diseñan como una proyección de las posiciones de tracking sobre un campo dibujado a escala (Figura 2.8), seguida de una estimación de densidad por núcleos (*Kernel Density Estimation*, KDE) que suaviza la nube de puntos discretos en un mapa continuo. Se diseñan dos variantes, que comparten el mismo procedimiento de proyección y KDE pero difieren en el conjunto de posiciones de entrada: el mapa **general**, con todas las posiciones de la secuencia, y el mapa **por equipo** (sección 5.5.1), que separa las posiciones según la clasificación de equipos y usa la misma escala para ambos equipos, de modo que sean directamente comparables

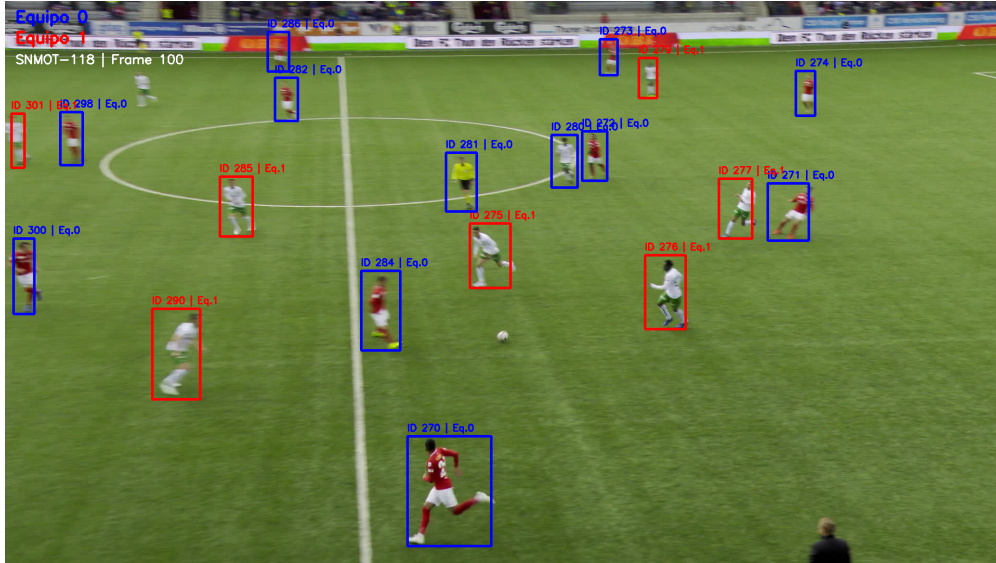


Figura 5.9: Resultado real de la clasificación de equipos sobre un frame de SNMOT-118: cada jugador coloreado según el equipo asignado por K-means.

entre sí.

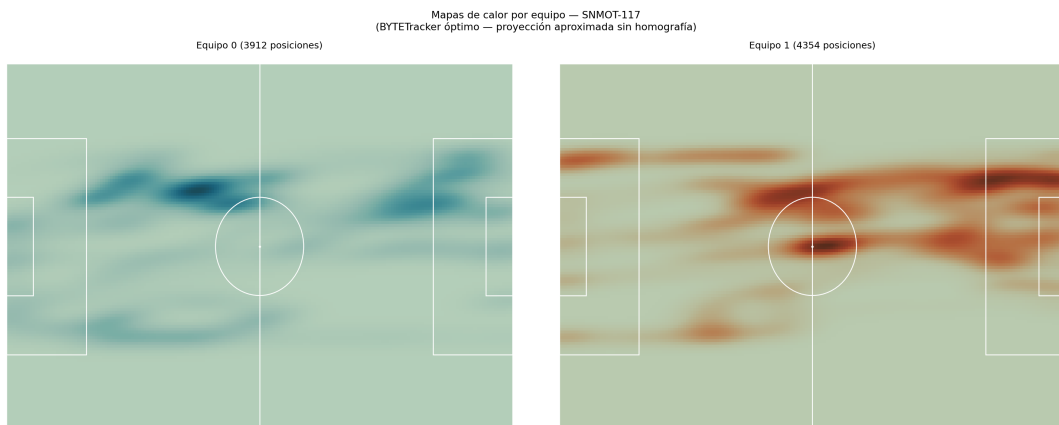


Figura 5.10: Resultado real del mapa de calor por equipo sobre SNMOT-117.

5.5.3. Diseño de la homografía y la estimación de distancia/velocidad

Como se explicó en la sección 2.9, este proyecto distingue dos niveles de fiabilidad en la proyección de coordenadas. El primero, usado de forma general en los mapas de calor, es una **proyección aproximada** que escala las posiciones observadas por su rango directamente sobre las dimensiones del campo. El segundo, diseñado como prueba de concepto sobre un tramo concreto de cámara estable, es una **homografía real**, calculada a partir de 4 puntos de referencia. La Figura 5.11 representa el diseño de este segundo nivel, el único que permite calcular distancia y velocidad en unidades reales (RF-014).

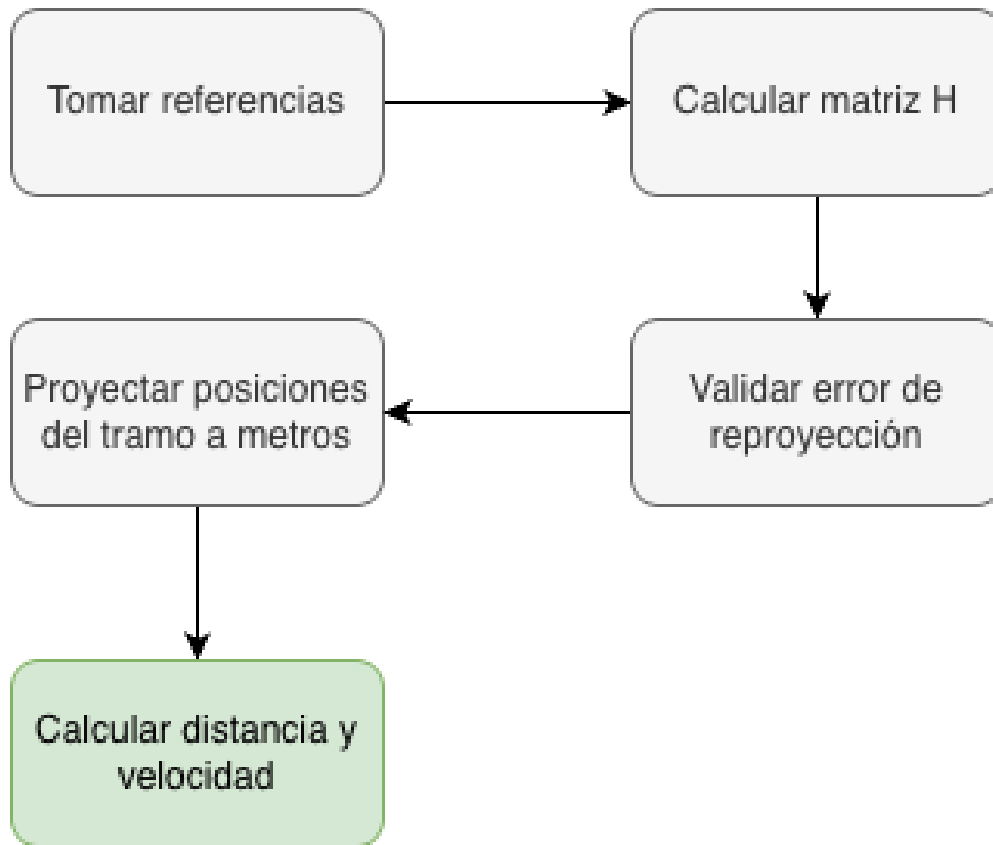


Figura 5.11: Diseño de la homografía

Un elemento de diseño importante, que se retoma como limitación en el capítulo 7, es que la zona del campo fuera del entorno cercano a los 4 puntos de calibración no es fiable con esta homografía. Figura 5.12.

5.5.4. Diseño de la estimación de posesión

La posesión de balón se diseña como una heurística de proximidad espacial frame a frame. Para cada frame en que el filtro geométrico (sección 5.2.1) detecta un balón, se calcula la distancia entre su posición y la de todos los jugadores de ese frame; si el jugador más cercano está por debajo de un umbral máximo de distancia, la posesión de ese frame se asigna a su equipo; en caso contrario, se considera balón suelto y el frame no cuenta para ningún equipo. La Figura 5.13 resume este diseño.



Figura 5.12: Resultado real del mapa de calor con homografía real sobre un tramo de SNMOT-116.

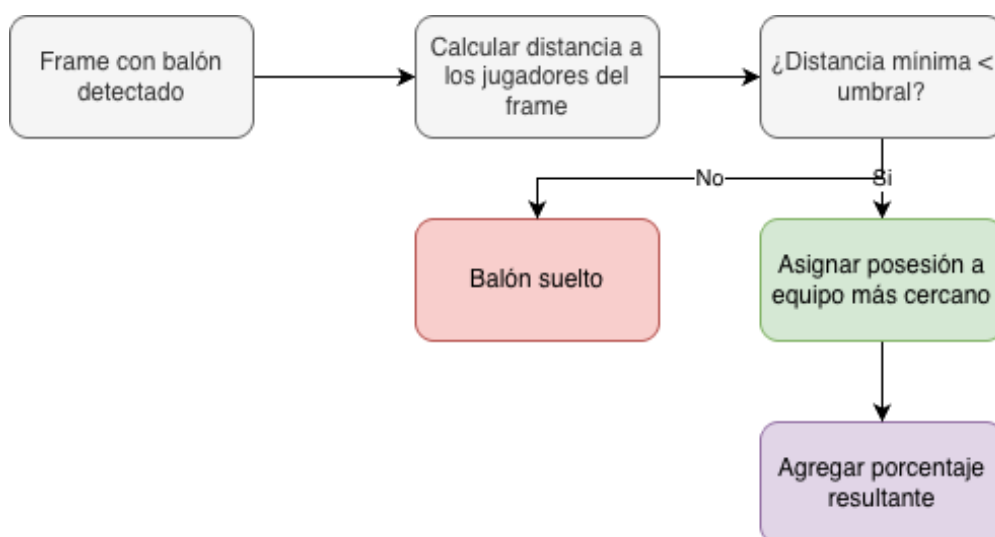


Figura 5.13: Diseño de la estimación de posesión.

5.6. Diseño de datos y artefactos

Como se ha señalado en la sección 5.1, el contrato entre fases de este sistema son los artefactos que cada una persiste en disco. La Tabla 5.1 cataloga estos artefactos por fase, con su formato y su carpeta de salida.

Tabla 5.1: Catálogo de artefactos generados por cada fase del sistema.

Fase	Artefacto	Formato	Carpeta
Tracking	Trayectorias por secuencia y algoritmo	.txt (MOT)	results_train/, results_test/
Evaluación	Métricas por secuencia y combinadas	.csv, .txt	evaluation_train/, evaluation_test/
Hiperparámetros	Trayectorias y métricas por configuración	.txt, .csv	results_hyperparam/, evaluation_hyperparam/
Hiperparámetros	Tabla resumen y gráficas comparativas	.csv, .png	evaluation_hyperparam/plots/
Equipos	Muestras de color y asignación final por jugador	.csv	metadata_output/teams/
Equipos	Visualización de la clasificación sobre un frame	.png	metadata_output/teams/
Mapas de calor	Mapa general, por equipo y con homografía	.png	metadata_output/heatmaps/, metadata_output/heatmap_homography_demo/
Homografía	Distancia y velocidad por jugador	.csv	metadata_output/distance_homography_demo/
Posesión	Posesión por frame y resumen por equipo	.csv	metadata_output/possession/
Exportación	Vídeos anotados	.mp4	videos/

5.7. Diseño de la aplicación web

Esta sección presenta el diseño de la aplicación web, ya implementada como parte de este proyecto. Se mantiene aquí, en el capítulo de diseño, porque las decisiones arquitectónicas que se describen a continuación se tomaron antes de programarla y se respetaron sin cambios durante la implementación.

5.7.1. Arquitectura de la aplicación web

Siguiendo el mismo principio de artefactos persistidos de la sección 5.1, la aplicación web se implementa como una aplicación de una sola página (*Single Page Application*, React + Vite + TypeScript) que consume exclusivamente artefactos estáticos ya exportados por el pipeline, sin ejecutar Python ni recalcular nada en el navegador, ni base de datos, ni llamadas de red más allá de peticiones `fetch` a ficheros JSON, PNG y MP4. Esta decisión, coherente con la arquitectura general del sistema, permite que la interfaz se pueda desplegar en cualquier alojamiento estático (o servirse en local).

Los artefactos que consume la aplicación no son los mismos ficheros intermedios de la Tabla 5.1, sino un paquete derivado y empaquetado específicamente para la web por `src/web_export/export_bundle.py`.

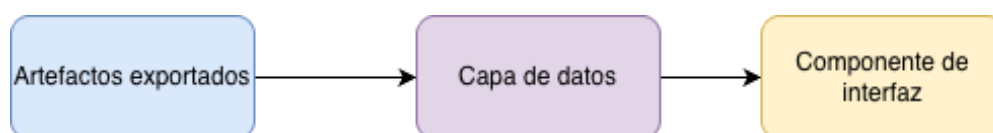


Figura 5.14: Arquitectura de la aplicación web: los artefactos exportados por `export_bundle.py` alimentan una capa de datos (peticiones `fetch` a JSON/PNG/MP4) que a su vez alimenta los componentes de interfaz de React.

5.7.2. Diseño de la interfaz de usuario

La interfaz se organiza en dos pestañas, cada una centrada en una tarea distinta: comparar el rendimiento agregado de los dos algoritmos, o explorar en detalle los resultados de una secuencia concreta.

La pestaña **Comparativa de algoritmos** reproduce los resultados ya presentados en el capítulo 7: las tablas de HOTA/HOTA(0)/DetA/AssA/MOTA/IDF1 por *split* (entrenamiento y test), la configuración óptima de hiperparámetros de cada algoritmo, y las gráficas del estudio de hiperparámetros y de la comparativa HOTA frente a HOTA(0).

La pestaña **Explorador de secuencias** (Figura 5.15) está organizada en torno a una única tarea principal: seleccionar una secuencia (SNMOT-116/117/118) evaluadas y un algoritmo (ByteTrack u OC-SORT) mediante un selector, y explorar los resultados de esa combinación.

No todas las combinaciones de secuencia y algoritmo tienen los mismos datos disponibles: SNMOT-117 no tiene puntos de calibración de homografía (sección 6.6.3), y la combinación ByteTrack/SNMOT-118 nunca llegó a generar un mapa de calor por equipo en el pipeline original. La interfaz muestra explícitamente una nota indicando que el dato no está disponible para esa combinación, para que la ausencia se lea como una característica real de los datos y no como un error de carga.

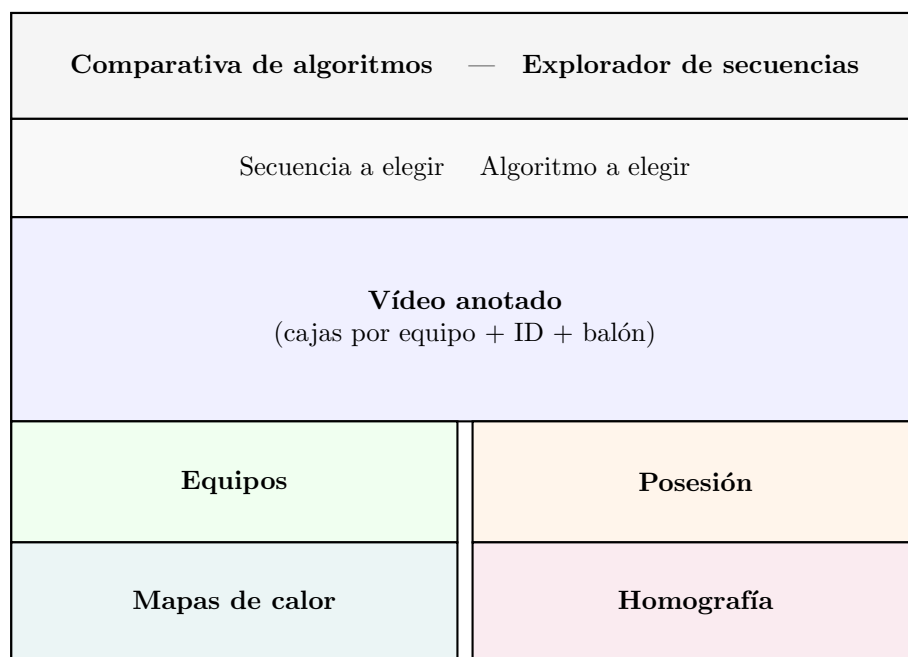


Figura 5.15: Croquis de la disposición implementada de la pestaña Explorador de secuencias

6. Implementación

Este capítulo describe cómo se ha implementado, en la práctica, el diseño presentado en el capítulo 5. El capítulo se cierra con una sección dedicada explícitamente a los enfoques que se exploraron y finalmente no se incorporaron al sistema.

6.1. Entorno y herramientas

Todo el desarrollo se ha realizado en Python 3.11, dentro de un entorno virtual (`venv_tfg`) situado dentro del propio repositorio, con las dependencias fijadas en `requirements.txt` para garantizar la reproducibilidad. La Tabla 6.1 recoge las librerías con un papel directo en el sistema. Se omiten las dependencias transitivas de bajo nivel, como `torch` o `pandas`, que `requirements.txt` incluye por ser un volcado completo del entorno pero que ningún script del proyecto importa directamente: llegan al entorno como dependencias de `boxmot` y de `ultralalytics`—.

Tabla 6.1: Librerías principales empleadas y su función en el sistema.

Librería	Versión	Función en el sistema
boxmot	10.0.43	Implementaciones de ByteTrack y OC-SORT bajo una interfaz común. Versión fijada deliberadamente por el problema P4 de la Tabla 6.2.
trackeval	sn-trackeval 0.4.0	Evaluación con métricas MOT estándar (sección 2.6). Fork de TrackEval adaptado por SoccerNet al formato de su benchmark.
opencv-python	4.13.0.92	Lectura y anotación de frames, cálculo de la homografía, (sección 6.6.3).
filterpy	1.4.5	Implementación del filtro de Kalman (sección 2.4.1) usada internamente por los trackers de boxmot.
lapx	0.9.4	Resolución del problema de asignación mediante el algoritmo húngaro (sección 2.4.3), usada internamente por los trackers de boxmot.
scikit-learn	1.8.0	K-means para la clasificación de equipos (sección 2.8).
scipy	1.17.1	Estimación de densidad por núcleos (gaussian_kde) en los mapas de calor.
matplotlib	3.10.9	Generación de todas las figuras y gráficas del proyecto.

Cabe señalar una inconsistencia menor detectada al redactar esta memoria: aunque `trackeval` está instalado y en uso real en el entorno de desarrollo (a través del paquete `sn-trackeval`), no aparece listado de forma explícita en `requirements.txt`, lo que sugiere que ese fichero no se regeneró tras instalarlo por separado. No afecta al funcionamiento del proyecto en el equipo de desarrollo actual, pero es una brecha de reproducibilidad frente al requisito RNF-001 que convendría corregir –regenerando `requirements.txt` con `sn-trackeval` explícitamente incluido– antes de que un tercero intente reproducir el entorno desde cero.

6.1.1. Traslados de equipo de desarrollo

El desarrollo no se ha mantenido en un único equipo, lo que ha exigido recrear el entorno de trabajo en dos ocasiones: el proyecto se inició en un portátil personal, se trasladó a mitad de desarrollo a un ordenador de sobremesa con GPU AMD Radeon RX 6900 XT (16 GB VRAM) en Windows, y finalmente a un equipo macOS. Cada traslado se apoyó en el mismo mecanismo: dependencias fijadas en `requirements.txt` y código versionado en Git, lo que permitió recrear el entorno funcional en el nuevo

equipo sin más incidencias que las propias de cada sistema operativo.

6.1.2. Problemas de entorno encontrados

Configurar y mantener este entorno no fue inmediato. La Tabla 6.2 documenta, siguiendo el mismo criterio de transparencia del resto de esta memoria, los catorce problemas de entorno registrados durante el desarrollo, con su solución.

De estos catorce problemas, la mayoría son propios de la configuración inicial de cualquier entorno Python (P1-P3, P8-P10) y no volvieron a aparecer una vez resueltos. Los más relevantes para entender decisiones posteriores de este proyecto son P4 y P6. P17, ya en la fase de aplicación web, muestra que ese mismo tipo de fricción de entorno no es exclusivo de Python: elegir un códec de vídeo compatible con el navegador es un problema de la misma naturaleza en un *stack* tecnológico distinto.

¹El `requirements.txt` final del proyecto fija `numpy==2.4.4`, una versión posterior a la que resolvió este problema en su momento. Esto sugiere que la incompatibilidad original fue resuelta más adelante en versiones posteriores de `boxmot` o de `OC-SORT`, aunque no se ha vuelto a verificar explícitamente la causa original tras la actualización.

Tabla 6.2: Problemas de entorno encontrados durante el desarrollo y su solución.

ID	Problema	Solución
P1	Python 3.14 incompatible con las librerías del proyecto	Fijar Python 3.11 en el entorno virtual
P2	Mezcla de entornos conda y venv activos a la vez, con conflictos de paquetes	Desactivar conda y trabajar exclusivamente con venv
P3	conda activate venv_tfg fallaba, ya que venv_tfg no es un entorno conda	Activar el entorno con el mecanismo propio de venv (source venv_tfg/bin/activate)
P4	boxmot v18 no expone las clases de cada tracker por separado, necesarias para la comparativa	Fijar la versión 10.0.43 en requirements.txt
P5	Error de tipo en kalman_filter.py de OC-SORT por incompatibilidad con la versión de NumPy instalada	Fijar numpy==1.26.4 en el momento de detectarlo ¹
P6	Detecciones de det.txt con 5 columnas en lugar de las 6 que espera boxmot	Añadir manualmente la columna de clase (valor 0) antes de pasar las detecciones al tracker
P7	La variable de entorno \$HOME se colaba en comandos al copiar y pegar instrucciones entre terminales	Revisar manualmente los comandos antes de ejecutarlos
P8	El entorno virtual se creó fuera de la carpeta del proyecto y VS Code no lo detectaba automáticamente	Recrear el entorno dentro del repositorio
P9	Los entornos virtuales de Python no son reubicables (las rutas absolutas quedan grabadas en sus scripts de activación)	Recrearlos siempre en su ubicación final, no moverlos tras crearlos
P10	requirements.txt estaba vacío	Regenerarlo con pip freeze >requirements.txt
P13	load_dotenv() no recargaba el fichero .env tras modificarlo	Forzar la recarga con load_dotenv(override=True)
P14	Carpetas basura generadas por rutas mal resueltas (path/, \$HOME/, .venv/, models/)	Identificarlas y eliminarlas manualmente
P15	Traslado del proyecto a macOS: posible incompatibilidad de entorno	Entorno recreado desde requirements.txt sin incidencias destacables; solo hubo que ajustar alguna ruta de Windows a macOS
P17	Los vídeos exportados con el fourcc mp4v de OpenCV no se reproducían en el navegador	Usar el fourcc avc1 (H.264), disponible porque OpenCV, con esto, el tamaño de cada vídeo se redujo en torno a un 75 %

6.2. Estructura del repositorio

El código del pipeline offline reside en `src/`, organizado en paquetes por responsabilidad, resumidos en la Tabla 6.3.

Tabla 6.3: Paquetes del repositorio y su responsabilidad.

Paquete	Responsabilidad
<code>src/visualizacion/</code>	Scripts exploratorios iniciales sobre el <i>ground truth</i> .
<code>src/tracking/</code>	Implementación de ByteTrack y OC-SORT, ejecución batch sobre un split completo, y exportación de vídeos anotados (<code>src/tracking/export/</code>).
<code>src/evaluation/</code>	Generación de <i>seqmaps</i> y evaluación con TrackEval sobre train y test.
<code>src/hyperparameter_study/</code>	Barrido de hiperparámetros de cada algoritmo, configuración óptima y resumen comparativo.
<code>src/metadata/</code>	Todo lo relacionado con la generación y gestión de metadatos.
<code>src/utils/</code>	Scripts de utilidad para tareas de un solo uso.
<code>src/web_export/</code>	Generación de metadatos de OC-SORT y empaquetado de todos los artefactos estáticos que consume la aplicación web (sección 6.7).

El código de la aplicación web, al no ser Python, vive fuera de `src/`: en `frontend/`, un proyecto independiente de React.(sección 6.7).

6.3. Implementación del tracking

6.3.1. Scripts de visualización inicial

Antes de implementar ningún algoritmo de tracking, la primera fase del proyecto (sección 3.2) se dedicó a comprender el formato del dataset mediante scripts exploratorios, todos ellos trabajando directamente sobre el *ground truth* (`gt.txt`) y no sobre predicciones reales de ningún tracker: `reproducir_vid.py` reproduce una secuencia como vídeo con OpenCV; `bboxes_ids.py` dibuja las cajas del *ground truth* en verde con su identificador; y `bboxes_ids_dif_colors.py` añade un color único y persistente por identificador, para facilitar el seguimiento visual de cada jugador entre frames.

El último de estos scripts exploratorios, `bboxes_balltrack_geoprop.py`, ya

incorpora el filtro geométrico de separación del balón (sección 6.3.2) y dibuja el balón en rojo, distinto de los jugadores. Es interesante notar que `bboxes_ids.py`.

6.3.2. Filtrado geométrico del balón

Como se diseñó en la sección 5.2.1, el balón se separa de las detecciones de personas mediante dos condiciones sobre su caja delimitadora, aplicadas de forma idéntica en todos los scripts del proyecto que necesitan distinguir balón de jugadores:

```
BALL_MAX_AREA = 2000    BALL_MIN_RATIO = 0.7    BALL_MAX_RATIO = 1.3
```

Una detección se clasifica como balón cuando su área en píxeles ($w \times h$) es inferior a `BALL_MAX_AREA` y, a la vez, su proporción w/h está dentro del rango `[BALL_MIN_RATIO, BALL_MAX_RATIO]`, es decir, cuando la caja es aproximadamente cuadrada. Estos tres valores se fijaron de forma que, observando las dimensiones típicas del balón frente a las de un jugador en las secuencias de SoccerNet, y se mantienen constantes en todo el proyecto para que el balón se identifique de forma consistente en todas las fases.

6.3.3. ByteTrack y OC-SORT

Ambos algoritmos se implementan siguiendo el bucle diseñado en la sección 5.2.2, instanciando en cada caso la clase correspondiente de `boxmot`: `BYTETracker()` en `bytetracker_alg.py`, con su configuración por defecto como punto de partida, y `OCSORT` también con su configuración por defecto en `sort_alg.py`.

El nombre de este último fichero, `sort_alg.py`, es una muestra histórica, documentada en la sección 2.5.1: la intención original era implementar el SORT clásico, pero `boxmot` en su versión fijada (P4) no expone una clase `SORT` independiente, solo `OCSORT`, su evolución directa. El fichero conserva su nombre original como constancia de esa decisión, aunque su contenido implementa en realidad OC-SORT.

Las detecciones que llegan de `det.txt` en formato (x, y, w, h) se convierten al formato $(x_1, y_1, x_2, y_2, \text{conf}, \text{clase})$ que espera `boxmot` antes de invocar `tracker.update()` en cada frame, y el resultado se reasigna a una numeración secuencial mediante un diccionario de correspondencia, antes de escribirse en el fichero de resultados con el formato MOT estándar.

6.3.4. Ejecución batch y reorganización de resultados

La ejecución sobre la totalidad de un split se implementa en `run_all_train.py` y `run_all_test.py`, que recorren todas las secuencias del directorio del split correspondiente y aplican, para cada una y cada algoritmo configurado, el mismo bucle de tracking descrito en la sección anterior (Figura 5.4). Ambos scripts comparten la misma lógica de filtrado del balón y de escritura de resultados que las versiones de un único algoritmo y una única secuencia.

En una primera versión del proyecto, los resultados de cada secuencia se escribían en una subcarpeta propia (`results/bytetracker/SNMOT-060/SNMOT-060.txt`), estructura que `TrackEval` no esperaba de forma directa. Este script recorrió una única vez los resultados ya generados y los aplanó a la estructura que finalmente adopta el proyecto (`results/bytetracker/SNMOT-060.txt`), sin necesidad de rehacer el tracking desde cero. No forma parte del flujo habitual de ejecución del sistema, pero se conserva en el repositorio como constancia de esa reorganización.

6.4. Implementación de la evaluación

La evaluación (diseñada en la sección 5.3) se implementa en dos scripts, `evaluate_train.py` y `evaluate_test.py`, que configuran `TrackEval` con los mismos parámetros salvo el split evaluado:

```
dataset_config['BENCHMARK']          = 'SoccerNet'
dataset_config['TRACKERS_TO_EVAL']    = ['bytetracker', 'ocsort']
dataset_config['SKIP_SPLIT_FOL']      = True
dataset_config['DO_PREPROC']          = False
metrics_config = {'METRICS': ['HOTA', 'MOTA', 'IDF1'], 'THRESHOLD': 0.5}
```

donde `BENCHMARK = 'SoccerNet'` indica a `TrackEval` que use el *seqmap* generado por `seqmaps_creation_test.py/seqmaps_creation_train.py` (sección 5.3) en lugar de un benchmark predefinido, y `THRESHOLD = 0.5` fija el umbral de IoU al que se calculan MOTA e IDF1, ambas métricas de umbral único.

6.4.1. Aclaración metodológica: HOTA frente a HOTA(0)

Antes de presentar cualquier resultado numérico en el capítulo 7, es necesario dejar constancia de un matiz metodológico detectado al preparar esta memoria, en cumplimiento del requisito de documentación honesta. `TrackEval` exporta, entre sus columnas de salida, tanto **HOTA**, promediada sobre el rango completo de umbrales de IoU de 0,05 a 0,95, como **HOTA(0)**².

El script de resumen del estudio de hiperparámetros (`summarize_hyperparam.py`) lee explícitamente la columna `HOTA(0)` de cada resultado:

```
hota = float(row.get('HOTA(0)', row.get('HOTA__50', 0))) * 100
```

lo que significa que **todos los valores de “HOTA” reportados en el estudio de hiperparámetros de este proyecto son en realidad HOTA(0)**, es decir, HOTA evaluada únicamente al 50 % de solapamiento IoU, y no la métrica HOTA integrada que describe la sección 2.6.3. Ambas son métricas válidas y ambas las calcula `TrackEval` de forma estándar, pero no son intercambiables: al integrar sobre un rango de umbrales en lugar de uno solo, la HOTA completa es más exigente y, en

²La notación `HOTA(0)` es la que usa `TrackEval` internamente para referirse al valor de HOTA en el primer umbral de su rango de evaluación, que con la configuración de este proyecto (`THRESHOLD = 0.5`) coincide con el umbral de 0,5 usado también por MOTA e IDF1.

general, produce un valor distinto. El capítulo 7 presenta ambos valores por separado allí donde la distinción afecta a las conclusiones, y este matiz se retoma explícitamente donde corresponda para no inducir a una lectura equivocada de los resultados.

6.5. Implementación del estudio de hiperparámetros

El estudio de hiperparámetros se aplica siempre sobre el conjunto de **test**, para que la configuración resultante no esté ajustada a datos ya vistos durante el desarrollo inicial. Cada algoritmo tiene su propia configuración base y su propio rango de búsqueda por hiperparámetro, recogidos en la Tabla 6.4.

Tabla 6.4: Configuración base y rango de búsqueda de cada hiperparámetro.

Algoritmo	Hiperparámetro	Base	Rango de búsqueda
ByteTrack	<code>track_thresh</code>	0.45	0.2, 0.3, 0.45, 0.6
	<code>match_thresh</code>	0.8	0.6, 0.7, 0.8, 0.9
	<code>track_buffer</code>	25	10, 20, 25, 40
OC-SORT	<code>det_thresh</code>	0.3	0.1, 0.2, 0.3, 0.4, 0.5
	<code>max_age</code>	30	10, 20, 30, 50
	<code>min_hits</code>	3	1, 2, 3, 5
	<code>iou_threshold</code>	0.3	0.1, 0.2, 0.3, 0.4, 0.5

Los resultados de este barrido se resumen en la Tabla 6.5 para los hiperparámetros con efecto observable. Los hiperparámetros `track_thresh` (ByteTrack) y `det_thresh` (OC-SORT) no mostraron ningún efecto sobre la métrica en el rango probado, y se omiten de la tabla por brevedad.

A partir de estos resultados individuales, `bytetracker_optimal.py` y `ocsort_optimal.py` fijan y validan la configuración combinada que RF-008 exige confirmar de forma independiente: `match_thresh=0.9`, `track_buffer=10` para ByteTrack (manteniendo `track_thresh=0.45`, sin efecto), y `iou_threshold=0.1`, `max_age=10`, `min_hits=1` para OC-SORT (manteniendo `det_thresh=0.3`, sin efecto). Al ejecutar y evaluar estas configuraciones combinadas de forma independiente, se obtiene $\text{HOTA}(0) = 81,43$ para ByteTrack y $\text{HOTA}(0) = 80,92$ para OC-SORT.

Es interesante notar que la configuración combinada de ByteTrack (81,43) resulta ligeramente *inferior* al mejor valor individual de `match_thresh` por sí solo (81,48 con `match_thresh=0.9` y `track_buffer` sin modificar): los efectos de ambos hiperparámetros no son estrictamente aditivos, y combinar el mejor valor de cada uno por separado no garantiza el mejor resultado conjunto. En OC-SORT, en cambio, la combinación de los tres hiperparámetros (80,92) mejora claramente sobre cualquiera de ellos por separado (mejor individual: 80,17 con `iou_threshold=0.1`), lo que indica que en este caso sí se refuerzan entre sí. Esta configuración óptima de ByteTrack, por

Tabla 6.5: Efecto de cada hiperparámetro sobre HOTA(0) (%) en el conjunto de test.

Hiperparámetro	Valor del hiperparámetro				
match_thresh (BT)	0.6	0.7	0.8 (base)	0.9	–
HOTA(0)	74.62	78.03	80.20	81.48	–
track_buffer (BT)	10	20	25 (base)	40	–
HOTA(0)	80.36	80.26	80.20	80.12	–
iou_threshold (OC)	0.1	0.2	0.3 (base)	0.4	0.5
HOTA(0)	80.17	78.71	75.51	70.90	64.49
max_age (OC)	10	20	30 (base)	50	–
HOTA(0)	76.13	75.75	75.51	75.13	–
min_hits (OC)	1	2	3 (base)	5	–
HOTA(0)	76.26	75.90	75.51	74.78	–

ser la de mayor HOTA(0) absoluto entre ambos algoritmos, es la que se usa como base de todo el pipeline de metadatos de la sección 6.6.

Por último, `summarize_hyperparam.py` consolida todos estos resultados en `evaluation_hyperparam/summary_table.csv` y genera una gráfica de barras por hiperparámetro y algoritmo en `evaluation_hyperparam/plots/`.

6.6. Implementación del pipeline de metadatos

Todo el pipeline de metadatos parte de los resultados de tracking generados con la configuración óptima de ByteTrack, como se justificó en la sección anterior.

6.6.1. Clasificación de equipos

El script `team_classification.py` implementa el diseño en dos niveles de K-means de la sección 5.5.1. Para limitar el coste computacional, no procesa todos los frames de la secuencia, sino un muestreo de 1 de cada 5 (`SAMPLE_EVERY = 5`); dado que un jugador aparece en decenas de frames a lo largo de una secuencia, este muestreo basta para determinar su equipo mayoritario sin analizar la secuencia completa. Para cada detección muestreada, se recorta la región superior de su caja delimitadora para aislar la zona de la camiseta y evitar incluir piernas, césped o el balón en el recorte, antes de aplicar K-means con $k = 1$ para obtener su color dominante en espacio de color BGR. Los colores dominantes de todas las muestras de la secuencia se agrupan después con K-means $k = 2$, y el equipo final de cada `track_id` se determina por mayoría entre sus muestras individuales, así evita que el ruido puede clasificar mal alguna muestra puntual sin afectar al resultado global.

6.6.2. Mapas de calor

El pipeline de mapas de calor tiene, en realidad, dos implementaciones, resultado de una iteración de diseño explícita documentada aquí: `general_heatmap.py`, la primera versión, representa las posiciones directamente en píxeles de cámara mediante `hexbin` de `matplotlib`, sin proyectar sobre ningún campo dibujado; `heatmap_field.py`, la versión final y la que se usa en el resto del pipeline, sustituye `hexbin` por una estimación de densidad por núcleos (`gaussian_kde` de `scipy`) proyectada sobre un campo de $105\text{ m} \times 68\text{ m}$ dibujado a escala (Figura 2.8), con dos mejoras concretas sobre la primera versión: un degradado continuo que también representa las zonas de baja densidad y un escalado por el rango observado de posiciones (percentiles 5 y 95). El script `heatmap_by_team.py` reutiliza exactamente esta misma lógica de proyección y KDE ya validada en `heatmap_field.py`, aplicándola por separado a las posiciones de cada equipo pero con una única escala común a ambos, para que los dos mapas resultantes sean directamente comparables entre sí (Figura 5.10).

6.6.3. Homografía real: distancia y velocidad

Como se diseñó en la sección 5.5.3, la homografía real se calcula sobre un tramo concreto de cámara estable de cada secuencia de demostración, identificando manualmente 4 puntos de referencia en un frame representativo del tramo. Para SNMOT-116, el tramo usado es el intervalo de frames $[1, 380]$, correspondiente a una jugada de córner con cámara estable; para SNMOT-118, el intervalo $[350, 650]$. Los puntos de imagen se identifican sobre un frame de referencia de 1456×816 píxeles y se reescalan a la resolución real de las secuencias (1920×1080) antes de calcular la homografía, y se corresponden con las coordenadas reales del área pequeña según las medidas FIFA.

Antes de usarse, la matriz de homografía calculada se valida re proyectando estos mismos 4 puntos y comparando el resultado con las coordenadas de campo esperadas: para el tramo de SNMOT-116, el error medio de re proyección obtenido es de apenas unos pocos centímetros, lo que confirma que la matriz describe correctamente la transformación en la zona próxima a los puntos de calibración. A partir de esa matriz, la posición de cada jugador se proyecta a metros reales frame a frame, y de esa secuencia de posiciones proyectadas se calcula la distancia total recorrida (suma de las distancias euclídeas entre posiciones consecutivas) y la velocidad, tanto media como de pico –esta última como el percentil 95 de las velocidades instantáneas calculadas sobre una ventana de 1 segundo.

Para los mapas de calor calibrados con esta homografía real (`heatmap_homography_demo_116.py` y `heatmap_homography_demo_118.py`), el diseño incorpora además el sombreado de honestidad visual descrito en la sección 5.5.3 (Figura 5.12): las zonas del campo alejadas de los 4 puntos de calibración se oscurecen para indicar visualmente que la proyección en esa región no es fiable. Para SNMOT-118, además, dos identificadores de tracking (`EXCLUDED_TRACK_IDS = [328, 274]`) se excluyen manualmente del mapa de calor calibrado: corresponden a

jugadores cuya posición proyectada se aleja del campo de forma incompatible con un movimiento humano real, señal de que su trayectoria de tracking se desvía significativamente de la zona de cámara estable para la que se calculó esa homografía concreta.

6.6.4. Posesión de balón

El script `ball_possesion.py` implementa la heurística de proximidad diseñada en la sección 5.5.4: para cada frame en que el filtro geométrico detecta un balón en el `det.txt` original, se calcula la distancia euclídea entre su centro y el centro de cada jugador presente en ese frame según los resultados de tracking, y se asigna la posesión al equipo del jugador más cercano siempre que esa distancia mínima no supere `MAX_POSSESSION_DISTANCE_PX = 150` píxeles; por encima de ese umbral, el balón se considera suelto y el frame no computa para ningún equipo. El resultado se agrega tanto a nivel de frame (`_possession.csv`) como agregado en porcentaje por equipo (`_possession_summary.csv`).

6.7. Implementación de la aplicación web

Esta sección detalla la implementación de la aplicación web diseñada en la sección 5.7: los dos scripts nuevos de `src/web_export/` que preparan los datos, y el proyecto `frontend/` que los consume.

6.7.1. Generación de metadatos para OC-SORT

El pipeline de metadatos original solo se había ejecutado sobre la configuración óptima de ByteTrack. Para que el selector de algoritmo de la aplicación web tuviera sentido también en el explorador de secuencias, `src/web_export/compute_metadata.py` reproduce exactamente la misma lógica parametrizada por algoritmo, en lugar de duplicar los cinco scripts de `src/metadata/` o modificarlos. Los 4 puntos de calibración de homografía descritos en la sección 6.6.3 se reutilizan sin modificación para OC-SORT, porque dependen únicamente de la posición fija de la cámara en ese tramo de vídeo, no del algoritmo de tracking.

Para no mezclar ni sobrescribir los artefactos de ByteTrack ya generados, la salida de OC-SORT se escribe en un árbol de carpetas paralelo, con el mismo formato de nombre de fichero que la rama de ByteTrack. La ejecución sobre las tres secuencias de demostración produjo por ejemplo porcentajes de posesión por equipo muy próximos a los ya obtenidos con ByteTrack en el capítulo 7 para las mismas secuencias: por ejemplo, en SNMOT-116 el equipo dominante se queda con el 86,2 % de la posesión con OC-SORT frente al 86,0 % con ByteTrack (aunque a qué equipo, 0 o 1, le corresponde ese porcentaje mayoritario puede no coincidir entre ambas ejecuciones, ya que la etiqueta numérica que K-means asigna a cada grupo es arbitraria y no se fija entre ejecuciones independientes). Esta cercanía es una comprobación indirecta de que la heurística de

posesión por proximidad (sección 6.6.4) es razonablemente estable frente al algoritmo de tracking subyacente, ya que ambos parten de las mismas detecciones y solo difieren en cómo las asocian entre frames.

6.7.2. Empaquetado de datos estáticos para la web

`src/web_export/export_bundle.py` genera, para cada una de las combinaciones de secuencia y algoritmo, el paquete de artefactos que consume la interfaz, bajo `frontend/public/data/`:

- **Vídeo anotado:** Este vídeo se dibuja directamente sobre el fichero de resultados ya calculado con la configuración óptima, sin recomputar nada: las cajas se colorean por equipo a partir del CSV de clasificación correspondiente, con el balón resaltado en un color distinto, sobre el tramo de frames ya usado en la homografía o un tramo por defecto de longitud comparable. Cada vídeo se reescala a 960px de ancho antes de codificarse, para mantener un tamaño razonable sin perder legibilidad de las cajas.
- **Imágenes:** copia directa de los mapas de calor y del mapa homografiado ya generados en las secciones 6.7.1 y 6.6.
- **sequences.json:** un registro por combinación de secuencia y algoritmo, con el recuento de jugadores por equipo, el porcentaje de posesión, el top-5 de distancia/velocidad y las rutas a vídeo e imágenes; los campos correspondientes a datos que no existen para esa combinación concreta se serializan como `null` en lugar de omitirse, para que la interfaz pueda distinguir explícitamente «no calculado» de «campo ausente por error».

La primera versión de los vídeos, codificada con el *fourcc* `mp4v` (MPEG-4 Part 2) de OpenCV, no se reproducía en ningún navegador (problema P17, Tabla 6.2): los navegadores modernos solo soportan H.264/H.265 dentro de un contenedor `.mp4`, no MPEG-4 Part 2. Cambiar el *fourcc* a `avc1` (fue posible porque la instalación de OpenCV usada en este proyecto está compilada con soporte de FFmpeg) resolvió el problema.

6.7.3. Frontend

La parte del frontend es un proyecto independiente creado con `vite`, y en producción se compila a HTML/CSS/JS estático. La configuración de Vite fija `base: './'` para que las rutas del build resultante sean relativas y el sitio se pueda desplegar en cualquier subruta o servirse en local sin tocar configuración.

La interfaz reproduce el diseño de la sección 5.7.2: un componente `App.tsx` con una cabecera fija y dos pestañas que alternan entre `ComparativaAlgoritmos` y `ExploradorSecuencias`. Este último mantiene en estado local la secuencia y el algoritmo seleccionados, y renderiza cinco componentes de panel (`VideoPanel`, `TeamsPanel`, `PossessionPanel`, `HeatmapPanel`, `HomografiaPanel`) a partir de la entrada de `sequences.json` que coincide con esa combinación; ninguno de estos

componentes calcula nada, se limitan a formatear y mostrar los campos ya presentes en el JSON, incluyendo el caso en que un campo llega a `null`, donde el panel muestra una nota explícita de «no disponible para esta combinación» para que se lea como una característica de los datos y no como un fallo de carga.

La paleta de la interfaz sigue un esquema oscuro, con un único color de acento reutilizado en el subrayado de la pestaña activa, los selectores de secuencia y algoritmo, y el propio logotipo del proyecto, el mismo tono ya usado dentro de la tabla comparativa para resaltar el mejor valor de cada métrica entre los dos algoritmos: una única decisión de color reutilizada en todo el sitio en lugar de introducir varias.

6.8. Enfoques explorados y descartados

Como se ha ido señalando a lo largo de este capítulo y del proyecto en general, no todo lo probado durante este proyecto llegó a incorporarse al sistema final. Esta sección reúne, a modo de resumen, los enfoques explorados y descartados y la razón de su descarte.

6.8.1. Detección del balón con YOLO

Como se explicó en la sección 2.7, la primera aproximación para separar el balón de los jugadores no fue el filtro geométrico finalmente adoptado (sección 6.3.2), sino un detector de objetos entrenado con aprendizaje profundo, **YOLOv8n** [22], aplicado directamente sobre los frames. Se descartó tras las primeras pruebas porque, sin reentrenamiento específico para balones de fútbol en las condiciones de cámara e iluminación de SoccerNet, el modelo preentrenado genérico obtenía resultados muy pobres, con falsos negativos frecuentes y falsos positivos sobre otros objetos redondeados y claros del campo.

6.8.2. Clasificación de equipos con un tercer grupo para el árbitro

Como se documentó en la sección 2.8, la clasificación de equipos por K-means asume $k = 2$ grupos, lo que provoca que el árbitro (que tiene una camiseta de color distinto a la de ambos equipos) se asigne incorrectamente al centroide de color más próximo. Se valoró una variante con $k = 3$, combinada con una heurística adicional que etiquetaría como árbitro al grupo con menos detecciones totales (teniendo en cuenta que solo hay un árbitro principal frente a unos once jugadores por equipo), pero no llegó a implementarse porque el enfoque con $k = 2$ ya ofrecía resultados suficientemente buenos para los objetivos de este proyecto, y la mejora quedó pospuesta como línea de trabajo futuro (capítulo 10) en lugar de invertir tiempo adicional en una heurística no estrictamente necesaria en esta fase.

6.8.3. Homografía generalizada a toda la secuencia

La homografía real (sección 6.6.3) se ha limitado deliberadamente, desde el principio, a los tramos concretos de cámara estable identificados en cada secuencia de demostración, y nunca se intentó generalizarla al resto del dataset. La razón es estructural, no de tiempo disponible: la mayoría de las secuencias de SoccerNet tienen movimiento de cámara continuo a lo largo de todo su vídeo, lo que exigiría recalcular la matriz de homografía de forma dinámica, frame a frame (lo que es un problema de calibración de cámara automática considerablemente más complejo que el cálculo manual y estático empleado aquí, y que queda fuera del alcance de este proyecto). Acotar expresamente el alcance de esta técnica a una prueba de concepto, en lugar de forzar su aplicación al resto del dataset con resultados poco fiables, es coherente con el riesgo identificado y mitigado en la Tabla 3.10 del capítulo 3.

7. Resultados

Este capítulo presenta los resultados obtenidos con el sistema implementado en el capítulo 6: la comparativa de ByteTrack y OC-SORT sobre entrenamiento y test, el efecto del estudio de hiperparámetros, y los resultados del pipeline de metadatos sobre las secuencias de demostración. El capítulo dedica una atención especial a la distinción entre HOTA y HOTA(0) introducida en la sección 6.4.1, ya que cambia de forma sustancial la conclusión comparativa entre ambos algoritmos. El capítulo se cierra con una discusión consolidada de las limitaciones de todos los resultados presentados.

7.1. Descripción de experimentos

Todos los resultados de este capítulo se han obtenido con la configuración descrita en el capítulo 6: TrackEval configurado con BENCHMARK='SoccerNet' y THRESHOLD=0.5 (sección 6.4), sobre las 57 secuencias del conjunto de entrenamiento y las 44 del conjunto de test de SoccerNet-Tracking. Por cada combinación de algoritmo y split se reportan tres métricas (HOTA, MOTA e IDF1 (sección 2.6)), y adicionalmente HOTA(0) allí donde la distinción con HOTA integrada es relevante para la lectura correcta del resultado.

7.2. Estrategia de validación

La comparación entre algoritmos se apoya en dos evaluaciones con propósitos distintos: una primera evaluación sobre el conjunto de **entrenamiento**, obtenida durante el desarrollo inicial y usada para validar que el pipeline de tracking y evaluación funcionaba correctamente, y una segunda evaluación sobre el conjunto de **test**, realizada para sustentar las conclusiones comparativas de este capítulo. Ninguna decisión de diseño ni de hiperparámetros se ha tomado consultando los resultados de test antes de esta evaluación final, de modo que el conjunto de test se comporta como datos no vistos.

7.3. Comparativa de algoritmos: ByteTrack frente a OC-SORT

La Tabla 7.1 recoge los resultados de ambos algoritmos, en su configuración por defecto (sin el ajuste de hiperparámetros de la sección 7.4), sobre entrenamiento y test.

La lectura inmediata de la columna HOTA, la métrica integrada y más completa de las tres empleadas en este proyecto (sección 2.6.3) es clara: **OC-SORT supera a**

Tabla 7.1: Resultados de ByteTrack y OC-SORT sobre entrenamiento y test con la configuración por defecto.

Split	Algoritmo	HOTA	HOTA(0)	DetA	AssA	MOTA	IDSW
Train	ByteTrack	73.27	82.97	81.59	65.86	92.15	3 866
	OC-SORT	80.23	80.34	92.11	69.88	92.01	2 205
Test	ByteTrack	70.73	80.20	80.54	62.18	90.78	5 561
	OC-SORT	75.43	75.51	87.23	65.22	87.01	2 046

ByteTrack tanto en entrenamiento como en test, tanto en detección (DetA) como en asociación (AssA), y con considerablemente menos cambios de identidad (IDSW). Solo en MOTA la métrica más sesgada hacia la detección y menos exigente con la asociación ByteTrack iguala o supera ligeramente a OC-SORT.

7.3.1. Por qué HOTA y HOTA(0) son distintas

La columna HOTA(0) de la Tabla 7.1 tiene como valor de HOTA en el único umbral IoU de 0,5, en lugar de integrado sobre el rango completo (sección 6.4.1) cuenta una historia distinta y, en test, directamente contraria: bajo HOTA(0), ByteTrack (80,20) supera claramente a OC-SORT (75,51). Es precisamente esta lectura la que se usó, sin advertirlo explícitamente en su momento, en las anotaciones de seguimiento de este proyecto durante el desarrollo, y la que llevó a concluir entonces que *“en test los resultados se invierten a favor de ByteTrack”*. La Figura 7.1 contrasta ambas lecturas.

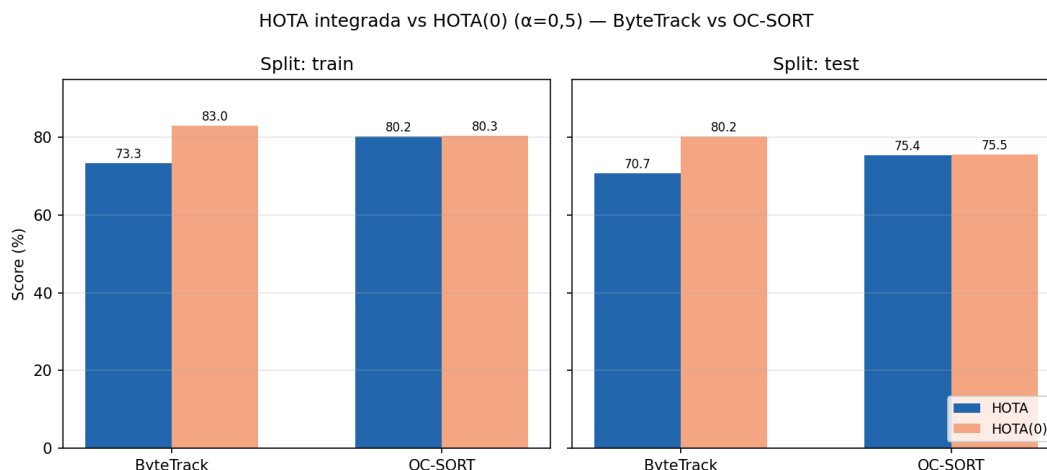


Figura 7.1: HOTA integrada frente a HOTA(0) para ambos algoritmos. Bajo HOTA integrada, OC-SORT gana en ambos splits; bajo HOTA(0), la lectura se invierte a favor de ByteTrack.

Esta discrepancia no es un error de cálculo ni una casualidad estadística: tiene una explicación mecánica concreta, visible en las columnas de precisión de localización (LocA) del fichero `pedestrian_summary.txt` de `TrackEval`. OC-SORT alcanza una LocA y una DetPr prácticamente perfectas (superiores al 99,9 % en ambos splits), lo que

indica que **OC-SORT reutiliza directamente las cajas del `det.txt` original sin modificarlas**, coherente con su diseño centrado en la observación. ByteTrack, en cambio, obtiene una LocA sensiblemente más baja (89,8 % en train, 89,7 % en test), lo que indica que sus cajas de salida se desvían algo más de las detecciones originales (previsiblemente por la interpolación de posición que realiza el filtro de Kalman durante su segunda ronda de asociación con detecciones de baja confianza (sección 2.5.3)).

Esa imprecisión de localización de ByteTrack apenas penaliza a HOTA(0), calculada en un único umbral de IoU relativamente permisivo (0,5), pero se paga cara en la HOTA integrada, que promedia también sobre umbrales mucho más exigentes (hasta 0,95): a esos umbrales estrictos, una caja ligeramente desplazada dejará de solapar lo suficiente con el *ground truth* para contar como acierto, mientras que las cajas casi perfectas de OC-SORT siguen acertando. En definitiva, **ByteTrack detecta más objetos y con más consistencia de identidad relativa (mayor DetRe), pero los localiza con menos precisión; OC-SORT detecta algo menos pero con una fidelidad casi perfecta a la detección original**, y es esta diferencia de precisión geométrica, no de robustez de asociación, la que explica por qué la métrica más exigente (HOTA integrada) favorece a OC-SORT mientras que la métrica de umbral único (HOTA(0)) favorece a ByteTrack.

La conclusión que se extrae de este contraste, y que corrige la lectura inicial de las anotaciones de desarrollo de este proyecto, es que **OC-SORT es el algoritmo con mejor rendimiento global en este dataset**, tanto en entrenamiento como en test, cuando se evalúa con la métrica íntegra y más robusta del campo. Dicho esto, ByteTrack no queda descartado sin matices: su menor precisión de localización es perfectamente compatible con aplicaciones menos exigentes en exactitud geométrica de la caja, y su comportamiento bajo ajuste de hiperparámetros resulta, de hecho, más estable que el de OC-SORT.

7.4. Resultados del estudio de hiperparámetros

El estudio de hiperparámetros se reporta en términos de HOTA(0) y no de HOTA integrada. La Figura 7.2 muestra el efecto de los dos hiperparámetros con mayor impacto observado, uno por algoritmo.

El resultado más relevante de este estudio, ya adelantado en la sección 6.5, es que **el ajuste de hiperparámetros beneficia mucho más a OC-SORT que a ByteTrack**: OC-SORT mejora su HOTA(0) en test de 75,51 a 80,92 (+5,41 puntos) con la configuración óptima combinada, mientras que ByteTrack mejora de 80,20 a 81,43 (+1,23 puntos). Dicho de otro modo, ByteTrack ya operaba cerca de su óptimo con los valores por defecto, mientras que OC-SORT requería un ajuste explícito para desplegar todo su rendimiento. Esta observación es coherente con la comparativa de la sección 7.3: si se repitiera la comparación de HOTA integrada usando las configuraciones óptimas de ambos algoritmos en lugar de sus valores por defecto, es razonable esperar que la ventaja de OC-SORT sobre ByteTrack se mantenga o incluso se amplíase, dado que es OC-SORT quien más partido saca del ajuste.

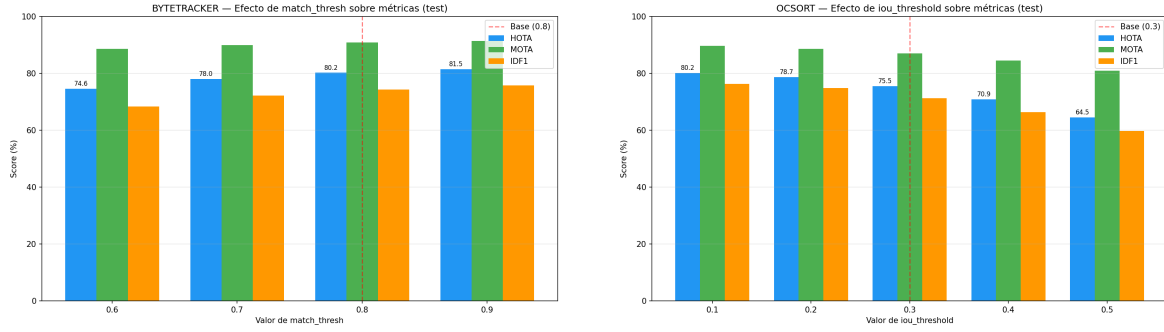


Figura 7.2: Efecto de `match_thresh` sobre ByteTrack (izquierda) y de `iou_threshold` sobre OC-SORT (derecha), los dos hiperparámetros de mayor impacto individual encontrados en el estudio (sección 6.5).

Por ser la configuración de mayor HOTA(0) absoluto entre las dos configuraciones óptimas (81,43 frente a 80,92), **ByteTrack con `match_thresh=0.9` y `track_buffer=10`** es la que se ha usado como base de todo el pipeline de metadatos de la siguiente sección, una decisión tomada en su momento con la información de HOTA(0) disponible entonces.

7.5. Resultados del pipeline de metadatos

Esta sección presenta los resultados del pipeline de metadatos sobre las secuencias de demostración SNMOT-116, SNMOT-117 y SNMOT-118, obtenidos sobre la configuración óptima de ByteTrack.

7.5.1. Clasificación de equipos

La Tabla 7.2 resume el resultado de la clasificación de equipos por K-means (sección 6.6.1) sobre las tres secuencias de demostración.

Tabla 7.2: Resultado de la clasificación de equipos por secuencia.

Secuencia	Jugadores clasificados	Equipo 0	Equipo 1
SNMOT-116	61	29	32
SNMOT-117	46	16	30
SNMOT-118	44	24	20

La verificación visual (`team_visualization.py`) confirma que la separación por color de camiseta funciona correctamente para los jugadores de ambos equipos en las

tres secuencias, como muestra la Figura 7.3. Como se documentó en la sección 2.8, ningún `track_id` queda etiquetado como “árbitro”: con $k = 2$, el árbitro presente en la secuencia se asigna incorrectamente al equipo de color más cercano al de su uniforme, una limitación conocida.

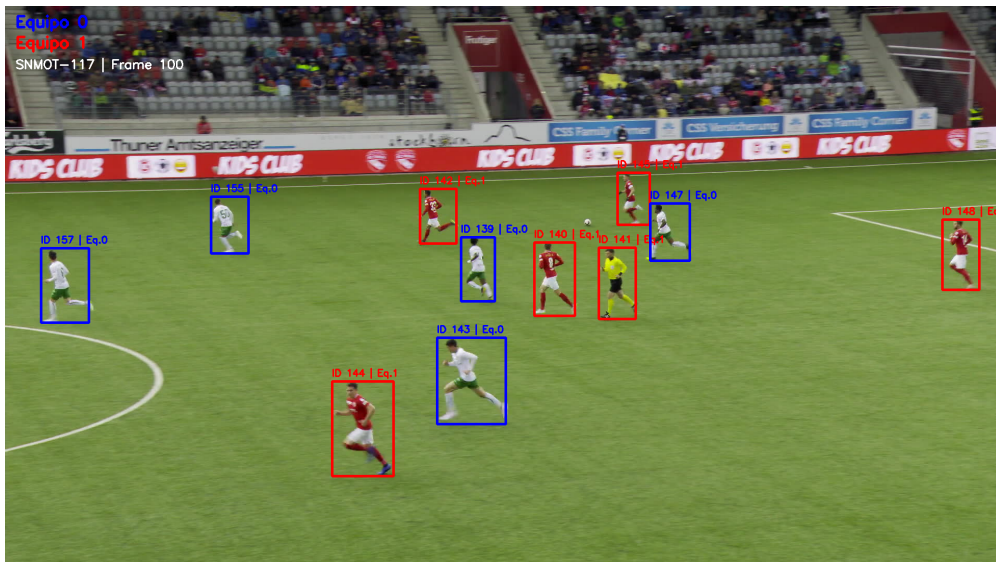


Figura 7.3: Clasificación de equipos sobre un frame de SNMOT-117 (`team_visualization.py`).

7.5.2. Mapas de calor

La Figura 7.4 muestra el mapa de calor general sin homografía, con la proyección aproximada por rango observado de SNMOT-116, complementario al mapa de calor por equipo de SNMOT-117 ya mostrado en la Figura 5.10 del capítulo 5.

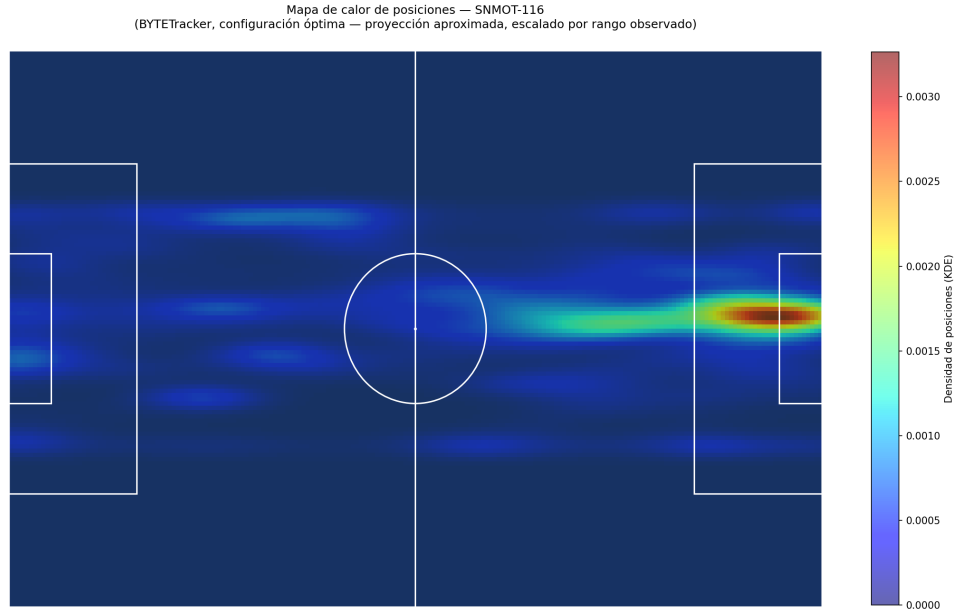


Figura 7.4: Mapa de calor general de SNMOT-116, proyección aproximada sin homografía.

7.5.3. Distancia y velocidad por homografía

La Tabla 7.3 recoge los cinco jugadores con mayor distancia recorrida en cada uno de los dos tramos de demostración analizados con homografía real.

Tabla 7.3: Jugadores con mayor distancia recorrida en cada tramo analizado con homografía real.

Secuencia	ID	Distancia (m)	Vel. media (km/h)	Vel. pico p95 (km/h)
SNMOT-116	14	46,81	7,12	21,23
	13	45,06	7,67	17,42
	11	41,12	6,46	18,83
	9	41,07	8,15	17,63
	8	40,55	7,34	16,73
SNMOT-118	279	48,07	12,92	28,76
	273	33,03	8,68	28,11
	309	23,18	7,55	13,50
	335	22,82	6,53	12,16
	271	22,77	5,67	10,58

Las magnitudes resultan físicamente plausibles para tramos tan cortos: 25 y 16 jugadores rastreados respectivamente en cada tramo (todos los `track_id` con posiciones dentro de la ventana analizada), con distancias de hasta 48m y velocidades de pico de hasta 28,8km/h en el tramo de SNMOT-118, coherentes con un sprint corto durante una jugada ofensiva. La Figura 7.5 muestra el mapa de calor con homografía real de SNMOT-118, complementario al de SNMOT-116 ya mostrado en la Figura 5.12.

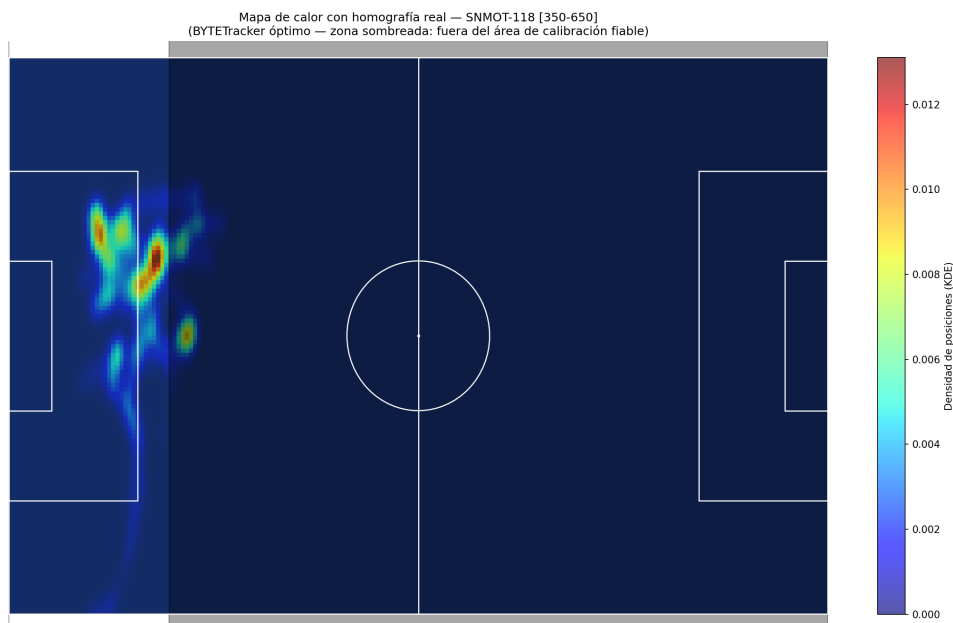


Figura 7.5: Mapa de calor con homografía real sobre el tramo de SNMOT-118; la zona sombreada indica dónde la calibración no es fiable.

7.5.4. Posesión de balón

La Tabla 7.4 recoge el porcentaje de posesión por equipo, estimado por proximidad espacial, en las tres secuencias de demostración.

Tabla 7.4: Posesión de balón por equipo y secuencia (estimada por proximidad).

Secuencia	Equipo 0	Equipo 1	Frames con posesión asignada
SNMOT-116	85,95 %	14,05 %	612
SNMOT-117	71,81 %	28,19 %	667
SNMOT-118	25,06 %	74,94 %	427

Los repartos obtenidos varían considerablemente de una secuencia a otra , lo que es consistente con tratarse de tramos cortos de partido en los que un equipo puede dominar claramente la posesión del balón durante una jugada concreta sin que eso sea representativo de la posesión a lo largo de todo el partido. Como se documentó en el diseño de esta métrica (sección 5.5.4), estos porcentajes deben leerse como lo que son, una aproximación por proximidad espacial frame a frame y no como una estadística oficial de partido.

7.6. Limitaciones de los resultados

Al igual que en el resto de esta memoria, conviene cerrar este capítulo resumiendo las limitaciones que afectan a la validez de los resultados presentados:

- **HOTA(0) frente a HOTA integrada:** el estudio de hiperparámetros completo se reporta en términos de HOTA(0), no de la métrica integrada más robusta usada en la comparativa principal . Recalcular el estudio completo con HOTA integrada queda como trabajo futuro.
- **Configuraciones óptimas no recombinadas:** no se ha vuelto a evaluar la comparativa de HOTA integrada de la sección 7.3 usando las configuraciones óptimas de hiperparámetros de cada algoritmo, solo sus valores por defecto.
- **Clasificación de equipos sin árbitro:** los porcentajes de posesión y los mapas de calor por equipo incluyen al árbitro clasificado incorrectamente en uno de los dos equipos, lo que introduce un sesgo pequeño en ambos resultados.
- **Homografía real limitada a dos tramos cortos:** los resultados de distancia y velocidad se basan en 12-15 segundos de vídeo de dos secuencias concretas, no en partidos completos.
- **Posesión como heurística, no como estadística oficial:** los porcentajes de la Tabla 7.4 miden proximidad espacial al balón, no posesión táctica real, y se calculan sobre secuencias muy cortas.
- **Detección del balón por criterio geométrico:** cualquier resultado que dependa de la localización del balón hereda las limitaciones del filtro geométrico frente a un detector entrenado.

8. Manual de despliegue

Este capítulo describe cómo instalar, configurar y ejecutar el sistema, qué artefactos produce cada script y cómo interpretar los resultados. Este capítulo es una guía práctica de uso paso a paso, pensada para que cualquier persona con el dataset descargado pueda reproducir los resultados de esta memoria.

8.1. Requisitos previos

La Tabla 8.1 resume el software necesario para ejecutar tanto el pipeline offline en Python como la aplicación web. El proyecto se ha desarrollado y probado en Windows y macOS (sección 3.6), sin dependencias específicas de sistema operativo más allá de las propias de Python y Node.js.

Tabla 8.1: Requisitos previos para desplegar el sistema completo.

Requisito	Detalle
Python	3.11: versiones más recientes de Python no son compatibles con las versiones de las librerías usadas en este proyecto.
Node.js	18 o superior, con <code>npm</code> incluido (necesario solo para la aplicación web).
Git	Cualquier versión reciente, para clonar el repositorio.
Espacio en disco	Al menos 60 GB libres: el conjunto completo de la tarea tracking de SoccerNet (<i>splits</i> train, test y challenge) ocupa unos 55 GB una vez descomprimido.
Conexión a internet	Estable y sin límite de datos ajustado durante la descarga inicial del dataset, dado su tamaño.

No se requiere GPU: los algoritmos de tracking usados (ByteTrack, OC-SORT) son clásicos y todo el pipeline se ha ejecutado sobre CPU sin que ello supusiera una limitación relevante (sección 3.6).

8.2. Instalación

8.2.1. Obtención del código

El código fuente está alojado en el repositorio de GitHub del proyecto:

```
1 git clone https://github.com/cgaleam/Soccernet-Tracking-Analysis.git
2 cd Soccernet-Tracking-Analysis
```

Todas las rutas relativas del pipeline (`../SoccerNet/...`, usadas de forma consistente en los scripts de `src/`) asumen que el dataset se descarga como una carpeta hermana del repositorio, no dentro de él.

8.2.2. Entorno Python

El proyecto usa un entorno virtual estándar de Python, no conda (problema P2, Tabla 6.2):

```
1 python3.11 -m venv venv_tfg
2
3 # macOS / Linux
4 source venv_tfg/bin/activate
5
6 # Windows (PowerShell)
7 venv_tfg\Scripts\Activate.ps1
8
9 pip install -r requirements.txt
```

`requirements.txt` fija las versiones exactas de todas las dependencias, incluyendo `numpy==2.4.4`, `boxmot==10.0.43` y `opencv-python==4.13.0.92`. Existe una inconsistencia conocida y ya documentada: `trackeval` no aparece listado explícitamente en `requirements.txt` aunque el resto del pipeline lo importa directamente; si `pip install -r requirements.txt` no lo instala como dependencia transitiva de otro paquete, instalarlo aparte con `pip install trackeval`.

8.2.3. Descarga del dataset SoccerNet

La descarga se automatiza con el SDK oficial de SoccerNet, ya incluido en `requirements.txt`:

```
1 python src/utils/soccernet_download.py
```

Este script descarga los *splits* train, test y challenge de las tareas `tracking` y `tracking-2023` en una carpeta `SoccerNet/` creada como hermana del repositorio, reproduciendo la estructura `SoccerNet/tracking/{train,test,challenge}/SNMOT-XXX/` que consumen el resto de scripts del proyecto. A diferencia de otras tareas del ecosistema SoccerNet que exigen solicitar acceso mediante un formulario NDA (por ejemplo, los vídeos completos de partido), la tarea `tracking` se descarga con una contraseña genérica de acceso público, incluida en el propio SDK, sin gestión adicional por parte del usuario. En la ejecución realizada durante este proyecto, la tarea `tracking-2023` no llegó a

descargar ningún dato (el propio SDK informa de que ese *split* concreto no estaba disponible en el servidor en ese momento), sin que ello afecte al resto del pipeline, que usa exclusivamente la tarea `tracking`.

8.3. Estructura del repositorio

La Tabla 6.3 detalla la responsabilidad de cada paquete de `src/`. A efectos prácticos de navegación, el árbol de directorios relevante tras completar la instalación y una primera ejecución completa es el siguiente:

1	Soccernet-Tracking-Analysis/	
2	src/	
3	visualizacion/	# scripts exploratorios iniciales
4	tracking/	# ByteTrack, OC-SORT, ejecucion batch, export de video
5	evaluation/	# seqmaps + evaluacion con TrackEval
6	hyperparameter_study/	
7	metadata/	# equipos, heatmaps, homografia, posesion
8	web_export/	# metadatos de OC-SORT + paquete de datos para la web
9	utils/	
10	frontend/	# aplicacion web
11	results_train/, results_test/	# .txt de tracking por secuencia
12	results_hyperparam/	# resultados del barrido + configuraciones optimas
13	evaluation_train/, evaluation_test/	# salida de TrackEval
14	evaluation_hyperparam/	# resumen + graficas del estudio de hiperparametros
15	metadata_output/	# equipos, heatmaps, homografia, posesion
16	videos/	# videos anotados exportados
17	memoria/	# este documento
18	requirements.txt	
19	venv_tfg/	# entorno virtual

`results_train/`, `evaluation_train/` y el resto de carpetas de salida no se incluyen en el repositorio: las genera cada script la primera vez que se ejecuta.

8.4. Ejecución del pipeline de tracking

Los scripts de `src/tracking/` cubren dos casos de uso distintos: explorar una única secuencia con un algoritmo concreto, o procesar un *split* completo con ambos algoritmos (sección 6.3).

Para explorar una única secuencia, `bytetracker_alg.py` y `sort_alg.py` no aceptan argumentos de línea de comandos: la secuencia a procesar se fija editando la variable `base_path` al principio del fichero antes de ejecutarlo.

```
1 # Editar antes de ejecutar: base_path = "../SoccerNet/tracking/train/
  SNMOT-XXX/"
2 python src/tracking/bytetracker_alg.py
3 python src/tracking/sort_alg.py
```

Para reproducir los resultados de esta memoria, el flujo real es la ejecución batch sobre el *split* completo, que sí recorre todas las secuencias automáticamente:

```
1 python src/tracking/run_all_train.py    # -> results_train/{bytetracker,
  ocsort}/
2 python src/tracking/run_all_test.py     # -> results_test/{bytetracker,
  ocsort}/
```

Cada ejecución tarda del orden de varias horas sobre CPU, al procesar las 57 secuencias de train o las 49 de test con ambos algoritmos frame a frame. Para exportar un vídeo anotado de una secuencia concreta (por defecto, SNMOT-076 de train):

```
1 python src/tracking/export/export_bytetracker.py    # -> videos/SNMOT-076
  _bytetracker.mp4
2 python src/tracking/export/export_sort.py           # -> videos/SNMOT-076
  _ocsort.mp4
```

8.5. Ejecución de la evaluación

TrackEval necesita un fichero *seqmap* que le indique qué secuencias evaluar, que debe generarse antes de la evaluación propiamente dicha:

```
1 python src/evaluation/seqmaps_creation_train.py
2 python src/evaluation/seqmaps_creation_test.py
3 python src/evaluation/evaluate_train.py    # -> evaluation_train/
4 python src/evaluation/evaluate_test.py    # -> evaluation_test/
```

Cada carpeta de salida contiene, por algoritmo y secuencia, un `pedestrian_summary.txt` con las columnas de HOTA, HOTA(0), DetA, AssA, MOTA e IDF1 usadas en la comparativa del capítulo 7.

8.6. Ejecución del estudio de hiperparámetros

```
1 python src/hyperparameter_study/hyperparam_bytetracker.py # ->
  results_hyperparam/bytetracker/
2 python src/hyperparameter_study/hyperparam_ocsort.py      # ->
  results_hyperparam/ocsort/
```

```

3 python src/hyperparameter_study/summarize_hyperparam.py # ->
  evaluation_hyperparam/summary_table.csv y /plots/
4 python src/hyperparameter_study/bytetracker_optimal.py # ->
  results_hyperparam/bytetracker/bytetracker_optimal/
5 python src/hyperparameter_study/ocsort_optimal.py # ->
  results_hyperparam/ocsort_optimal/

```

Los dos primeros scripts varían un hiperparámetro a la vez sobre el *split* de test, manteniendo el resto en su valor base (sección 6.5); cada barrido completo tarda igualmente del orden de horas. `summarize_hyperparam.py` debe ejecutarse después de ambos barridos, ya que lee sus resultados y produce la tabla y las gráficas comparativas reutilizadas en el capítulo 7. Los dos últimos scripts no forman parte del barrido: ejecutan y evalúan cada algoritmo una única vez con la configuración óptima ya determinada, y son los que alimentan tanto el pipeline de metadatos como la aplicación web.

8.7. Ejecución del pipeline de metadatos

Todos los scripts de `src/metadata/` parten de `results_hyperparam/bytetracker/bytetracker_optimal/` y deben ejecutarse en este orden, ya que la clasificación de equipos es una dependencia de los mapas de calor por equipo y de la posesión:

```

1 python src/metadata/team_classification.py # -> metadata_output/
  teams/*_teams_final.csv
2 python src/metadata/team_visualization.py # -> metadata_output/
  teams/*_frame*.png
3 python src/metadata/heatmap_field.py # -> metadata_output/
  heatmaps/*_heatmap_field.png
4 python src/metadata/heatmap_by_team.py # -> metadata_output/
  heatmaps/*_heatmap_by_team.png
5 python src/metadata/distance_homography_demo_116.py
6 python src/metadata/distance_homography_demo_118.py
7 python src/metadata/heatmap_homography_demo_116.py
8 python src/metadata/heatmap_homography_demo_118.py
9 python src/metadata/ball_possession.py # -> metadata_output/
  possession/

```

`general_heatmap.py` es la primera versión del mapa de calor general y se conserva en el repositorio a título documental, pero no forma parte del flujo actual: `heatmap_field.py` la sustituye por completo. `distance_homography_demo_*.py` y `heatmap_homography_demo_*.py` solo existen para SNMOT-116 y SNMOT-118, las dos secuencias con calibración de homografía manual.

Para generar el mismo conjunto de metadatos sobre la configuración óptima de OC-SORT, necesario para el selector de algoritmo de la aplicación web, no para el

pipeline principal, que solo usa ByteTrack, el paquete `src/web_export/` reproduce esta misma lógica parametrizada por algoritmo en lugar de duplicar cada script.

8.8. Manual de la aplicación web

8.8.1. Generación del paquete de datos

La aplicación web no recalcula nada: consume un paquete de artefactos estáticos que debe generarse antes de arrancarla, una vez completados los pasos anteriores para ambos algoritmos:

```
1 python -m src.web_export.export_bundle
2 # -> frontend/public/data/{videos,images}/, sequences.json,
   global_metrics.json
```

Este paso reescribe por completo la carpeta `frontend/public/data/` y debe repetirse cada vez que cambien los resultados de tracking, evaluación o metadatos que consume.

8.8.2. Instalación y ejecución del frontend

```
1 cd frontend
2 npm install
3
4 npm run dev      # servidor de desarrollo, http://localhost:5173
5 npm run build    # build estatico
6 npm run preview  # sirve el build de dist/ en local para verificarlo
```

`npm run dev` recarga automáticamente al editar el código de `frontend/src/`, pero no vigila `frontend/public/data/`: si se regenera el paquete de datos con `export_bundle.py` mientras el servidor de desarrollo está en marcha, hay que reiniciarlo para que recoja los ficheros nuevos. El build de producción (`npm run build`) genera un sitio completamente estático en `frontend/dist/`, con rutas relativas (`base: './` en `vite.config.ts`), desplegable en cualquier alojamiento estático o servible directamente en local.

8.8.3. Uso de la interfaz

La aplicación se organiza en dos pestañas (sección [5.7.2](#)). **Comparativa de algoritmos** muestra, sin necesidad de seleccionar nada, la tabla de métricas HOTA/HOTA(0)/DetA/AssA/MOTA/IDF1 por *split*, la configuración óptima de cada algoritmo y las gráficas del estudio de hiperparámetros. **Explorador de secuencias** requiere elegir una secuencia y un algoritmo mediante los selectores de

chips; el resto de la pestaña se actualiza automáticamente con los datos de esa combinación, mostrando una nota de «no disponible» en los paneles sin datos.

Las Figuras 8.1 a 8.4 son capturas reales de la aplicación en ejecución (`npm run dev`) sobre el paquete de datos descrito en la sección 6.7.2.



SoccerNet Tracking Analysis

Demostración de resultados del pipeline offline de tracking multiobjeto sobre SoccerNet-Tracking.

Comparativa de algoritmos

Explorador de secuencias

HOTA frente a HOTA(0)

HOTA(0) es HOTA evaluado en un único umbral de IoU (0.5); HOTA integra el promedio en el rango 0.05–0.95. La configuración óptima de hiperparámetros de este proyecto se seleccionó con HOTA(0), lo que invierte el orden entre algoritmos respecto a HOTA integrado: bajo HOTA(0) ByteTrack aventaja a OC-SORT en test, pero bajo HOTA integrado es OC-SORT quien gana en ambos splits, porque OC-SORT reutiliza las detecciones sin interpolarlas (LocA/DetPr casi perfectos) mientras que ByteTrack sí interpola en su segunda ronda de asociación, lo que le penaliza en los umbrales de IoU más estrictos.

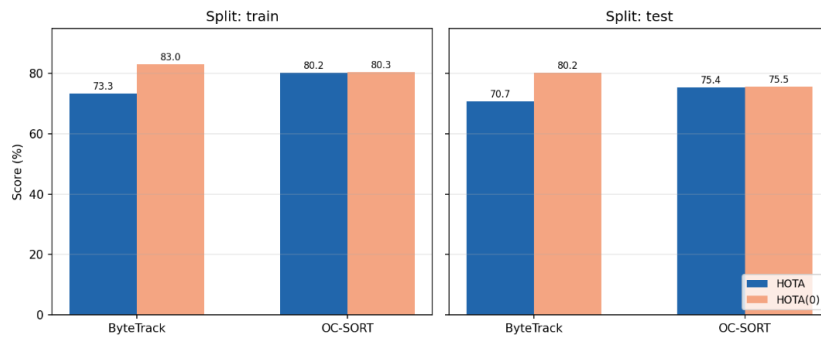
Split train

Algoritmo	HOTA	HOTA(0)	DetA	AssA	MOTA	IDF1	IDSW
ByteTrack	73.27	82.97	81.58	65.86	92.15	77.99	3866
OC-SORT	80.23	80.34	92.11	69.88	92.01	75.34	2205

Split test

Algoritmo	HOTA	HOTA(0)	DetA	AssA	MOTA	IDF1	IDSW
ByteTrack	70.73	80.20	80.54	62.18	90.78	74.36	5561
OC-SORT	75.43	75.51	87.23	65.22	87.01	71.19	2046

HOTA integrada vs HOTA(0) ($\alpha=0.5$) — ByteTrack vs OC-SORT



Configuración óptima de hiperparámetros

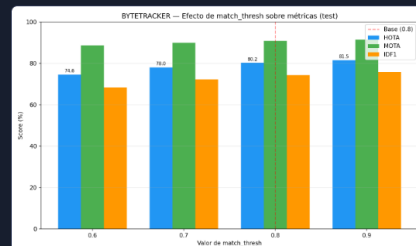
Aquí se muestra la mejor combinación encontrada en el estudio de hiperparámetros sobre el conjunto de split de test, configuración usada para generar todos los resultados del explorador de secuencias.

Algoritmo	Parámetros	HOTA(0)	MOTA	IDF1	IDSW
ByteTrack	match_thresh=0.9, track_buffer=10, track_thresh=0.45	81.43	91.41	75.65	3625
OC-SORT	iou_threshold=0.1, max_age=10, min_hits=1, det_thresh=0.3	80.92	91.55	75.93	2681

Estudio de hiperparámetros

Estudio que muestra el efecto de variar cada hiperparámetro por separado sobre HOTA(0) en el split de test, manteniendo el resto en su valor óptimo.

ByteTrack



OC-SORT

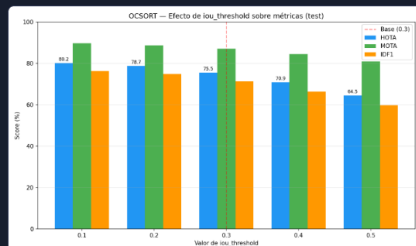


Figura 8.1: Pestaña «Comparativa de algoritmos».



Figura 8.2: Pestaña «Explorador de secuencias» con SNMOT-116 y ByteTrack.





Figura 8.4: SNMOT-118 con ByteTrack.

8.9. Problemas frecuentes

La Tabla 8.2 recoge, de forma complementaria, los problemas más probables al desplegar el sistema desde cero siguiendo este manual, con su causa y solución.

Tabla 8.2: Problemas frecuentes al desplegar el sistema y su solución.

Síntoma	Causa	Solución
FileNotFoundError al leer secuencias del dataset	El dataset no está en ../SoccerNet relativo al repositorio	Clonar el repositorio y descargar el dataset como carpetas hermanas (sección 8.2.3)
Un script de src/tracking/ o src/metadata/ procesa siempre la misma secuencia	La secuencia está hardcodeada en una variable al principio del fichero (base_path, SEQUENCE)	Editar esa variable antes de ejecutar, o usar los scripts run_all_*.py para procesar el split completo
ModuleNotFoundError: trackeval al evaluar	trackeval no quedó instalado como dependencia transitiva	pip install trackeval (sección 8.2.2)
heatmap_by_team.py o ball_possesion.py fallan o dan resultados vacíos	No se ejecutó antes team_classification.py para ese algoritmo	Respetar el orden de ejecución de la sección 8.7
Los vídeos generados por export_bundle.py no se reproducen en el navegador	Fourcc de OpenCV no soportado por navegadores (problema P17)	Ya corregido en el código del proyecto (fourcc avc1); si se modifica el script, mantener ese fourcc
La pestaña Explorador de secuencias no refleja cambios recientes en los datos	npm run dev no vigila frontend/public/data/	Reiniciar el servidor de desarrollo tras regenerar el paquete de datos

9. Uso de inteligencia artificial

Durante el desarrollo de este Trabajo de Fin de Grado se utilizaron herramientas de inteligencia artificial generativa como apoyo al desarrollo de software, al análisis del código existente y a la redacción de esta memoria. Este capítulo detalla en qué medida y de qué forma se han utilizado, siguiendo el mismo criterio de transparencia aplicado al resto del documento. El uso ha sido sobre distintas áreas del proyecto. Las decisiones de diseño y de desarrollo han sido siempre del autor y del tutor mediante su orientación. Sin embargo, las herramientas de IA han sido un apoyo en la redacción de código y resolución de dudas de implementación, así como en la redacción de esta memoria.

9.1. Herramientas

La herramienta empleada ha sido **Claude Code**.

Conviene distinguir dos usos claramente diferenciados a lo largo del proyecto, que se detallan en las secciones siguientes:

- Como asistente de desarrollo de software, en la construcción de la aplicación web y revisor de scripts del pipeline.
- Como revisor y en casos redactor partes de esta memoria, a partir de una base del autor y de las notas que el autor ha tomado durante el desarrollo del proyecto y del análisis directo del código del proyecto.

No se ha usado ninguna otra herramienta de IA generativa en ningún punto del proyecto.

9.2. Uso en el desarrollo de software

El pipeline offline en Python es trabajo previo del autor, desarrollado de forma independiente antes de emplear Claude Code en este proyecto. En esa parte, el papel de la herramienta se ha limitado a leer, analizar y corregir o editar el código ya existente para poder documentarlo con precisión en esta memoria (sección 9.3), no a escribirlo.

El uso de Claude Code como herramienta de desarrollo activo se concentra en la aplicación web (sección 6.7), construida durante este proyecto con su ayuda:

- El paquete `src/web_export/` `–compute_metadata.py` y `export_bundle.py` se escribió con Claude Code a partir de una instrucción explícita: reutilizar la lógica ya validada de `src/metadata/` sin modificar ni duplicar esos scripts, parametrizándola por algoritmo. La verificación de que la lógica reproducida era

efectivamente equivalente se hizo contrastando el código generado contra los scripts originales.

- Todo el código de base de `frontend/` se generó igualmente con Claude Code, a partir de decisiones de diseño explícitas del autor: el *stack* tecnológico (React, Vite, TypeScript), el alcance del MVP fiel al boceto ya planificado en el capítulo 5, y una ronda posterior de ajustes visuales concretos (color de acento, cabecera con logotipo, alineación de tablas) solicitados de forma iterativa tras revisar la aplicación en ejecución.
- Antes de implementar la aplicación web se detectó, durante la fase de planificación con la herramienta, un desajuste entre el diseño original y los datos realmente disponibles. Se plantearon al autor dos alternativas.
- Durante la implementación se encontraron y corrigieron con la herramienta algunos problemas de implementación, como la incompatibilidad del `fourcc mp4v` de OpenCV con la reproducción de vídeo en navegador, ya documentada como P17 en la Tabla 6.2.

En todos los casos, el código generado se ha ejecutado y verificado antes de darlo por válido, no se ha incorporado código sin comprobar que efectivamente hace lo que se le pidió.

9.3. Uso en documentación

El uso de Claude Code en este proyecto ha sido en gran medida en la redacción y revisión de esta propia memoria.

La herramienta ayudó en consultar ideas y alternativas para la estructura, la redacción de secciones y subsecciones, la revisión de estilo y la corrección de errores tipográficos y de sintaxis o reestructurar texto a formato LaTeX. En ningún caso se ha usado para generar contenido técnico o resultados que no hayan sido verificados por el autor.

Todo con ayuda de una base de memoria que el autor fue construyendo poco a poco y unos documentos de notas que el autor fue tomando durante el desarrollo del proyecto con todo detalle y del análisis directo del código del proyecto.

- El hallazgo más relevante de todos los detectados durante esta redacción es la distinción entre HOTA(0) y HOTA integrada : tanto las notas del autor como el propio script `summarize_hyperparam.py` habían usado sistemáticamente la columna HOTA(0) de `TrackEval` bajo el nombre «HOTA», lo que llevaba a una conclusión que se invierte con claridad al recalcular la comparativa con la métrica HOTA integrada estándar. Este hallazgo no estaba en las notas ni memoria originales: surgió de contrastar el código de evaluación con la documentación oficial de `TrackEval` al consultar con Claude la revisión de la parte de métricas en la documentación.

9.4. Límites del uso de inteligencia artificial

Todo el contenido generado con Claude Code ha sido revisado y dirigido por el autor en cada paso, no aceptado de forma automática:

- Los cambios de alcance o de diseño con más de una alternativa razonable, por ejemplo, la decisión de generar metadatos de OC-SORT en vez de recortar el selector de algoritmo de la aplicación web, se plantearon explícitamente al autor como una decisión suya, no se resolvieron de forma autónoma por la herramienta.
- Todos los datos numéricos, tablas y resultados presentados en esta memoria proceden de ficheros realmente generados por el pipeline del proyecto (`evaluation_train/`, `evaluation_test/`, `evaluation_hyperparam/`, `metadata_output/`), no de estimaciones ni de contenido inventado por la IA.
- Todo el código generado se ha ejecutado y su salida se ha inspeccionado antes de incorporarla a la memoria o de darla por válida (sección 9.2); ningún resultado cuantitativo de este documento se ha aceptado sin volver a comprobar que procede realmente de la ejecución del sistema descrito.

En conjunto, el criterio seguido ha sido que Claude Code actúe como herramienta de apoyo en la redacción y de desarrollo bajo dirección explícita del autor en algunas decisiones, y que ninguna afirmación de esta memoria se dé por buena sin haberla contrastado contra una fuente verificable del proyecto.

10. Conclusiones y trabajo futuro

Este capítulo cierra la memoria repasando en qué medida se han alcanzado los objetivos planteados en el capítulo 1, sintetizando los resultados más relevantes del proyecto, y proponiendo las líneas de trabajo futuro, algunas de las cuales se han ido señalando a lo largo del documento.

10.1. Cumplimiento de los objetivos

Los once objetivos específicos planteados se han cumplido en su totalidad.

OE-01 a OE-05 el núcleo de la comparación de algoritmos: estudiar el dataset, implementar ByteTrack y OC-SORT, ejecutarlos sobre la totalidad de train y test, evaluarlos con las métricas estándar del campo y realizar un estudio sistemático de hiperparámetros, se han cumplido de forma directa y están documentados con resultados reales. OE-06 a OE-09 se han cumplido igualmente, con la salvedad, ya señalada en su momento, de que OE-08 se resolvió como prueba de concepto sobre dos tramos concretos de cámara estable y no como una solución general al movimiento continuo de cámara del resto del dataset. OE-10, documentar de forma honesta las limitaciones y los enfoques descartados, no es un objetivo que se cumpla en un capítulo concreto sino un criterio aplicado a todo el documento, desde la tabla de problemas de entorno del capítulo 6 hasta el propio capítulo 9. OE-11, desarrollar una aplicación web como demostrador visual e interactivo, se ha completado como un MVP funcional fiel al diseño planificado en el capítulo 5, aunque limitado a las tres secuencias de demostración usadas a lo largo de la memoria y no al dataset completo.

10.2. Resultados principales

Este proyecto como resultado ha dado una comparativa entre dos algoritmos de tracking: ByteTrack y OC-SORT. El resultado más relevante de este proyecto no es una cifra aislada, sino la comparativa entre ByteTrack y OC-SORT depende de forma decisiva de qué variante de HOTA se use para medirla. Bajo HOTA(0) ByteTrack aventaja a OC-SORT en el conjunto de test; bajo HOTA integrada ese resultado se invierte con claridad en ambos conjuntos, porque OC-SORT reutiliza las detecciones del *dataset* sin modificarlas mientras que ByteTrack las interpola en su segunda ronda de asociación, lo que penaliza a ByteTrack en los umbrales más estrictos que HOTA(0) no llega a evaluar. Que dos formas igual de legítimas de medir el mismo sistema lleven a conclusiones opuestas sobre qué algoritmo es mejor no es un detalle menor de este proyecto, es uno de sus resultados.

Sobre la configuración óptima resultante, el pipeline de metadatos ha demostrado ser aplicable sobre secuencias reales de SoccerNet, con resultados numéricos

consistentes entre sí: por ejemplo, el porcentaje de posesión del equipo dominante en SNMOT-116 se mantiene prácticamente igual pese a partir de asociaciones de tracking distintas. Finalmente, la aplicación web ha demostrado que ese mismo pipeline puede exponerse como una interfaz interactiva y visual para el usuario.

10.3. Limitaciones

Las limitaciones de este proyecto se han ido documentando explícitamente en los capítulos correspondientes del proyecto, pero se resumen aquí de forma conjunta:

- El estudio de hiperparámetros completo se reporta en HOTA(0), no en la métrica integrada usada en la comparativa principal (sección 7.6).
- La comparativa de HOTA integrada usa los valores por defecto de cada algoritmo, no sus configuraciones óptimas de hiperparámetros (sección 7.6).
- La clasificación de equipos por K-means con $k = 2$ asigna incorrectamente al árbitro a uno de los dos equipos (sección 2.8).
- La homografía real se ha limitado a dos tramos cortos de cámara estable, no al dataset completo (sección 6.6.3).
- La posesión de balón es una aproximación por proximidad espacial, no una estadística táctica real (sección 5.5.4).
- La detección del balón se basa en un filtro geométrico sobre las detecciones del *dataset*, no en un detector entrenado específicamente para esa clase.
- La aplicación web cubre únicamente las tres secuencias de demostración usadas en esta memoria, no el dataset completo para los metadatos.

10.4. Trabajo futuro

De las limitaciones anteriores, y de los enfoques explorados y descartados a lo largo del proyecto, se derivan directamente las líneas de continuación más naturales de este trabajo:

- **Recalcular el estudio de hiperparámetros con HOTA integrada:** repetir el barrido completo de ambos algoritmos con la métrica integrada en vez de HOTA(0), y evaluar si la configuración óptima resultante coincide con la ya encontrada.
- **Evaluar las configuraciones óptimas con HOTA integrada:** confirmar si la ventaja de OC-SORT sobre ByteTrack observada con los valores por defecto de cada algoritmo se mantiene, se amplía o se reduce al usar sus configuraciones óptimas de hiperparámetros.
- **Incorporar algoritmos con re-identificación visual (Re-ID),** como DeepSORT, aunque previsiblemente con un margen de mejora menor que en

otros dominios de MOT, dada la escasa discriminabilidad visual entre compañeros de un mismo equipo.

- **Clasificación de equipos con un tercer grupo para el árbitro** ($k = 3$, combinado con una heurística sobre el tamaño de cada grupo), para eliminar el sesgo que introduce actualmente en los mapas de calor por equipo y en la posesión de balón.
- **Homografía generalizada y dinámica**, que recalcula la matriz de proyección frame a frame para tramos con movimiento de cámara continuo, en lugar de limitarse a tramos concretos con cámara estática.
- **Extender la aplicación web al resto del dataset**, generando el paquete de metadatos y vídeos anotados para más secuencias o para su totalidad en lugar de las tres actuales, y explorando un detector de balón entrenado que sustituya al filtro geométrico actual como base de una posesión más fiable.
- **Tracking sobre un vídeo subido por el usuario**: permitir que el propio usuario suba un vídeo de fútbol arbitrario a la aplicación web y ejecutar el pipeline de tracking sobre él, en lugar de limitarse a las secuencias de demostración ya procesadas. A diferencia del resto de este proyecto, que parte de detecciones ya calculadas por SoccerNet (sección 1.3), esta vía exigiría incorporar primero un detector de objetos propio, ya que un vídeo arbitrario no viene con `det.txt`. Es técnicamente viable con el enfoque ya desarrollado, pero queda como línea futura.
- **Participación real en el *leaderboard* oficial de SoccerNet**: el dataset incluye un tercer split, *challenge*, sin `gt.txt`, pensado precisamente para enviar resultados al servidor oficial de evaluación y obtener una comparación pública y directa con otros equipos, en la línea de los ya publicados en ediciones anteriores del reto [28], ofreciendo esta vez una cifra oficial en lugar de la referencia de contexto de la sección 2.10.

10.5. Reflexión final

Este proyecto partió de una pregunta relativamente acotada y ha terminado abarcando un sistema completo, desde el seguimiento de jugadores hasta la extracción de metadatos deportivos interpretables y su exposición en una aplicación web. La constatación de que la propia pregunta «qué algoritmo es mejor» depende de una decisión metodológica, la métrica elegida para responderla, que resultó menos trivial de lo que parecía al principio del proyecto.

Bibliografía

- [1] Plan de Recuperación, Transformación y Resiliencia, Gobierno de España. Qué es la inteligencia artificial, 2023. URL <https://planderecuperacion.gob.es/noticias/que-es-inteligencia-artificial-ia-prtr>.
- [2] IBM. ¿qué es el machine learning (ml) o aprendizaje automático?, 2026. URL <https://www.ibm.com/es-es/think/topics/machine-learning>.
- [3] IBM. ¿qué es el deep learning?, 2026. URL <https://www.ibm.com/es-es/think/topics/deep-learning>.
- [4] IBM. ¿qué es la visión artificial?, 2026. URL <https://www.ibm.com/es-es/think/topics/computer-vision>.
- [5] Coursera. ¿qué es el análisis de datos? (con ejemplos), 2026. URL <https://www.coursera.org/mx/articles/what-is-data-analysis-with-examples>.
- [6] IBM. ¿qué es un conjunto de datos?, 2026. URL <https://www.ibm.com/es-es/think/topics/dataset>.
- [7] MOTChallenge. MOTChallenge: Multi-object tracking benchmark, 2026. URL <https://motchallenge.net/>.
- [8] Anthony Cioppa, Silvio Giancola, Adrien Delière, Le Kang, Xin Zhou, Zhiyu Cheng, Bernard Ghanem, and Marc Van Droogenbroeck. SoccerNet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3491–3502, June 2022. URL https://openaccess.thecvf.com/content/CVPR2022W/CVSports/html/Cioppa_SoccerNet-Tracking_Multiple_Object_Tracking_Dataset_and_Benchmark_in_Soccer_Videos_CVPRW_2022_paper.html.
- [9] SoccerNet. SoccerNet: A large-scale dataset for soccer video understanding, 2026. URL <https://www.soccer-net.org/>.
- [10] Ultralytics. Seguimiento de múltiples objetos con ultralytics YOLO, 2026. URL <https://docs.ultralytics.com/es/modes/track>.
- [11] Wikipedia. Filtro de Kalman, 2026. URL https://es.wikipedia.org/wiki/Filtro_de_Kalman.
- [12] Wikipedia. Algoritmo húngaro, 2026. URL https://es.wikipedia.org/wiki/Algoritmo_h%C3%BAngaro.
- [13] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003. URL <https://doi.org/10.1109/ICIP.2016.7533003>.

- [14] Ultralytics. 8 mejores herramientas de seguimiento de objetos: ByteTrack y más, 2026. URL <https://www.ultralytics.com/es/blog/top-8-open-source-object-tracking-tools-and-algorithms>.
- [15] Jinkun Cao, Jiangmiao Pang, Xinshuo Weng, Rawal Khirodkar, and Kris Kitani. Observation-centric SORT: Rethinking SORT for robust multi-object tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9686–9696, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Cao_Observation-Centric_SORT_Rethinking_SORT_for_Robust_Multi-Object_Tracking_CVPR_2023_paper.html.
- [16] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *Computer Vision – ECCV 2022*, pages 1–21. Springer, 2022. doi: 10.1007/978-3-031-20047-2\1. URL https://doi.org/10.1007/978-3-031-20047-2_1.
- [17] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649, 2017. doi: 10.1109/ICIP.2017.8296962. URL <https://doi.org/10.1109/ICIP.2017.8296962>.
- [18] Keni Bernardin and Rainer Stiefelhagen. Evaluating multiple object tracking performance: The CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing*, 2008:1–10, 2008. doi: 10.1155/2008/246309. URL <https://doi.org/10.1155/2008/246309>.
- [19] Jonathon Luiten, Aljosa Osep, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe. HOTA: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021. doi: 10.1007/s11263-020-01375-2. URL <https://doi.org/10.1007/s11263-020-01375-2>.
- [20] Ergys Ristani, Francesco Solera, Roger Zou, Rita Cucchiara, and Carlo Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. In *Computer Vision – ECCV 2016 Workshops*, pages 17–35. Springer, 2016. doi: 10.1007/978-3-319-48881-3\2. URL https://doi.org/10.1007/978-3-319-48881-3_2.
- [21] Jonathon Luiten. How to evaluate tracking with the HOTA metrics, 2021. URL <https://autonomousvision.github.io/hota-metrics/>.
- [22] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. URL https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html.
- [23] Ultralytics. Explora ultralytics YOLOv8, 2026. URL <https://docs.ultralytics.com/es/models/yolov8/>.

- [24] IBM. ¿qué es la agrupación en clústeres k-means?, 2026. URL <https://www.ibm.com/mx-es/think/topics/k-means-clustering>.
- [25] Gate Vidyalay. K-means clustering algorithm — examples, 2026. URL <https://www.gatevidyalay.com/k-means-clustering-algorithm-example/>.
- [26] Vecteezy. Football soccer field illustration, coach table for tactic presentation for players, 2026. URL https://static.vecteezy.com/system/resources/previews/006/796/879/non_2x/football-soccer-field-illustration-coach-table-for-tactic-presentation-for-players.jpg.
- [27] OpenCV. Basic concepts of the homography explained with code, 2026. URL https://docs.opencv.org/4.13.0/d9/dab/tutorial_homography.html.
- [28] Anthony Cioppa, Silvio Giancola, Vladimir Somers, et al. SoccerNet 2023 challenges results, 2023. URL <https://arxiv.org/abs/2309.06006>. arXiv:2309.06006.
- [29] Universidad de Sevilla. Página principal de la escuela técnica superior de ingeniería informática, 2020. URL <https://www.informatica.us.es>.
- [30] Junta de Andalucía, Dirección General de Transformación Digital. Instrucción 2/2020, de 30 de junio, de la dirección general de transformación digital, sobre perfiles y precios de referencia de la contratación de bienes y servicios TIC, 2020. URL <https://www.juntadeandalucia.es/contratacion/document/download?refCode=2021-0000155155&refDoc=2021-0000155155-3>. Tabla de precios de referencia reproducida en la Memoria Complementaria del expediente CONTR2020 864348, Consejería de la Presidencia, Administración Pública e Interior.

Glosario de términos

Este glosario recoge, ordenados alfabéticamente, los términos técnicos y las siglas usadas a lo largo de la memoria que pueden no resultar familiares fuera del ámbito del seguimiento multiobjeto y la visión por computador. Cada capítulo introduce y explica estos conceptos la primera vez que aparece.

AssA (*Association Accuracy*) Componente de HOTA que mide la calidad de la asociación entre detecciones de un mismo objeto a lo largo del tiempo, con independencia de si la detección en sí es correcta (sección 2.6.3).

ByteTrack Algoritmo de *tracking-by-detection* que asocia también las detecciones de baja confianza en una segunda ronda, en vez de descartarlas directamente (sección 2.5.3).

det.txt Fichero de detecciones precomputadas de una secuencia de SoccerNet-Tracking, en formato MOT: una fila por detección, con *frame*, coordenadas de la caja delimitadora y confianza (sección 2.2.1).

DetA (*Detection Accuracy*) Componente de HOTA que mide la calidad de las detecciones por sí solas, sin tener en cuenta si están bien asociadas entre *frames* (sección 2.6.3).

Fourcc Código de cuatro caracteres que identifica el códec de vídeo usado al codificar un fichero con OpenCV; el proyecto usa `avc1` (H.264) en vez de `mp4v` para que los vídeos se reproduzcan en el navegador (problema P17, Tabla 6.2).

Ground truth Anotación de referencia considerada correcta, usada para evaluar un sistema; en SoccerNet-Tracking, el fichero `gt.txt` con las posiciones e identidades reales de cada jugador (sección 2.2.1).

Homografía Transformación geométrica que proyecta puntos de un plano (la imagen de la cámara) a otro plano (el campo real), usada en este proyecto para convertir posiciones en píxeles a metros reales (sección 2.9).

HOTA (*Higher Order Tracking Accuracy*) Métrica de evaluación de *tracking* que combina DetA y AssA, promediada sobre un rango de umbrales de IoU (0.05 a 0.95); en este proyecto se distingue explícitamente de HOTA(0) (sección 2.6.3, sección 6.4.1).

HOTA(0) Notación usada en esta memoria para el valor de HOTA evaluado en un único umbral de IoU (0.5), en vez del promedio integrado de la métrica HOTA estándar; la distinción entre ambas es uno de los hallazgos centrales del proyecto (sección 6.4.1).

Húngaro, algoritmo Algoritmo de optimización combinatoria que resuelve el problema de asignación óptima entre dos conjuntos –en este contexto, qué detección se asocia a qué *track*– en tiempo polinómico (sección 2.4.3).

- IDF1** Métrica de evaluación de *tracking* centrada en la identidad: mide qué proporción de las detecciones se asocian al identificador correcto a lo largo de toda la secuencia (sección 2.6).
- IDSW (*ID Switches*)** Número de veces que un algoritmo de *tracking* cambia por error el identificador asignado a un mismo objeto real (sección 2.6).
- IoU (*Intersection over Union*)** Medida de solapamiento entre dos cajas delimitadoras: el área de su intersección dividida entre el área de su unión, usada tanto para asociar detecciones como para evaluar *trackers* (sección 2.4.2).
- K-means** Algoritmo de agrupamiento no supervisado que reparte un conjunto de puntos en k grupos según su cercanía; usado en este proyecto para clasificar jugadores por equipo a partir del color de su camiseta (sección 2.8).
- Kalman, filtro de** Algoritmo recursivo de estimación que predice el estado futuro de un sistema (aquí, la posición de un jugador) y lo corrige con cada nueva observación, base de los algoritmos de *tracking-by-detection* usados en este proyecto (sección 2.4.1).
- KDE (*Kernel Density Estimation*)** Técnica estadística para estimar la densidad de un conjunto de puntos de forma continua, usada para generar los mapas de calor de posiciones a partir de las trayectorias de *tracking* (sección 6.6).
- MOT (*Multi-Object Tracking*)** Seguimiento multiobjeto: la tarea de detectar y seguir la identidad de múltiples objetos a la vez a lo largo de una secuencia de vídeo, tarea central de este proyecto (sección 2.3).
- MOTA (*Multiple Object Tracking Accuracy*)** Métrica clásica de evaluación de *tracking* que combina falsos positivos, falsos negativos y cambios de identidad en un único valor (sección 2.6).
- MVP (*Minimum Viable Product*)** Versión mínima funcional de un producto, usada aquí para describir el alcance de la aplicación web implementada: fiel al diseño planificado, sin todas las ampliaciones posibles (sección 6.7).
- OC-SORT (*Observation-Centric SORT*)** Variante de SORT que reutiliza las detecciones originales sin interpolarlas y corrige la deriva del filtro de Kalman durante oclusiones prolongadas (sección 2.5.2).
- Re-ID (re-identificación)** Técnica que usa la apariencia visual de un objeto, además de su posición, para reasociar su identidad tras una oclusión; considerada y descartada para este proyecto (sección 2.5.4).
- Seqmap** Fichero que indica a TrackEval qué secuencias evaluar en una ejecución (sección 8.5).
- SORT (*Simple Online and Realtime Tracking*)** Algoritmo clásico de *tracking-by-detection* que combina el filtro de Kalman con el algoritmo húngaro sobre IoU, base de la que parte OC-SORT (sección 2.5.1).
- SPA (*Single Page Application*)** Aplicación web que carga una única página HTML y actualiza su contenido dinámicamente en el navegador, sin recargas completas; arquitectura de la aplicación web de este proyecto (sección 5.7).

SoccerNet-Tracking Dataset público de vídeo de fútbol con detecciones precomputadas y *ground truth*, usado como base de datos de este proyecto (sección [2.2.1](#)).

Split Cada una de las particiones del *dataset* (entrenamiento, test) sobre las que se ejecutan y evalúan los algoritmos por separado (sección [2.2.1](#)).

Tracking-by-detection Enfoque de seguimiento multiobjeto que parte de detecciones ya calculadas en cada *frame* y se centra en asociarlas entre sí, en vez de detectar y seguir a la vez (sección [2.4](#)).

TrackEval Librería de referencia usada en este proyecto para calcular HOTA, MOTA e IDF1 de forma estandarizada (sección [6.4](#)).

Tracker Nombre genérico para un algoritmo de seguimiento multiobjeto; en este proyecto, ByteTrack u OC-SORT.